

KT AIVLE School

Kubernetes와 CICD on AWS



Contents

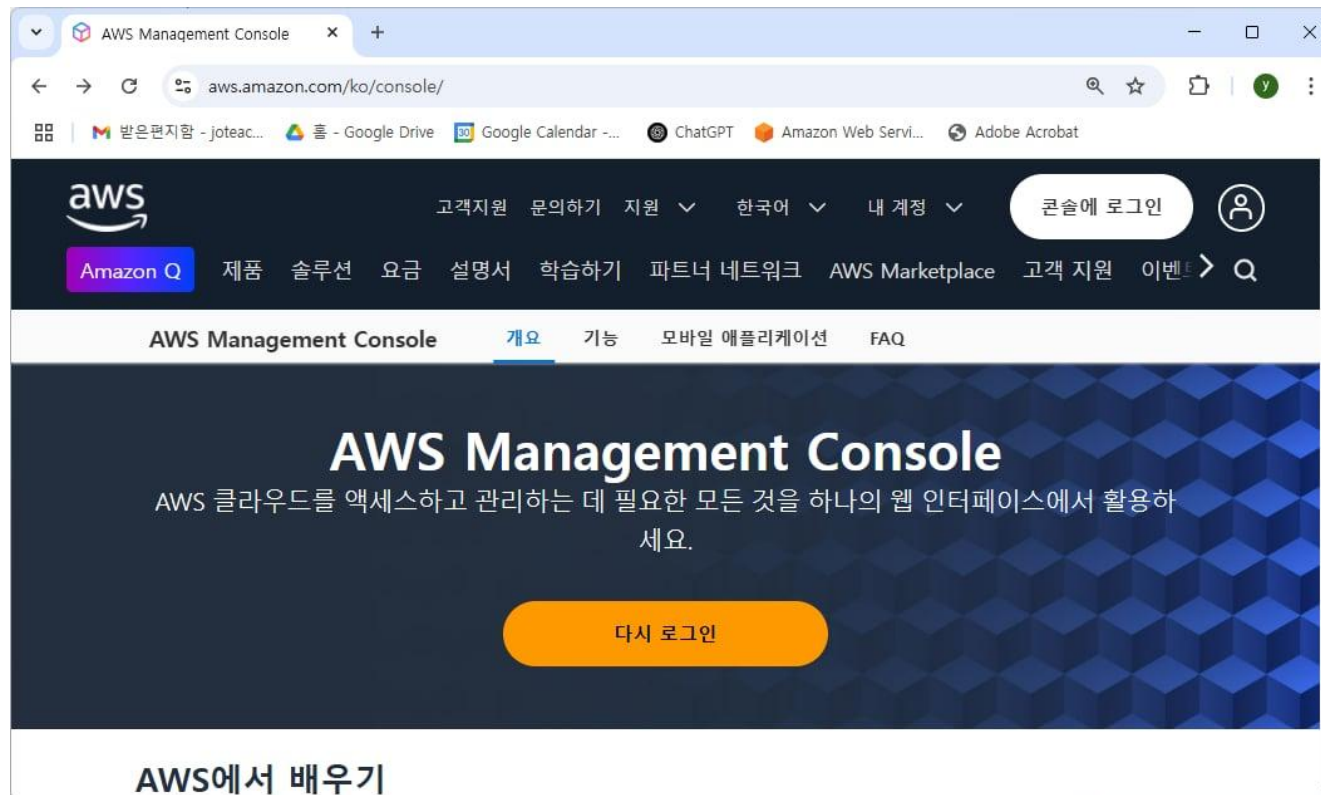
사전 준비	Virtual Machine 생성
Lab 01.	Kubernetes 설치
Lab 02.	Cluster init, join
Lab 03.	Pod
Lab 04.	Replicaset
Lab 05.	Deployment
Lab 06.	Service
Lab 07.	Namespace
Lab 08.	ConfigMap, Secret
Lab 09.	emptyDir, hostPath, PV/PVC
Lab 10.	소스 위치로 git 저장소 활용하기
Lab 11.	AWS CodeBuild로 코드 빌드하기
Lab 12.	AWS CodeDeploy로 EC2 인스턴스에 배포하기

사전 준비.

Virtual Machine 생성

Step 1. AWS 로그인 사이트 접속

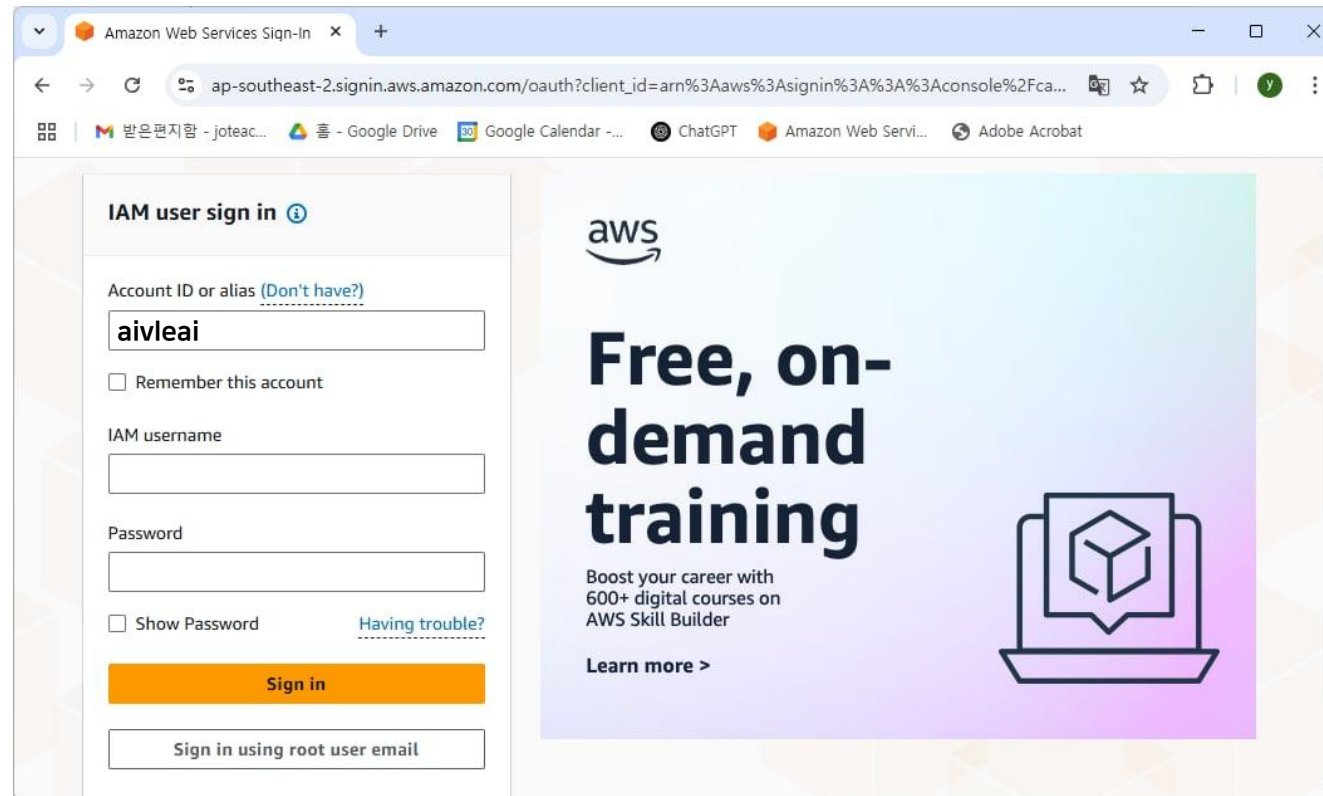
제공된 링크(<https://aws.amazon.com/ko/console/>)로 접속해 "콘솔에 로그인"을 클릭합니다.



Step 2. 계정 로그인

제공받은 IAM username과 Password를 입력하고 Sign in을 클릭합니다.

(Account ID: **aivleai**)



The screenshot shows the AWS IAM user sign-in page in a web browser. The browser tab is titled "Amazon Web Services Sign-In". The address bar shows the URL: `ap-southeast-2.signin.aws.amazon.com/oauth?client_id=arn%3Aaws%3Asign%3A%3A%3Aconsole%2Fca...`. The page has a light blue and white background. On the left, there is a form titled "IAM user sign in" with a help icon. The form contains the following fields and options:

- Account ID or alias** (Don't have?): A text input field containing the value "aivleai".
- ☐ Remember this account
- IAM username**: A text input field.
- Password**: A text input field.
- ☐ Show Password
- [Having trouble?](#)
- Sign in**: An orange button.
- Sign in using root user email**: A button.

On the right side of the page, there is a large promotional banner for "Free, on-demand training". The banner features the AWS logo, the text "Free, on-demand training", and a sub-headline "Boost your career with 600+ digital courses on AWS Skill Builder". Below the text is a "Learn more >" link and an illustration of a laptop with a cube icon on its screen.

Step 3. 리전(Region) 확인

오른쪽 상단에서 선택된 리전을 확인할 수 있습니다. **안내받은 리전으로 선택합니다.**



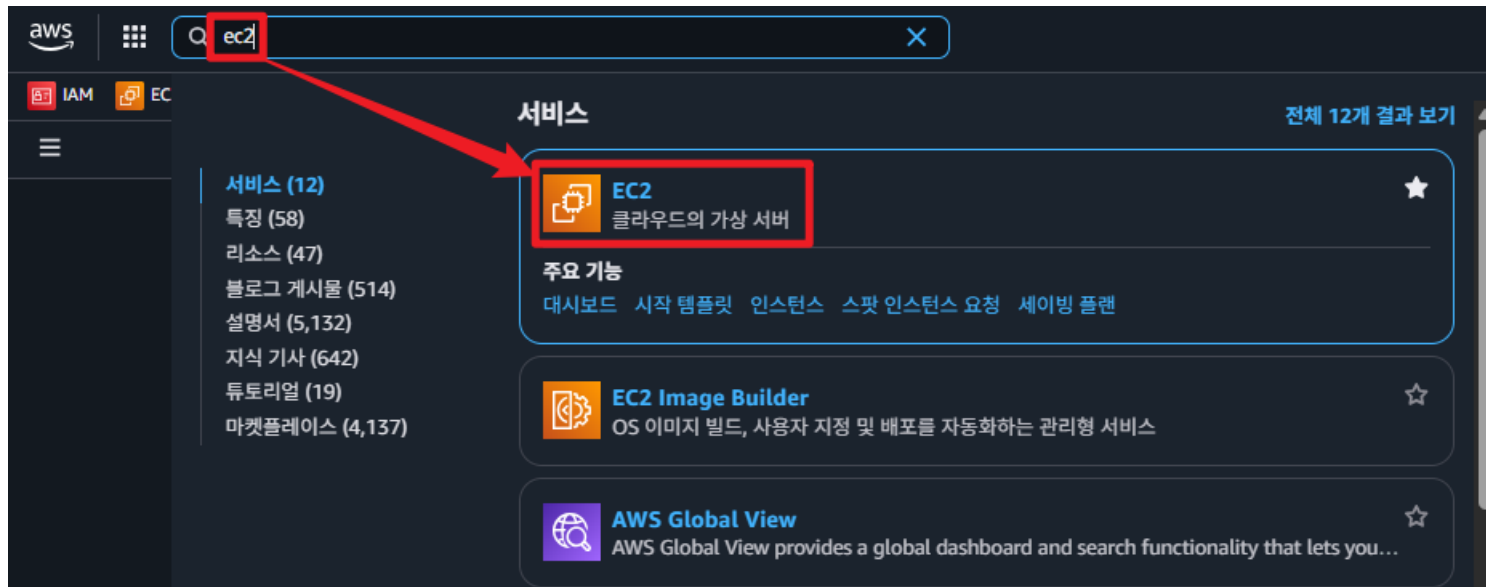
The image shows a screenshot of the AWS Region selection dropdown menu. The menu is open, displaying a list of regions categorized by continent. The 'Asia Pacific (Seoul)' region is highlighted in blue, indicating it is the selected region. The menu also shows the current region 'Asia Pacific (Seoul)' at the top right.

아시아 태평양 (서울) ▲	
미국	
버지니아 북부	us-east-1
오하이오	us-east-2
캘리포니아	us-west-1
오레곤	us-west-2
아시아 태평양	
뭄바이	ap-south-1
오사카	ap-northeast-3
서울	ap-northeast-2
싱가포르	ap-southeast-1
시드니	ap-southeast-2
도쿄	ap-northeast-1

Master Node 생성

Step 1. 인스턴스 시작

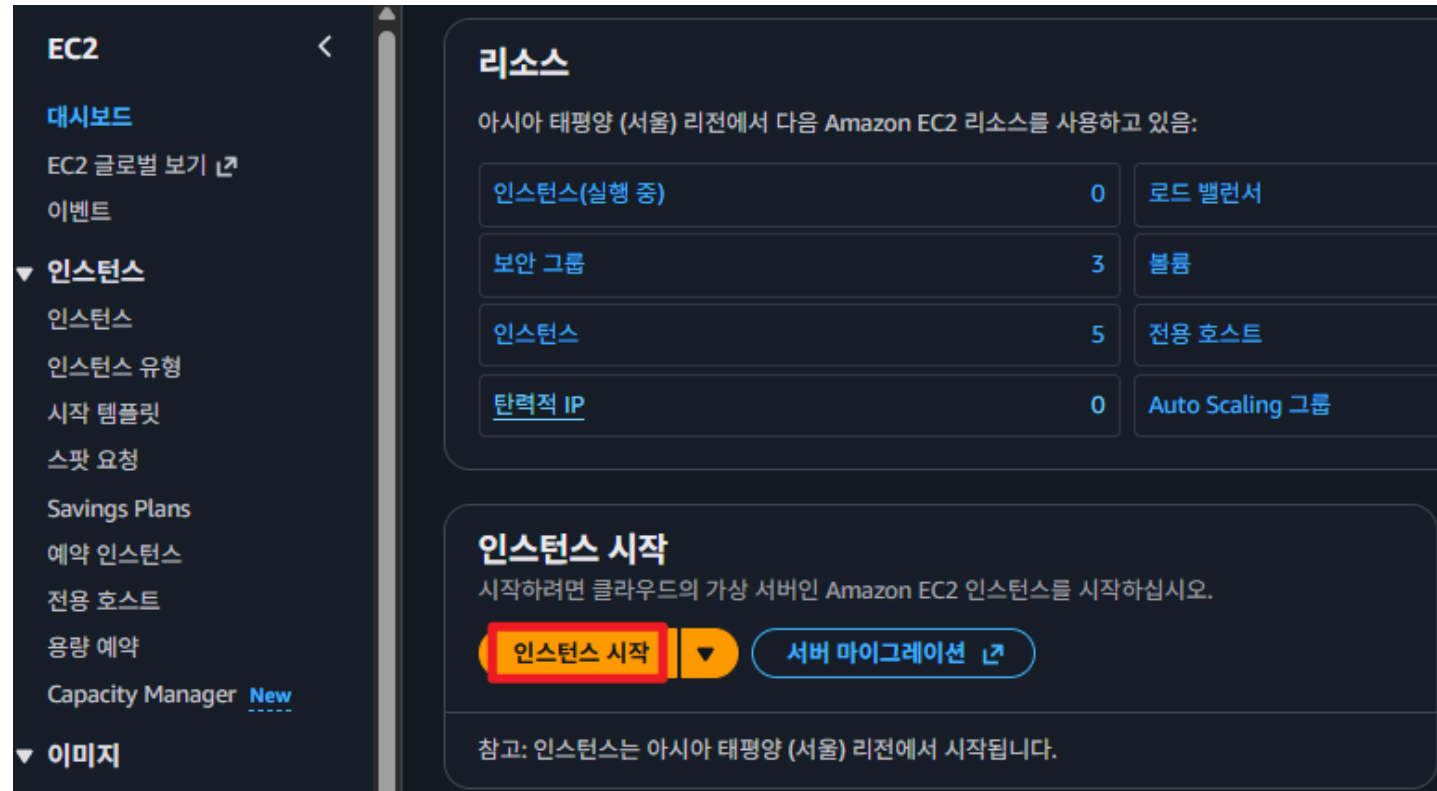
AWS Management Console에서 EC2를 검색합니다.



Master Node 생성

Step 1. 인스턴스 시작

EC2 대시보드에서 '인스턴스 시작'을 클릭합니다.



The screenshot shows the Amazon EC2 console interface. On the left sidebar, the '인스턴스' (Instances) option is selected under the 'EC2' section. The main content area displays the '리소스' (Resources) section, which lists various EC2 resources and their counts. Below this, the '인스턴스 시작' (Start Instance) section is visible, featuring a prominent orange '인스턴스 시작' button. The text below the button indicates that starting an instance will create a virtual server in the Asia Pacific (Seoul) region.

리소스	
인스턴스(실행 중)	0
보안 그룹	3
인스턴스	5
탄력적 IP	0
로드 밸런서	
블록	
전용 호스트	
Auto Scaling 그룹	

인스턴스 시작
시작하려면 클라우드의 가상 서버인 Amazon EC2 인스턴스를 시작하십시오.

인스턴스 시작 (highlighted button) | 서버 마이그레이션

참고: 인스턴스는 아시아 태평양 (서울) 리전에서 시작됩니다.

Master Node 생성

Step 2. Master Node용 인스턴스 생성

- 이름: user##-master-node(##은 할당된 aws 계정의 이름을 사용합니다.)
- 애플리케이션 및 OS 이미지: Ubuntu Server 24.04 LTS
- 인스턴스 유형: **t3.medium**
- 키 페어: 키 페어 생성 후(user##-keypair) 선택

키 페어 생성

키 페어 이름

키 페어를 사용하면 인스턴스에 안전하게 연결할 수 있습니다.

user00-keypair

이름에는 최대 255개의 ASCII 문자가 포함됩니다. 앞 또는 뒤에 공백을 포함할 수 없습니다.

키 페어 유형

☒ RSA
RSA 암호화된 프라이빗 및 퍼블릭 키 페어

☐ ED25519
ED25519 암호화된 프라이빗 및 퍼블릭 키 페어

프라이빗 키 파일 형식

☒ .pem
OpenSSH와 함께 사용

☐ .ppk
PuTTY와 함께 사용

⚠ 메시지가 표시되면 프라이빗 키를 사용자 컴퓨터의 안전하고 액세스 가능한 위치에 저장합니다. 나중에 인스턴스에 연결할 때 필요합니다. 자세히 알아보기

취소

키 페어 생성

Master Node 생성

Step 2. Master Node용 인스턴스 생성

- 네트워크 설정
 - 퍼블릭 IP 자동 할당: 활성화
 - 방화벽: 기존 보안 그룹(default) 선택

참고사항

인스턴스 유형을 선택할 때, 2GB 이상의 RAM과 2 이상의 CPU를 권장합니다.

(참고: <https://kubernetes.io/ko/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>)

키페어 생성 시 private key(.pem)는 로컬에 저장됩니다.

가이드에 기술되어 있지 않는 내용은 기본 값으로 설정했습니다.

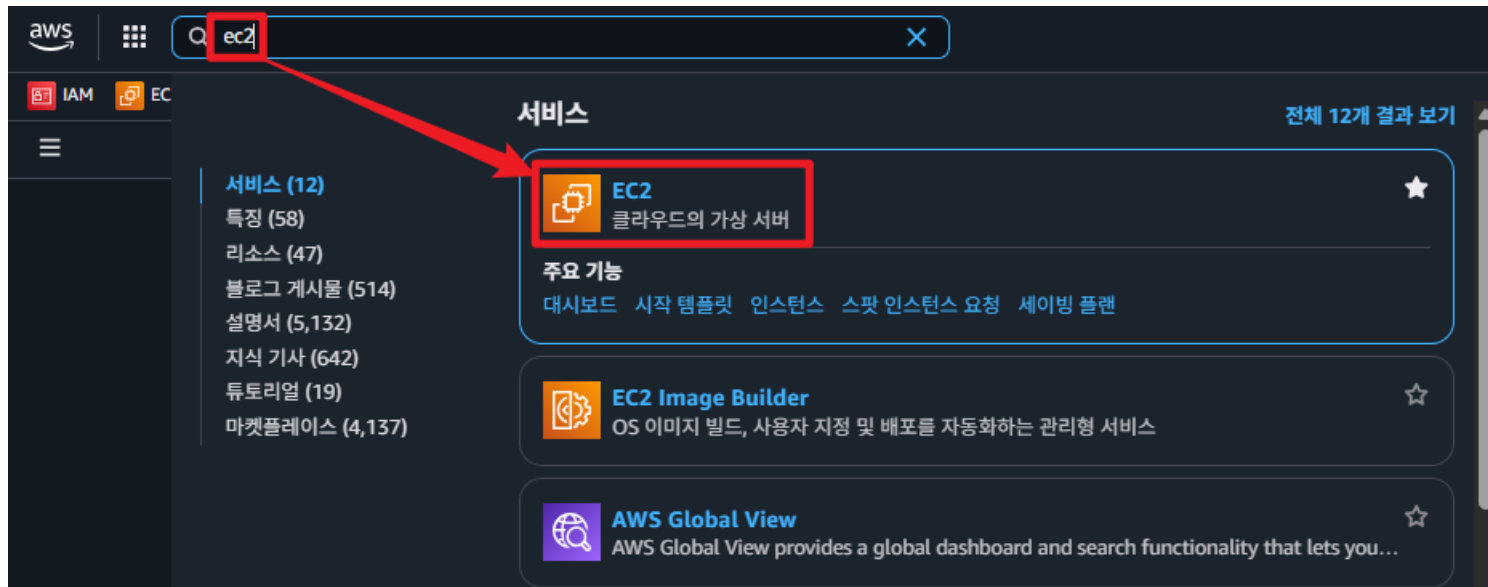
인스턴스 유형

t3.micro		프리 티어 사용 가능
패밀리: t3	2 vCPU 1 GiB 메모리 현재 세대: true 온디맨드 Windows 기본 요금: 0.0222 USD 시간당	
온디맨드 Linux 기본 요금: 0.013 USD 시간당	온디맨드 RHEL 기본 요금: 0.0418 USD 시간당	
온디맨드 SUSE 기본 요금: 0.013 USD 시간당	온디맨드 Ubuntu Pro 기본 요금: 0.0165 USD 시간당	
Q		
온디맨드 Ubuntu Pro 기본 요금: 0.01 USD 시간당		
t3.micro		프리 티어 사용 가능
패밀리: t3	2 vCPU 1 GiB 메모리 현재 세대: true 온디맨드 Windows 기본 요금: 0.0222 USD 시간당	
온디맨드 Linux 기본 요금: 0.013 USD 시간당	온디맨드 RHEL 기본 요금: 0.0418 USD 시간당	
온디맨드 SUSE 기본 요금: 0.013 USD 시간당	온디맨드 Ubuntu Pro 기본 요금: 0.0165 USD 시간당	
t3.small		프리 티어 사용 가능
패밀리: t3	2 vCPU 2 GiB 메모리 현재 세대: true 온디맨드 Linux 기본 요금: 0.026 USD 시간당	
온디맨드 SUSE 기본 요금: 0.057 USD 시간당	온디맨드 Ubuntu Pro 기본 요금: 0.0295 USD 시간당	
온디맨드 RHEL 기본 요금: 0.0548 USD 시간당	온디맨드 Windows 기본 요금: 0.0444 USD 시간당	
t3.medium		
패밀리: t3	4 vCPU 4 GiB 메모리 현재 세대: true 온디맨드 SUSE 기본 요금: 0.1083 USD 시간당	
온디맨드 Ubuntu Pro 기본 요금: 0.0555 USD 시간당	온디맨드 Linux 기본 요금: 0.052 USD 시간당	
온디맨드 Windows 기본 요금: 0.0704 USD 시간당	온디맨드 RHEL 기본 요금: 0.0808 USD 시간당	

Worker Node 생성 1

Step 1. 인스턴스 시작

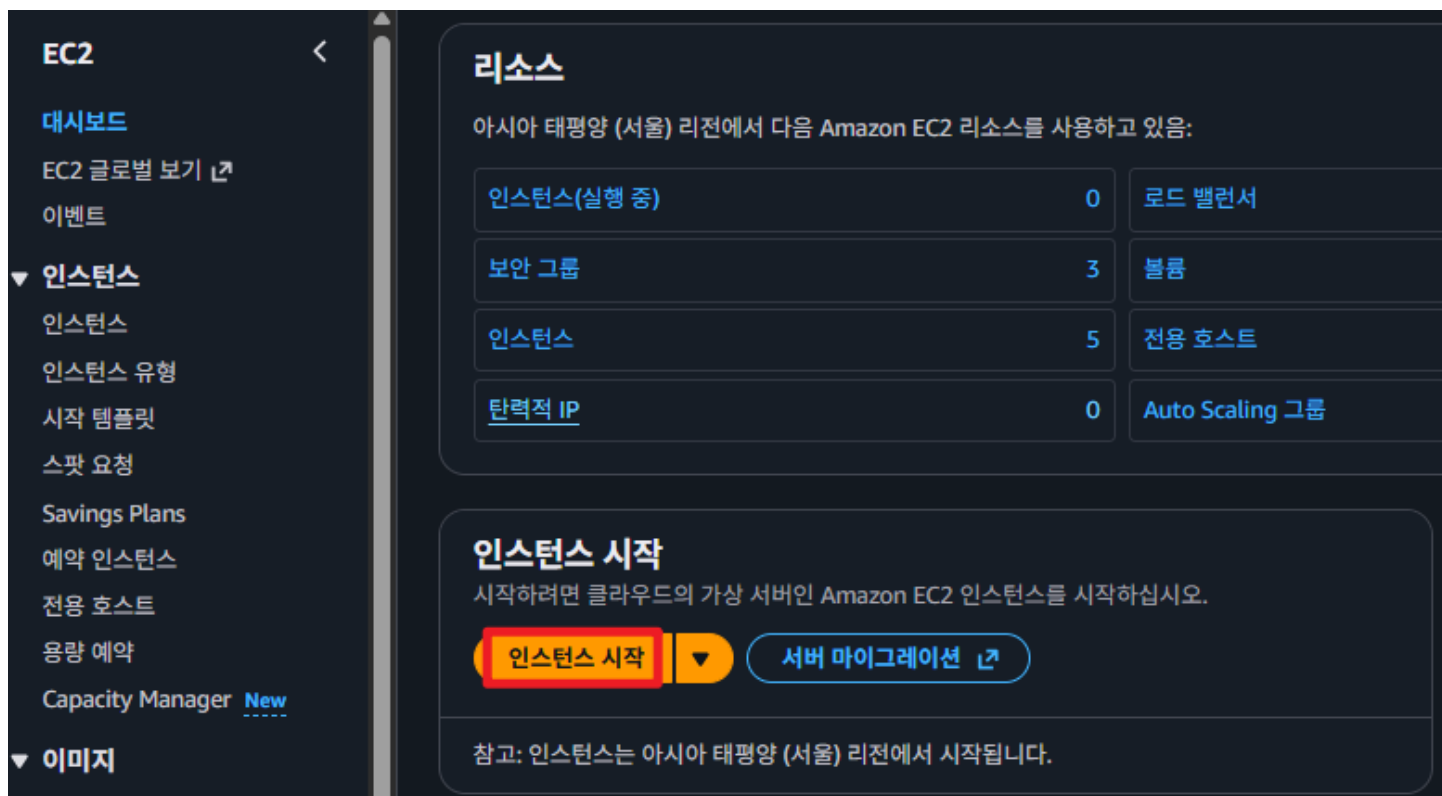
AWS Management Console에서 EC2를 검색합니다.



Worker Node 생성 1

Step 1. 인스턴스 시작

EC2 대시보드에서 '인스턴스 시작'을 클릭합니다.



The screenshot shows the Amazon EC2 console interface. On the left sidebar, the '인스턴스' (Instances) menu item is selected and highlighted. The main content area displays the '리소스' (Resources) section, which lists various EC2 resources. Below this, the '인스턴스 시작' (Start Instance) section is visible, featuring a prominent orange '인스턴스 시작' button. The text below the button indicates that starting an instance will create a virtual server in the Asia Pacific (Seoul) region.

리소스	
인스턴스(실행 중)	0
보안 그룹	3
인스턴스	5
탄력적 IP	0

인스턴스 시작
시작하려면 클라우드의 가상 서버인 Amazon EC2 인스턴스를 시작하십시오.

[인스턴스 시작](#) [서버 마이그레이션](#)

참고: 인스턴스는 아시아 태평양 (서울) 리전에서 시작됩니다.

Worker Node 생성 1

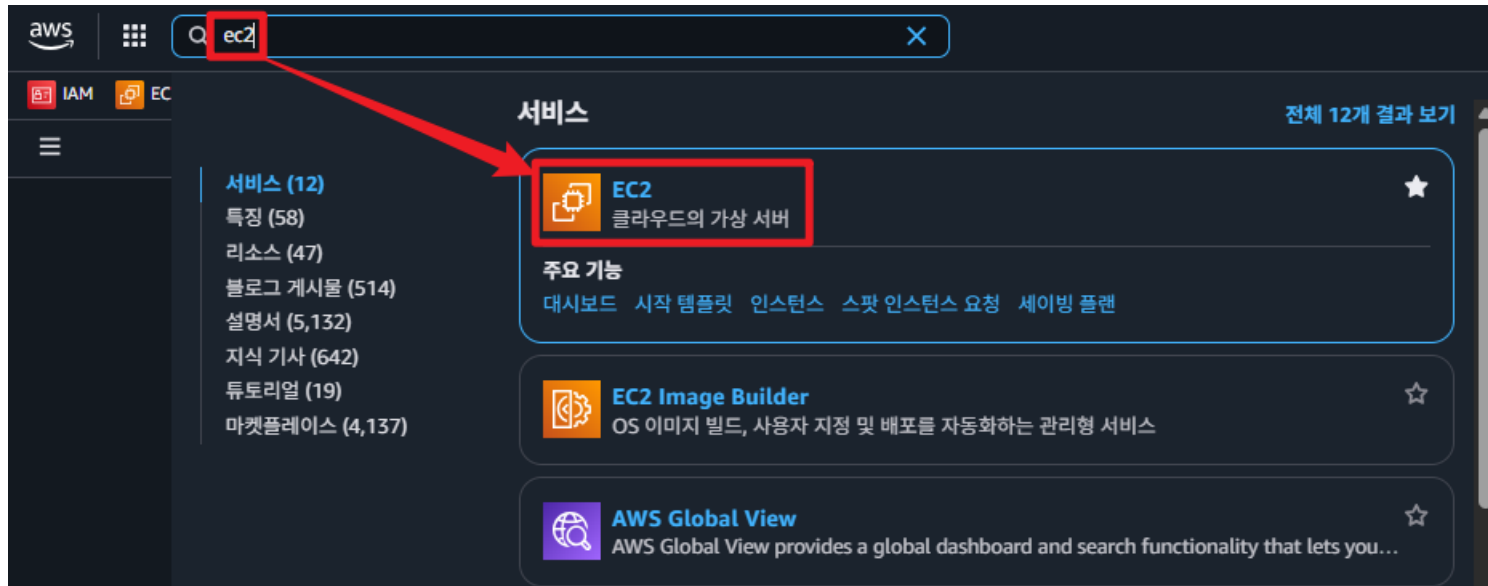
Step 2. Worker Node 1 인스턴스 생성

- 이름: user##-worker-node-1
- 애플리케이션 및 OS 이미지: Ubuntu Server 24.04 LTS
- 인스턴스 유형: **t3.small**
- 키 페어: user##-keypair(Master node에서 생성한 keypair 사용)
- 네트워크 설정
 - 퍼블릭 IP 자동 할당: 활성화
 - 방화벽: 기존 보안 그룹 선택

Worker Node 생성 2

Step 1. 인스턴스 시작

AWS Management Console에서 EC2를 검색합니다.



Worker Node 생성 2

Step 1. 인스턴스 시작

EC2 대시보드에서 '인스턴스 시작'을 클릭합니다.

EC2

대시보드

EC2 글로벌 보기

이벤트

인스턴스

인스턴스

인스턴스 유형

시작 템플릿

스팟 요청

Savings Plans

예약 인스턴스

전용 호스트

용량 예약

Capacity Manager

이미지

리소스

아시아 태평양 (서울) 리전에서 다음 Amazon EC2 리소스를 사용하고 있음:

인스턴스(실행 중)	0	로드 밸런서
보안 그룹	3	볼륨
인스턴스	5	전용 호스트
탄력적 IP	0	Auto Scaling 그룹

인스턴스 시작

시작하려면 클라우드의 가상 서버인 Amazon EC2 인스턴스를 시작하십시오.

인스턴스 시작

서버 마이그레이션

참고: 인스턴스는 아시아 태평양 (서울) 리전에서 시작됩니다.

Worker Node 생성 2

Step 2. Worker Node 2 인스턴스 생성

- 이름: user##-worker-node-2
- 애플리케이션 및 OS 이미지: Ubuntu Server 24.04 LTS
- 인스턴스 유형: **t3.small**
- 키 페어: user##-keypair(Master node에서 생성한 keypair 사용)
- 네트워크 설정
 - 퍼블릭 IP 자동 할당: 활성화
 - 방화벽: 기존 보안 그룹 선택

Instance에 연결

Step 1. 생성한 인스턴스 확인

연결한 노드를 선택하고, 우측 상단에 연결을 클릭합니다.

인스턴스 (1/3) 정보

최종 업데이트 날짜 less than a minute 전

연결 인스턴스 상태 작업 인스턴스 시작

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

모든 상태

Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사	경보 상태	가용 영역	퍼블릭 IPv4 DNS	퍼블릭 IPv4 ...	탄력적 II
user00-master-node	i-0de76be6766c089aa	실행 중	t2.micro	2/2개 검사 통과	경보 보기	ap-northeast-2a	ec2-13-124-45-183.ap-...	13.124.45.183	-

SSH 클라이언트에서 Instance에 연결하기 위한 순서를 확인합니다.

인스턴스에 연결 정보

다음 옵션 중 하나를 사용하여 인스턴스 i-0de76be6766c089aa (user00-master-node)에 연결

EC2 인스턴스 연결 Session Manager **SSH 클라이언트** EC2 직렬 콘솔

인스턴스 ID
i-0de76be6766c089aa (user00-master-node)

1. SSH 클라이언트를 엽니다.
2. 프라이빗 키 파일을 찾습니다. 이 인스턴스를 시작하는 데 사용되는 키는 user00-keypair.pem입니다.
3. 필요한 경우 이 명령을 실행하여 키를 공개적으로 볼 수 없도록 합니다.
chmod 400 "user00-keypair.pem"
4. 퍼블릭 DNS을(를) 사용하여 인스턴스에 연결:
ec2-13-124-45-183.ap-northeast-2.compute.amazonaws.com

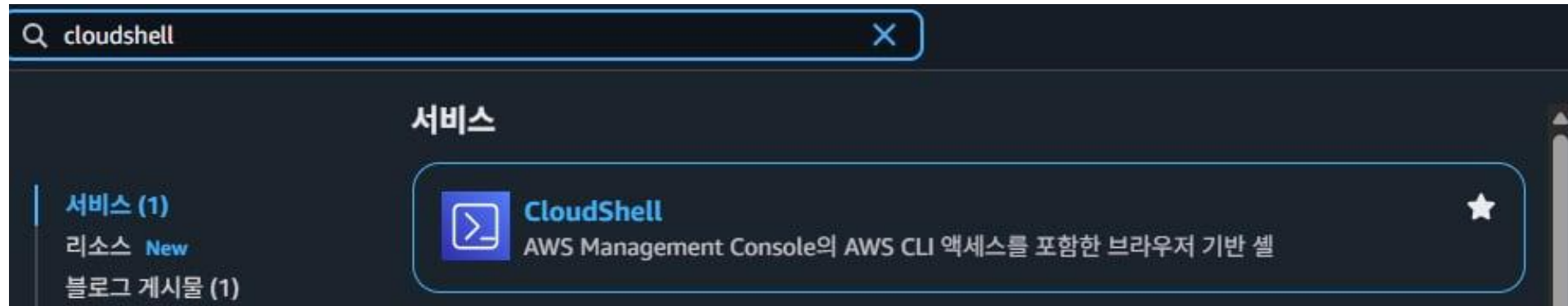
예:
ssh -i "user00-keypair.pem" ubuntu@ec2-13-124-45-183.ap-northeast-2.compute.amazonaws.com

복사

Instance에 연결

Step 1. 생성한 인스턴스 확인

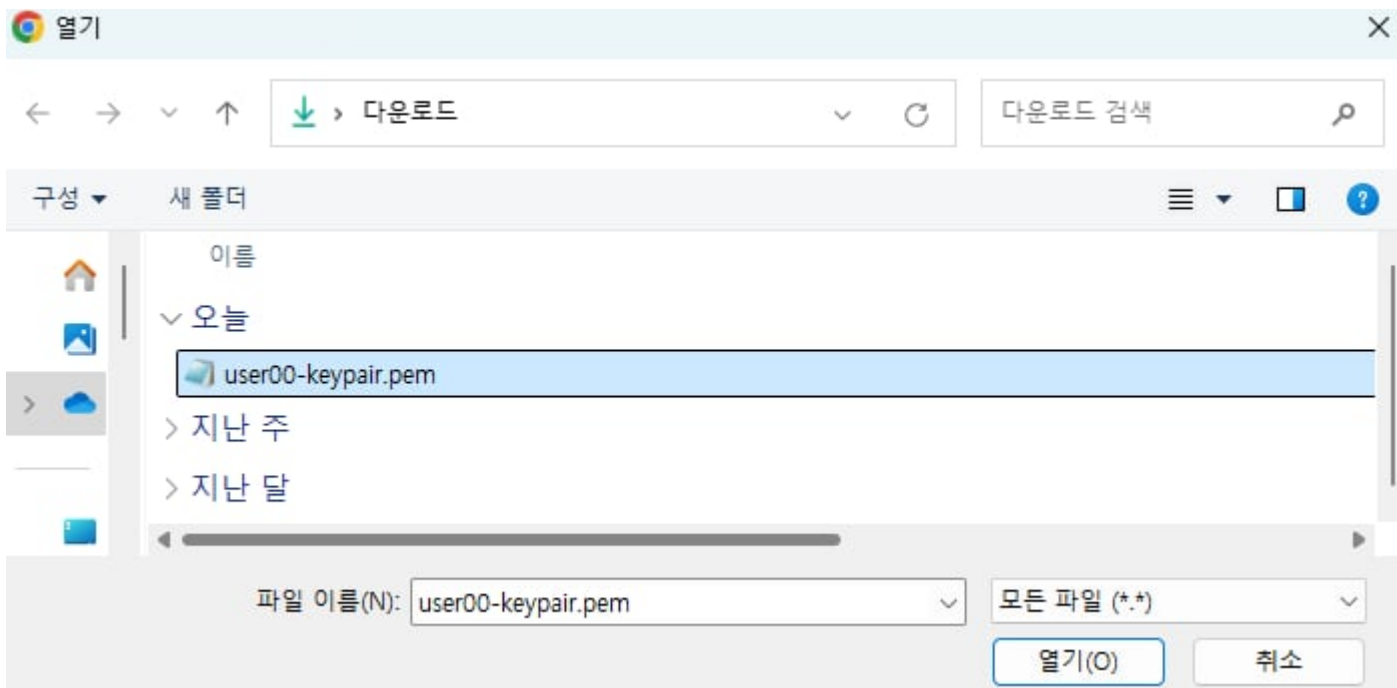
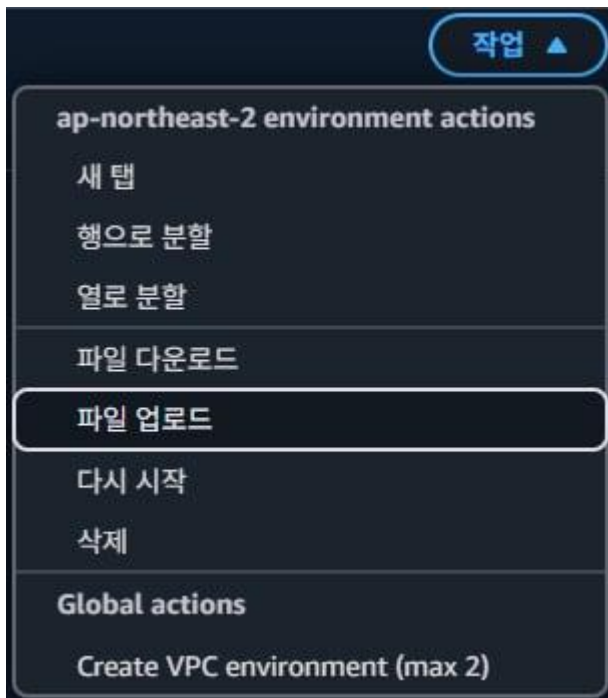
AWS 서비스 검색 창에 "CloudShell"을 검색하고, 선택합니다. (우클릭 > 새 탭에서 열기)



Instance에 연결

Step 2. 인스턴스에 연결

작업 > 파일 업로드를 클릭해 Instance를 만들 때 생성한 keypair를 업로드 합니다.

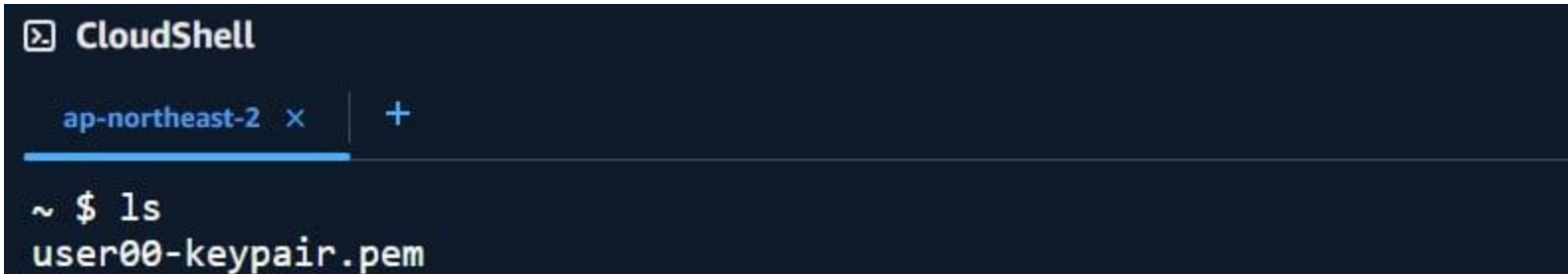


Instance에 연결

Step 2. 인스턴스에 연결

.pem이 업로드 되었는지 확인합니다.

```
ls
```



```
CloudShell
ap-northeast-2 x +
~ $ ls
user00-keypair.pem
```

pem key의 권한을 설정합니다.

```
chmod 400 "user##-keypair.pem"
```


Instance에 연결

Step 2. 인스턴스에 연결

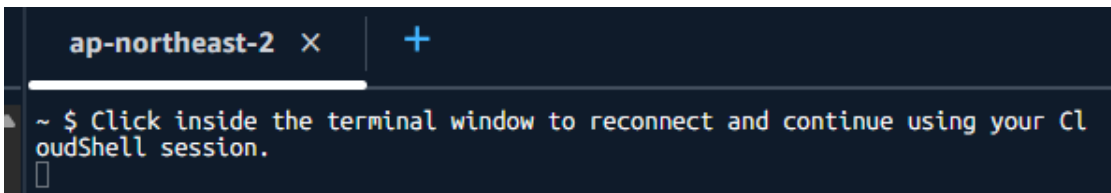
퍼블릭 DNS를 사용하여 Instance에 연결합니다. *17p에서 복사한 코드 (ec2-x-x-x-x에서 x는 Instance의 Public IP로 작성합니다)

```
ssh -i "user##-keypair.pem" ubuntu@ec2-x-x-x-x.ap-northeast-2.compute.amazonaws.com
```

Are you sure you want to continue connecting (yes/no/[fingerprint])? 에서 **yes**를 입력합니다.

```
~ $ ssh -i "user00-keypair.pem" ubuntu@ec2-3-38-112-136.ap-northeast-2.compute.amazonaws.com
The authenticity of host 'ec2-3-38-112-136.ap-northeast-2.compute.amazonaws.com (3.38.112.136)' can't be established.
ED25519 key fingerprint is SHA256:JGV06n29CWIIzf/Edc8vJx6SQUN5a8bWj+4euudcqnU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

★ 세션이 종료되면 위의 ssh 명령어를 통해 각 노드에 **다시 접속**을 해야 합니다.(설치 및 설정 내용은 유지됨)



```
ap-northeast-2 x +
~ $ Click inside the terminal window to reconnect and continue using your CloudShell session.
```

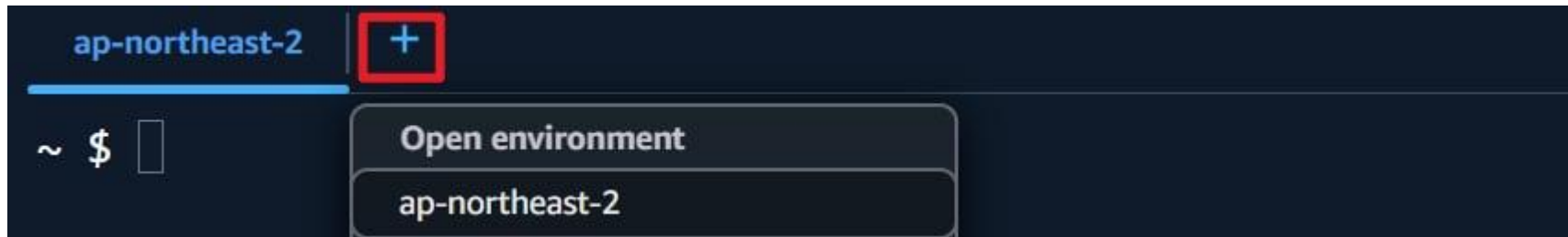
Instance에 연결

Step 2. 인스턴스에 연결

Master Node의 Prompt를 확인합니다.

```
ubuntu@ip-172-31-9-82:~$  
ubuntu@ip-172-31-9-82:~$
```

동일한 방법으로 Worker Node1&2에 SSH로 접속합니다. (17&21p 반복)



Lab 01.

Kubernetes 설치

Step 1. Update and Upgrade Ubuntu (all nodes)

모든 노드에서 다음 명령어를 실행하여 시스템 패키지를 최신 상태로 업데이트하고 업그레이드하세요.

```
sudo apt update && sudo apt upgrade -y
```

```
ubuntu@ip-172-31-40-81:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Get:3 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages
Get:5 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble/universe Translation
```

Kubernetes 설치

Step 2. Disable Swap (all nodes)

Kubernetes 성능을 향상시키기 위해 스왑을 비활성화하고 필수 커널 매개변수를 설정합니다.

※ sudo 명령어는 여러 줄을 복사해서 한번에 실행할 시 오류가 발생할 수 있으므로, **한 줄씩** 실행하세요.

```
sudo swapoff -a  
sudo sed -i '/ swap / s/^W(.*)$/#W1/g' /etc/fstab
```

```
ubuntu@ip-172-31-40-81:~$ sudo swapoff -a  
ubuntu@ip-172-31-40-81:~$ sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
```

Step 3. Add Kernel Parameters (all nodes)

다음 명령어를 사용해 필요한 커널 모듈을 모든 노드에서 로드합니다.

```
sudo tee /etc/modules-load.d/containerd.conf <<EOF
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
```

```
ubuntu@ip-172-31-40-81:~$ sudo tee /etc/modules-load.d/containerd.conf <<EOF
> overlay
> br_netfilter
> EOF
overlay
br_netfilter
ubuntu@ip-172-31-40-81:~$ sudo modprobe overlay
ubuntu@ip-172-31-40-81:~$ sudo modprobe br_netfilter
```


Step 4: Install Containerd Runtime (all nodes)

Kubernetes는 CRI(Container Runtime Interface)를 지원하는 구현체를 통해 컨테이너를 사용합니다. containerd, cri-o 등이 CRI를 통해 컨테이너를 제어할 수 있는 방법을 제공합니다. 여기에서는 containerd 런타임을 사용합니다. 다음 명령어를 통해 containerd와 관련 의존성을 설치하세요.

```
sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-certificates
```

```
ubuntu@ip-172-31-40-81:~$ sudo apt install -y curl gnupg2 software-properties-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
```

Step 4: Install Containerd Runtime (all nodes)

Docker 저장소를 활성화하세요. 이후 시스템에 추가하기 위해 ENTER 키를 눌러주세요.

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.gpg  
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
```

```
ubuntu@ip-172-31-40-81:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu  
c/apt/trusted.gpg.d/docker.gpg  
ubuntu@ip-172-31-40-81:~$ sudo add-apt-repository "deb [arch=amd64] https://downlo  
release -cs) stable"  
Repository: 'deb [arch=amd64] https://download.docker.com/linux/ubuntu noble stabl  
Description:  
Archive for codename: noble components: stable  
More info: https://download.docker.com/linux/ubuntu  
Adding repository.  
Press [ENTER] to continue or Ctrl-c to cancel.  
Adding deb entry to /etc/apt/sources.list.d/archive_uri-https_download_docker_com_  
Adding disabled deb-src entry to /etc/apt/sources.list.d/archive_uri-https_downloa
```


Step 4: Install Containerd Runtime (all nodes)

패키지 목록을 업데이트하고 containerd를 설치하세요.

```
sudo apt update
sudo apt install -y containerd.io
```

```
ubuntu@ip-172-31-40-81:~$ sudo apt update
Hit:1 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-northeast-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
ubuntu@ip-172-31-40-81:~$ sudo apt install -y containerd.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  containerd.io
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
```

Kubernetes 설치

Step 4: Install Containerd Runtime (all nodes)

containerd가 systemd를 Cgroup으로 사용하도록 설정하세요.

```
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1
```

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
```

containerd 서비스를 재시작하고 자동 시작하도록 설정하세요.

```
sudo systemctl restart containerd
```

```
sudo systemctl enable containerd
```

Kubernetes 설치

Step 5: Add Apt Repository for Kubernetes (all nodes)

Kubernetes 패키지는 기본 Ubuntu 저장소에 포함되어 있지 않습니다. 다음 명령어를 사용해 Kubernetes 저장소를 추가하세요.

```
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" \
| sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

★입력주의★ 복사하지 말고 직접 입력하세요. 복붙할 경우 오류 발생합니다.

```
ubuntu@ip-172-31-44-20:~$ echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" \ | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /
ubuntu@ip-172-31-44-20:~$
```

```
ubuntu@ip-172-31-44-20:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
ubuntu@ip-172-31-44-20:~$
```

Kubernetes 설치

Step 6: Install Kubectl, Kubeadm, and Kublet (all nodes)

저장소 추가 후, 모든 노드에 kubectl, kublet, kubeadm 등 필수 Kubernetes 구성 요소를 다음 명령어로 설치합니다.

```
sudo apt update  
sudo apt install -y kubelet kubeadm kubectl
```

kubelt, kubeadm, kubectl 패키지를 자동 업데이트/업그레이드에서 제외시킵니다.

```
sudo apt-mark hold kubelet kubeadm kubectl
```

```
ubuntu@ip-172-31-40-81:~$ sudo apt-mark hold kubelet kubeadm kubectl  
kubelet set on hold.  
kubeadm set on hold.  
kubectl set on hold.
```

Step 7. Add Kernel Parameters (all nodes)

필요한 주요 커널 매개변수를 설정해 Kubernetes가 원활하게 동작하도록 합니다.
/etc/sysctl.d/kubernetes.conf 파일을 생성하고 아래 내용을 추가하세요.

```
sudo tee /etc/sysctl.d/kubernetes.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

```
ubuntu@ip-172-31-40-81:~$ sudo tee /etc/sysctl.d/kubernetes.conf <<EOF
> net.bridge.bridge-nf-call-ip6tables = 1
> net.bridge.bridge-nf-call-iptables = 1
> net.ipv4.ip_forward = 1
> EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```


Step 7. Add Kernel Parameters (all nodes)

설정된 내용이 적용되었는지 확인합니다.

```
sudo sysctl --system
```

```
ubuntu@ip-172-31-47-142:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-zero-page.conf ...
* Applying /etc/sysctl.d/50-cloudimg-settings.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/99-cloudimg-ipv6.conf ...
* Applying /usr/lib/sysctl.d/99-protect-links.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/kubernetes.conf ...
* Applying /etc/sysctl.conf ...
kernel.apparmor_restrict_unprivileged_userns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
```

```
* Applying /etc/sysctl.conf ...
kernel.apparmor_restrict_unprivileged_userns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
net.ipv4.neigh.default.gc_thresh2 = 15360
net.ipv4.neigh.default.gc_thresh3 = 16384
net.netfilter.nf_conntrack_max = 1048576
kernel.pid_max = 4194304
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.default.use_tempaddr = 0
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```

Lab 02.

Cluster init, join

Kubernetes 설치

Step 1: hostname 변경 (all nodes)

hostname을 변경해 인스턴스를 구분할 수 있게 합니다.

master node의 hostname을 변경합니다. 각 노드에서 hostname을 변경한 뒤 **exit(2번)**로 빠져나옵니다.

```
sudo -i
sudo hostnamectl set-hostname k8s-master
sudo -i
```

```
ubuntu@k8s-master:~$ sudo -i
root@k8s-master:~# sudo hostnamectl set-hostname k8s-master
root@k8s-master:~# sudo -i
root@k8s-master:~# exit
logout
root@k8s-master:~# exit
logout
ubuntu@k8s-master:~$ █
```

첫 번째 worker node도 변경하고, 두 번째 worker node도 변경합니다.

```
sudo -i
sudo hostnamectl set-hostname k8s-worker-1
sudo -i
```

```
sudo -i
sudo hostnamectl set-hostname k8s-worker-2
sudo -i
```


Step 2: Initialize Kubernetes Cluster with Kubeadm (master node)

kubeadm 명령어를 사용하여 master node에서 Kubernetes 클러스터를 초기화합니다.

```
sudo kubeadm init
```

Alternatively, if you are the root user, you can run:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:

<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 172.31.6.86:6443 --token 9mbmtt.5fw1tg4r51p2n1tr \  
--discovery-token-ca-cert-hash sha256:dbaa466c97fe4d698f21f6d7cf4ec85c471ee363b0ffd89055978bf5179fb53c
```

초기화가 완료되면 *worker node*를 클러스터에 조인할 수 있는 *kubeadm join* 명령어가 출력됩니다.
이 명령어는 *worker node* 설정 시 사용하므로 꼭 복사해두세요.

Step 2: Initialize Kubernetes Cluster with Kubeadm (master node)

master node에서 다음 명령어들을 실행하세요.

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
ubuntu@k8s-master:~$ mkdir -p $HOME/.kube  
ubuntu@k8s-master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
ubuntu@k8s-master:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config  
ubuntu@k8s-master:~$
```

다음으로, kubectl 명령어를 사용하여 클러스터와 노드 상태를 확인합니다.

```
kubectl get nodes
```

```
ubuntu@k8s-master:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	NotReady	control-plane	2m35s	v1.24.0

Step 3: Add Worker Nodes to the Cluster (worker nodes)

각 worker node에서 이전에 복사한 kubeadm join 명령어(37p) 앞에 **sudo**를 붙여 사용하세요.

```
sudo kubeadm join 172.31.x.x:6443 --token 9mbmtt.5fw1tg4r51p2n1tr W
--discovery-token-ca-cert-hash
sha256:dbaa466c97fe4d698f21f6d7cf4ec85c471ee363b0ffd89055978bf5179fb53c
```

```
ubuntu@k8s-worker-1:~$ sudo kubeadm join 172.31.5.174:6443 --token icjtl1.bs8fhxsjxpzfheml \
> --discovery-token-ca-cert-hash sha256:bf959ca7810fed3979595fffc803f7f5b3a32912995a9ddf60c8defe385b3
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.002423318s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

Step 3: Add Worker Nodes to the Cluster (master nodes)

master node에서 `kubectl get nodes` 명령어로 worker node 잘 연결되었는지 확인할 수 있습니다.

```
ubuntu@k8s-master:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	NotReady	control-plane	6m4s	v1.21.2
k8s-worker-1	NotReady	<none>	12s	v1.21.2
k8s-worker-2	NotReady	<none>	10s	v1.21.2

Kubernetes 설치

Step 4: Install Kubernetes Network Plugin (master node)

클러스터 내의 Pod 간 통신을 가능하게 하려면 네트워크 플러그인이 필요합니다. master node에서 아래 명령어를 사용하여 Calico 네트워크 플러그인을 설치합니다.

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.2/manifests/calico.yaml
```

설치 후 약간의 시간이 지나면, kubectl get nodes 명령어를 통해 모든 노드 상태가 Ready로 바뀌는 것을 확인할 수 있습니다.

```
ubuntu@k8s-master:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	9m42s	v1.24.2
k8s-worker-1	Ready	<none>	3m50s	v1.24.2
k8s-worker-2	Ready	<none>	3m48s	v1.24.2

Step 5: Verify the Cluster and Test (master node)

마지막으로 클러스터가 성공적으로 생성되었는지 확인해 봅시다.

```
kubectl get pods -n kube-system
```

```
ubuntu@k8s-master:~$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
calico-kube-controllers-78b7fdb74b-j4md9	1/1	Running	0	115s
calico-node-7j9ds	1/1	Running	0	115s
calico-node-fmp9m	1/1	Running	0	115s
calico-node-qkr5k	1/1	Running	0	115s
coredns-55cb58b774-25fcv	1/1	Running	0	9m45s

Step 6: Deploy Test Application on Cluster (master node)

클러스터가 정상적으로 동작하는지 확인하기 위해 간단한 테스트 애플리케이션을 배포해 봅시다.
예시로 Nginx를 사용했습니다.

```
kubectl run nginx --image=nginx  
kubectl get pods
```

```
ubuntu@k8s-master:~$ kubectl run nginx --image=nginx  
pod/nginx created  
ubuntu@k8s-master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx	0/1	ContainerCreating	0	3s

Lab 03.

Pod

Step 1: Pod 생성(명령어) (master node)

테스트용 pod를 생성합니다.

```
kubectl run pod-test1 --image=nginx --port=80
```

```
ubuntu@k8s-master:~$ kubectl run pod-test1 --image=nginx --port=80  
pod/pod-test1 created
```

생성한 pod 목록을 확인합니다.

```
kubectl get pods
```

```
ubuntu@k8s-master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	19m
pod-test1	1/1	Running	0	49s

Step 1: Pod 생성(명령어)

아래 명령어로 생성한 pod의 자세한 정보(IP 등)를 확인할 수 있습니다.

```
kubectl describe pods pod-test1
```

```
ubuntu@k8s-master:~$ kubectl describe pods pod-test1
```

```
Name: pod-test1
Namespace: default
Priority: 0
Service Account: default
Node: k8s-worker-2/172.31.39.154
Start Time: Mon, 02:30:33 +0000
Labels: run=pod-test1
Annotations: cni.projectcalico.org/containerID: 15ce88547ff4941f9907612bf21
39
cni.projectcalico.org/podIP: 192.168.140.1/32
cni.projectcalico.org/podIPs: 192.168.140.1/32
Status: Running
```

Step 1: Pod 생성(명령어)

확인한 IP를 이용해 nginx가 정상적으로 실행 중인 것을 확인합니다.

```
curl <Pod IP>
```

```
ubuntu@k8s-master:~$ curl 192.168.140.1
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
```

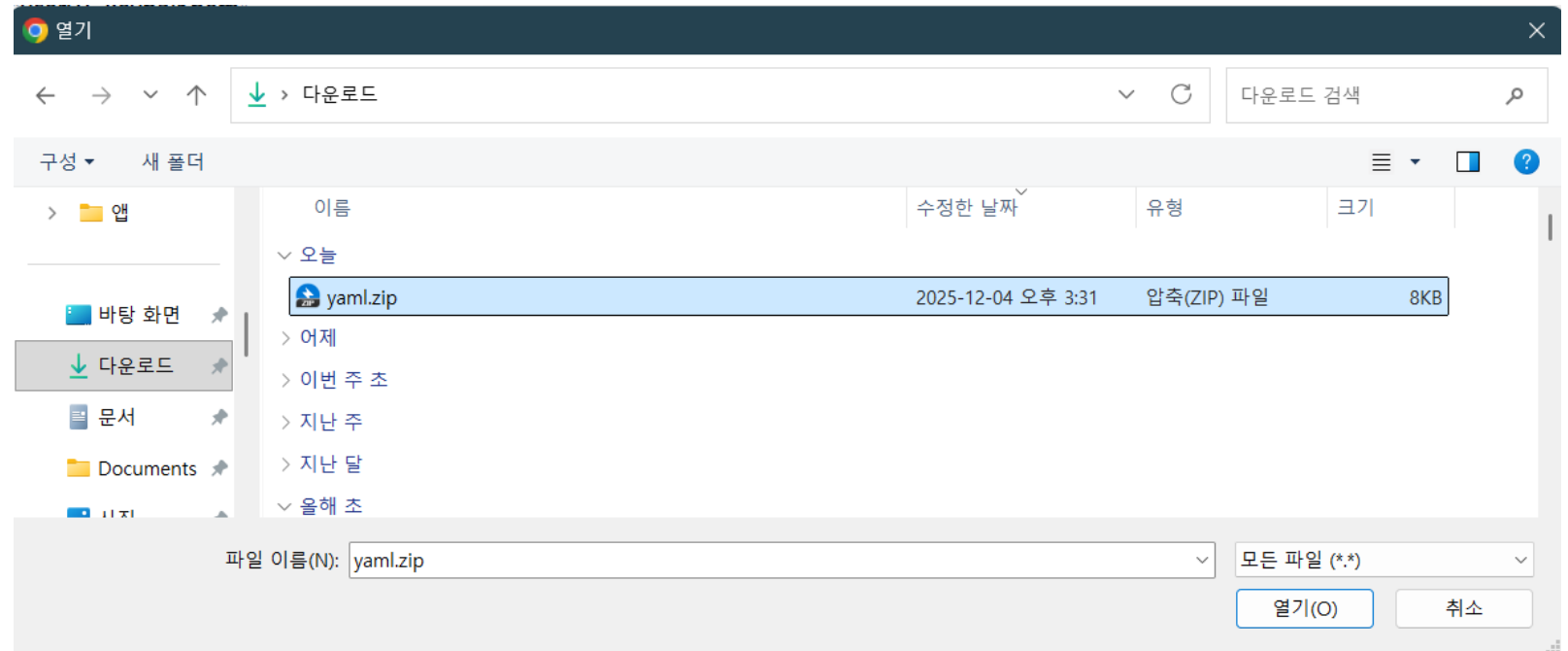
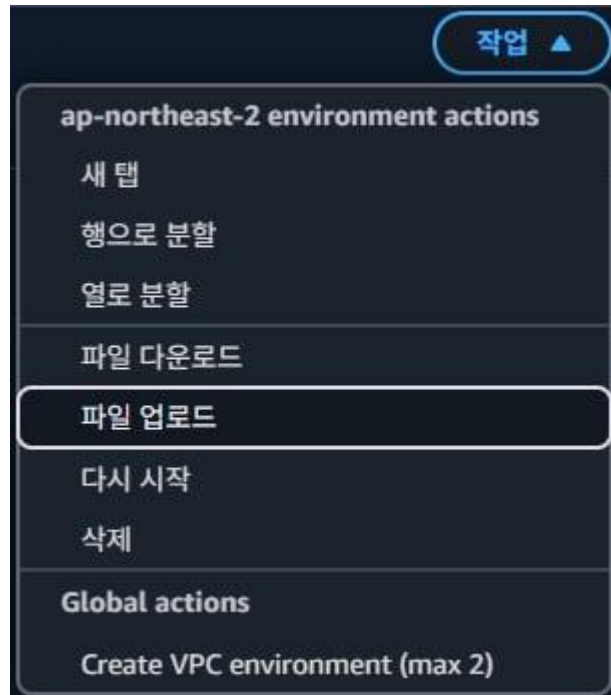
Step 2: 파일 확인 및 압축 해제

YAML 파일로 컨테이너 리소스를 생성할 수 있습니다. 컨테이너 자체는 물론이고, 컨테이너의 설정 값 (ConfigMap), 비밀 값(Secrets) 등도 모두 YAML 파일로 정의해 사용할 수 있습니다.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-test2
spec:
  containers:
  - name: test-nginx-container
    image: nginx:latest
    ports:
    - containerPort: 80
      protocol: TCP
```

Step 2: 파일 확인 및 압축 해제

작업 > 파일 업로드를 클릭해 제공된 yaml.zip 파일을 Cloudshell에 업로드합니다.



Step 2: 파일 확인 및 압축 해제

Cloudshell에서 exit하고 scp 명령어로 master-node 인스턴스에 복사합니다.

```
scp -i user##-keypair.pem yaml.zip ubuntu@<MasterIP>:~/yaml.zip
```

```
ubuntu@k8s-master:~$ exit
logout
Connection to ec2-13-124-75-93.ap-northeast-2.compute.amazonaws.com closed.
~ $
```

```
~ $ scp -i user03-keypair.pem yaml.zip ubuntu@13.124.75.93:~/yaml.zip
yaml.zip
~ $
~ $ ssh -i "user03-keypair.pem" ubuntu@ec2-13-124-75-93.ap-northeast-2.compute.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1016-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```


Step 2: 파일 확인 및 압축 해제

ssh 명령어를 사용해 master-node 인스턴스에 다시 접속하고 파일을 확인합니다.

```
ssh -i "user##-keypair.pem" ubuntu@ec2-x-x-x-x.ap-northeast-2.compute.amazonaws.com
```

```
~ $ scp -i user03-keypair.pem yaml.zip ubuntu@13.124.75.93:~/yaml.zip
yaml.zip
~ $
~ $ ssh -i "user03-keypair.pem" ubuntu@ec2-13-124-75-93.ap-northeast-2.compute.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1016-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

```
ubuntu@k8s-master:~$ ls
yaml.zip
```


Step 2: 파일 확인 및 압축 해제

설치한 파일의 압축을 해제하기 위해 unzip을 설치하고, 압축을 해제합니다.

```
sudo apt-get install unzip
```

```
ubuntu@k8s-master:~$ sudo apt-get install unzip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  zip
The following NEW packages will be installed:
  unzip
```

```
unzip yaml.zip
```

```
ubuntu@k8s-master:~$ unzip yaml.zip
Archive:  yaml.zip
  creating: cm/
  inflating: cm/multiple-keyvalue.env
  extracting: cm/pw1
  extracting: cm/index.html
  inflating: cm/volume-mount-configmap.yaml
```

Step 3: Pod 생성(YAML)

YAML 파일(pod.yaml)로 컨테이너를 생성합니다. ~/po는 해당 디렉터리로 들어간 상황입니다.

```
cd po
kubectl apply -f pod.yaml
```

```
ubuntu@k8s-master:~$ cd po
ubuntu@k8s-master:~/po$ kubectl apply -f pod.yaml
pod/pod-test2 created
```

생성한 컨테이너를 확인합니다.

```
cd ..
kubectl get pods
```

```
ubuntu@k8s-master:~/po$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	27m
pod-test1	1/1	Running	0	8m45s
pod-test2	1/1	Running	0	56s

Pod 접근 및 삭제

Step 1: 컨테이너 접근

kubectl exec 명령으로 pod의 컨테이너에 명령어를 전달할 수 있습니다.

```
kubectl exec -it <POD> -- bash
```

```
ubuntu@k8s-master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
nginx         1/1     Running   0           93m
pod-test1     1/1     Running   0           74m
pod-test2     1/1     Running   0           66m
ubuntu@k8s-master:~$
ubuntu@k8s-master:~$ kubectl exec -it pod-test1 -- bash
root@pod-test1:/#
root@pod-test1:/#
```

pod에서 빠져나옵니다.

```
exit
```

Pod 접근 및 삭제

Step 2: 컨테이너 접근

kubectl logs 명령어로 pod의 로그를 확인할 수 있습니다.

```
kubectl logs <POD>
```

```
ubuntu@k8s-master:~$ kubectl logs pod-test1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/c
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
```

Pod 접근 및 삭제

Step 3: Pod 삭제

```
cd po
kubectl delete -f <YAML FILE>
kubectl get pods
```

```
ubuntu@k8s-master:~/po$ kubectl delete -f pod.yaml
pod "pod-test2" deleted
ubuntu@k8s-master:~/po$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	100m
pod-test1	1/1	Running	0	81m

또는(pod-test를 모두 삭제해주세요)

```
kubectl delete pod <POD>
kubectl get pods
```

```
cd ..      상위 폴더로 이동하기
```

Lab 04.

Replicaset

Replicaset 사용하기

Step 1: YAML 파일 정의

Nginx Pod를 생성하는 Replicaset을 만듭니다.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  apiVersion: apps/v1
  kind: ReplicaSet
  metadata:
    name: replicaset-nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: test-nginx-pods-label
  template:
    metadata:
      name: test-nginx-pod
      labels:
        app: test-nginx-pods-label
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```


Replicaset 사용하기

Step 2: Replicaset 생성

YAML(rs-nginx.yaml)로 Replicaset을 생성합니다.

```
cd rs
kubectl apply -f rs-nginx.yaml
```

```
ubuntu@k8s-master:~/rs$ kubectl apply -f rs-nginx.yaml
replicaset.apps/replicaset-nginx created
```

생성된 Replicaset과 Pod를 확인합니다.

```
kubectl get pods
kubectl get replicaset
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
replicaset-nginx-88nk8              1/1     Running   0           12s
replicaset-nginx-g5lv5              1/1     Running   0           12s
replicaset-nginx-hdp2z              1/1     Running   0           12s
ubuntu@k8s-master:~/rs$
ubuntu@k8s-master:~/rs$ kubectl get replicaset
NAME            DESIRED   CURRENT   READY   AGE
replicaset-nginx 3          3         3       23s
```

Replicaset 사용하기

Step 3: Pod 개수 변경

Replicaset에 정의된 Pod의 개수를 변경(rs-nginx.yaml 파일 수정)합니다.

```
vi rs-nginx.yaml
```

i : 수정

```
-- INSERT --
```

esc : 수정 종료

:qa : 저장 후 닫기

```
:qa
```

```
1 apiVersion: apps/v1
2 kind: ReplicaSet
3 metadata:
4   name: replicaset-nginx
5 spec:
6   replicas: 5
7   selector:
8     matchLabels:
9       app: test-nginx-pods-label
10  template:
```

Replicaset 사용하기

Step 3: Pod 개수 변경

변경한 YAML 파일을 Kubernetes에 적용하면,

```
kubectl apply -f rs-nginx.yaml
```

```
ubuntu@k8s-master:~/rs$ kubectl apply -f rs-nginx.yaml
replicaset.apps/replicaset-nginx configured
```

새롭게 생성된 Pod를 확인할 수 있습니다.

```
kubectl get pods
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
replicaset-nginx-88nk8	1/1	Running	0	6m25s
replicaset-nginx-g5lv5	1/1	Running	0	6m25s
replicaset-nginx-hdp2z	1/1	Running	0	6m25s
replicaset-nginx-shcwj	1/1	Running	0	74s
replicaset-nginx-tn6zl	1/1	Running	0	74s

Replicaset 사용하기

Step 4: Replicaset 삭제

Replica Set을 삭제합니다.

```
kubectl delete rs <REPLICASET>
```

```
ubuntu@k8s-master:~/rs$ kubectl delete rs replicaset-nginx
replicaset.apps "replicaset-nginx" deleted
ubuntu@k8s-master:~/rs$
ubuntu@k8s-master:~/rs$ kubectl get rs,po
No resources found in default namespace.
```

또는

```
kubectl delete -f <YAML파일>
```

Replicaset의 동작 원리

Step 1: 파일 준비

Replicaset은 spec.selector.matchLabel에 정의된 레이블을 통해 생성해야 하는 Pod를 찾습니다.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: replicaset-nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: test-nginx-pods-label
  template:
    metadata:
      name: test-nginx-pod
    labels:
      app: test-nginx-pods-label
```


Replicaset의 동작 원리

Step 2: Pod 생성

metadata.labels.app이 test-nginx-pods-label인 Pod를 생성하는 YAML을 준비합니다.

```
apiVersion: v1
kind: Pod
metadata:
  name: test-nginx-pod
  labels:
    app: test-nginx-pods-label
spec:
  containers:
  - name: test-nginx-container
    image: nginx:latest
    ports:
    - containerPort: 80
```

Replicaset의 동작 원리

Step 2: Pod 생성

준비된 YAML(rs-nginx-2.yaml)로 Pod를 생성하고, 생성된 Pod의 label을 확인합니다.

```
kubectl apply -f rs-nginx-2.yaml  
kubectl get pods --show-labels
```

```
ubuntu@k8s-master:~/rs$ kubectl apply -f rs-nginx-2.yaml  
pod/test-nginx-pod created
```

```
ubuntu@k8s-master:~/rs$
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
test-nginx-pod	1/1	Running	0	23s	app=test-nginx-pods-label

Replicaset의 동작 원리

Step 3: Replicaset 생성

Replicaset을 생성하고,

```
kubectl apply -f rs-nginx.yaml
```

```
ubuntu@k8s-master:~/rs$ kubectl apply -f rs-nginx.yaml
replicaset.apps/replicaset-nginx created
```

생성된 Pod를 확인합니다.

```
kubectl get pods --show-labels
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
replicaset-nginx-8hps7	0/1	ContainerCreating	0	4s	app=test-nginx-pods-label
replicaset-nginx-lgrz6	0/1	ContainerCreating	0	4s	app=test-nginx-pods-label
replicaset-nginx-lhcfw	1/1	Running	0	4s	app=test-nginx-pods-label
replicaset-nginx-m94kf	1/1	Running	0	4s	app=test-nginx-pods-label
test-nginx-pod	1/1	Running	0	2m18s	app=test-nginx-pods-label

Replicaset의 동작 원리

Step 4: 수동으로 Pod 삭제하기

수동으로 생성한 pod를 삭제합니다.

```
kubectl delete pods test-nginx-pod
```

```
ubuntu@k8s-master:~/rs$ kubectl delete pods test-nginx-pod
pod "test-nginx-pod" deleted
```

test-nginx-pod 레이블을 갖는 Pod의 개수가 4개로 줄어들었기 때문에 새로운 Pod를 생성합니다.

```
kubectl get pods --show-labels
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
replicaset-nginx-8hps7	1/1	Running	0	3m42s	app=test-nginx-pods-label
replicaset-nginx-8lssf	1/1	Running	0	9s	app=test-nginx-pods-label
replicaset-nginx-lgrz6	1/1	Running	0	3m42s	app=test-nginx-pods-label
replicaset-nginx-lhcfw	1/1	Running	0	3m42s	app=test-nginx-pods-label
replicaset-nginx-m94kf	1/1	Running	0	3m42s	app=test-nginx-pods-label

Replicaset의 동작 원리

Step 5: Pod의 레이블 삭제

edit로 Replicaset이 생성해 놓은 Pod의 레이블을 삭제합니다.

```
kubectl edit pods <POD>
```

```
ubuntu@k8s-master:~/rs$ kubectl edit pods replicaset-nginx-8hps7
```

labels와 app을 삭제합니다.

```
12  creationTimestamp: "2023-04-10T04:51:35Z"
13  generateName: replicaset-nginx-
14  
15
16  name: replicaset-nginx-8hps7
17  namespace: default
18  ownerReferences:
19  - apiVersion: apps/v1
20    blockOwnerDeletion: true
```

Replicaset의 동작 원리

Step 5: Pod의 레이블 삭제

Pod의 목록을 확인합니다.

```
ubuntu@k8s-master:~/rs$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
replicaset-nginx-8hps7	1/1	Running	0	10m	<none>
replicaset-nginx-8lssf	1/1	Running	0	6m38s	app=test-nginx-pods-label
replicaset-nginx-kwrnk	1/1	Running	0	8s	app=test-nginx-pods-label
replicaset-nginx-lgrz6	1/1	Running	0	10m	app=test-nginx-pods-label
replicaset-nginx-lhcfw	1/1	Running	0	10m	app=test-nginx-pods-label
replicaset-nginx-m94kf	1/1	Running	0	10m	app=test-nginx-pods-label

Replicaset의 동작 원리

Step 6: Replicaset 삭제

Replicaset을 삭제합니다.

```
kubectl delete rs replicaset-nginx
```

```
ubuntu@k8s-master:~/rs$ kubectl delete rs replicaset-nginx
replicaset.apps "replicaset-nginx" deleted
```

레이블을 삭제한 Pod는 Replicaset과 일치하지 않으므로 Replicaset에 의해 관리되지 않습니다.

```
kubectl get pods --show-labels
```

```
ubuntu@k8s-master:~/rs$ kubectl get pods --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
replicaset-nginx-8hps7	1/1	Running	0	12m	<none>

Lab 05.

Deployment

Replicaset 사용하기

Step 1: YAML 파일 준비

kind 항목이 Deployment인 YAML 파일을 준비합니다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: test-nginx
  template:
    metadata:
      name: test-nginx-po
      labels:
        app: test-nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.10
        ports:
        - containerPort: 80
```


Replicaset 사용하기

Step 2: Deployment 생성

준비한 YAML 파일(deploy-nginx)로 Deployment를 생성하고,

```
cd ../deploy
kubectl apply -f deploy-nginx.yaml
```

```
ubuntu@k8s-master:~/deploy$ k apply -f deploy-nginx.yaml
deployment.apps/test-nginx-deploy created
```

생성된 Deployment를 확인합니다.

```
kubectl get deployment
```

```
ubuntu@k8s-master:~/deploy$ k get deploy
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
test-nginx-deploy   3/3      3              3            62s
```

Replicaset 사용하기

Step 3: Replicaset, Pod 확인

생성된 Replicaset과 Pod를 각각 확인합니다.

```
kubectl get replicaset,pods
```

```
ubuntu@k8s-master:~/deploy$ kubectl get rs,po
```

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/test-nginx-deploy-c8dd6c587	3	3	3	17s

NAME	READY	STATUS	RESTARTS	AGE
pod/test-nginx-deploy-c8dd6c587-62z8t	1/1	Running	0	17s
pod/test-nginx-deploy-c8dd6c587-8mw5t	1/1	Running	0	17s
pod/test-nginx-deploy-c8dd6c587-fqsn6	1/1	Running	0	17s

Replicaset 사용하기

Step 4: Deployment 삭제

Deployment를 삭제하고, Replicaset과 Pod를 확인합니다.

```
kubectl delete deploy test-nginx-deploy
```

```
ubuntu@k8s-master:~/deploy$ k delete deploy test-nginx-deploy  
deployment.apps "test-nginx-deploy" deleted
```

Deployment를 삭제하면, Replicaset과 pod 또한 함께 삭제됩니다.

```
kubectl get replicaset,pods
```

```
ubuntu@k8s-master:~/deploy$ k get rs,po  
No resources found in default namespace.
```

애플리케이션 버전 업데이트 배포

Step 1: Deployment 생성

Deployment를 생성합니다.

```
kubectl apply -f deploy-nginx.yaml
```

```
ubuntu@k8s-master:~/deploy$ k apply -f deploy-nginx.yaml
deployment.apps/test-nginx-deploy created
```

생성된 Replicaset과 pod를 확인합니다.

```
kubectl get replicaset,pods
```

```
ubuntu@k8s-master:~/deploy$ k get rs,po
```

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/test-nginx-deploy-c8dd6c587	3	3	3	4s

NAME	READY	STATUS	RESTARTS	AGE
pod/test-nginx-deploy-c8dd6c587-8kthd	1/1	Running	0	4s
pod/test-nginx-deploy-c8dd6c587-d9pn2	1/1	Running	0	4s
pod/test-nginx-deploy-c8dd6c587-dr1x7	1/1	Running	0	4s

애플리케이션 버전 업데이트 배포

Step 2: Pod 이미지 버전 변경

생성한 Pod의 이미지를 변경합니다.

```
kubectl set image deploy test-nginx-deploy nginx=nginx:1.19
```

```
ubuntu@k8s-master:~/deploy$ k set image deploy test-nginx-deploy nginx=nginx:1.19  
deployment.apps/test-nginx-deploy image updated
```

수정된 버전을 확인합니다.

```
kubectl describe pods <POD>
```

```
Containers:  
  nginx:  
    Container ID:   containerd://cef73eb57778faff3248272282924e4c162f800a4c768d94  
    Image:          nginx:1.19  
    Image ID:       docker.io/library/nginx@sha256:df13abe416e37eb3db4722840dd479  
    Port:           80/TCP  
    Host Port:      0/TCP  
    State:          Running  
      Started:      Mon, 05:36:46 +0000  
    Ready:          True  
    Restart Count:  0
```


애플리케이션 버전 업데이트 배포

Step 3: 새롭게 생성된 Pod 확인

다시 Pod를 확인해보면, 새롭게 생성된 Pod들이 있습니다.

```
kubectl get pods
```

```
ubuntu@k8s-master:~/deploy$ k get po
```

NAME		READY	STATUS	RESTARTS	AGE
test-nginx-deploy-6d79458468-57rvd		1/1	Running	0	2m50s
test-nginx-deploy-6d79458468-8pqgh		1/1	Running	0	2m34s
test-nginx-deploy-6d79458468-rbzrb		1/1	Running	0	2m42s

애플리케이션 버전 업데이트 배포

Step 4: Replicaset 확인

Replicaset의 목록을 출력합니다.

```
kubectl get replicaset
```

```
ubuntu@k8s-master:~/deploy$ k get rs
```

NAME	DESIRED	CURRENT	READY	AGE
test-nginx-deploy-6d79458468	3	3	3	8m58s
test-nginx-deploy-c8dd6c587	0	0	0	13m

애플리케이션 버전 업데이트 배포

Step 5: Replicaset Rollback

revision 정보를 확인합니다.

```
kubectl rollout history deployment test-nginx-deploy
```

```
ubuntu@k8s-master:~/deploy$ k rollout history deployment test-nginx-deploy
deployment.apps/test-nginx-deploy
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

애플리케이션 버전 업데이트 배포

Step 6: 이전 버전으로 되돌리기

이전 버전의 Replicaset으로 되돌립니다.

```
kubectl rollout undo deploy test-nginx-deploy --to-revision=1
```

```
ubuntu@k8s-master:~/deploy$ k rollout undo deploy test-nginx-deploy --to-revision=1
deployment.apps/test-nginx-deploy rolled back
```

Replicaset의 목록을 확인합니다.

```
kubectl get replicaset
```

```
ubuntu@k8s-master:~/deploy$ k get rs
```

NAME	DESIRED	CURRENT	READY	AGE
test-nginx-deploy-6d79458468	0	0	0	21m
test-nginx-deploy-c8dd6c587	3	3	3	26m

새롭게 생성했던 Replicaset의 Pod는 0으로 줄어들었고, 처음 생성했던 Replicaset이 다시 3개의 Pod를 생성했습니다.

애플리케이션 버전 업데이트 배포

Step 7: revision 정보와 활성화된 Replicaset 이름 확인

Deployment의 정보를 출력합니다.

```
kubectl describe deploy test-nginx-deploy
```

```
ubuntu@k8s-master:~/deploy$ k describe deploy test-nginx-deploy
Name: test-nginx-deploy
Namespace: default
CreationTimestamp: Thu, 05 Jun 2025 05:05:31 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 3
Selector: app=test-nginx
Replicas: 3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
```

Lab 06.

Service

ClusterIP 타입

Step 1: Service 생성

Service를 정의하는 YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Service
metadata:
  name: svc-clusterip
spec:
  ports:
    - name: web-port
      port: 8080
      targetPort: 80
  selector:
    app: webserver
  type: ClusterIP
```

Step 1: Service 생성

준비한 YAML 파일(svc-clusterip.yaml)로 Service를 생성합니다.

```
cd svc
kubectl apply -f svc-clusterip.yaml
```

```
ubuntu@k8s-master:~/svc$ k apply -f svc-clusterip.yaml
service/svc-clusterip created
```

생성된 Service의 목록을 확인합니다.

```
kubectl get services
```

```
ubuntu@k8s-master:~/svc$ k get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	27d
svc-clusterip	ClusterIP	10.106.38.57	<none>	8080/TCP	85s

Step 2: Pod 생성

YAML 파일을 준비합니다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hostname-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      name: test-webserver
    labels:
      app: webserver
    spec:
      containers:
        - name: test-webserver
          image: alicek106/rr-test:echo-hostname
          ports:
            - containerPort: 80
```

Step 2: Pod 생성

YAML 파일(deploy.yaml)을 이용해 Deployment를 생성합니다.

```
kubectl apply -f deploy.yaml
```

```
ubuntu@k8s-master:~/svc$ k apply -f deploy.yaml
deployment.apps/hostname-deployment created
```

생성된 Pod를 확인합니다.

```
kubectl get pods
cd ..
```

```
ubuntu@k8s-master:~/svc$ k get po
```

NAME	READY	STATUS	RESTARTS	AGE
hostname-deployment-cb987f77-br8d4	1/1	Running	0	57s
hostname-deployment-cb987f77-fnq97	1/1	Running	0	57s
hostname-deployment-cb987f77-rb811	1/1	Running	0	57s

Step 3: Service를 통해 Pod에 접근
생성한 Service의 ClusterIP를 확인합니다.

```
kubectl get pods,services
```

```
ubuntu@k8s-master:~$ k get po,svc
```

NAME	READY	STATUS	RESTARTS	AGE
pod/hostname-deployment-cb987f77-br8d4	1/1	Running	0	2m2s
pod/hostname-deployment-cb987f77-fnq97	1/1	Running	0	2m2s
pod/hostname-deployment-cb987f77-rb8l1	1/1	Running	0	2m2s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	27d
service/svc-clusterip	ClusterIP	10.106.38.57	<none>	8080/TCP	14m

Step 3: Service를 통해 Pod에 접근

Service의 ClusterIP와 Port를 통해 Pod에 접근합니다.

```
ubuntu@k8s-master:~$ curl 10.106.38.57:8080
<!DOCTYPE html>
<meta charset="utf-8" />
<link rel="stylesheet" type="text/css" href="./css/layout.css" />
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.2/

<div class="form-layout">
    <blockquote>
        <p>Hello,  hostname-deployment-cb987f77-fnq97</p>
    </blockquote>
</div>
```


Step 4: Service 이름으로 접근

서비스 이름으로 접근하기 위해 임시 Pod를 생성합니다.

```
kubectl run -i --tty --rm debug --image=alicek106/ubuntu:curl --restart=Never -- bash
```

```
ubuntu@k8s-master:~$ k run -i --tty --rm debug --image=alicek106/ubuntu:curl --restart=Never -- bash
If you don't see a command prompt, try pressing enter.
root@debug:/#
```

Service 이름으로 접근합니다. 접근 후 exit 명령어로 빠져나옵니다.

```
curl svc-clusterip:8080
```

```
root@debug:/# curl svc-clusterip:8080
<!DOCTYPE html>
<meta charset="utf-8" />
<link rel="stylesheet" type="text/css" href="./css/layout.css" />
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.2/

<div class="form-layout">
  <blockquote>
    <p>Hello,  hostname-deployment-cb987f77-fnq97</p>
  </blockquote>
```

ClusterIP 타입

Step 5: Service 삭제

생성한 Service를 삭제합니다.

```
cd svc  
kubectl delete -f svc-clusterip.yaml
```

```
ubuntu@k8s-master:~/svc$ k delete -f svc-clusterip.yaml  
service "svc-clusterip" deleted
```


NodePort 타입

Step 1: Service 생성

YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nodeport
spec:
  ports:
    - name: web-port
      port: 8080
      targetPort: 80
  selector:
    app: webserver
  type: NodePort
```

NodePort 타입

Step 1: Service 생성

준비한 YAML 파일(svc-nodeport.yaml)로 NodePort 타입의 Service를 생성합니다.

```
kubectl apply -f svc-nodeport.yaml
```

```
ubuntu@k8s-master:~/svc$ k apply -f svc-nodeport.yaml
service/svc-nodeport created
```

생성된 Service를 확인합니다.

```
kubectl get services
```

```
ubuntu@k8s-master:~$ k get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	27d
svc-nodeport	NodePort	10.108.105.73	<none>	8080:31896/TCP	4s

NodePort 타입

Step 2: NodePort를 이용한 Pod 접근

클러스터의 모든 노드에 내부 IP를 확인합니다.

```
kubectl get nodes -o wide
```

```
ubuntu@k8s-master:~$ k get no -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
k8s-master	Ready	control-plane	27d	v1.30.13	172.31.5.174	<none>
k8s-worker-1	Ready	<none>	27d	v1.30.13	172.31.13.90	<none>
k8s-worker-2	Ready	<none>	27d	v1.30.13	172.31.7.230	<none>

Node의 Internal IP와 Node Port를 통해 접근합니다.(Node Port는 **이전 페이지**에서 확인합니다.)

```
curl <Node의 Internal IP:Node Port>
```

```
ubuntu@k8s-master:~/svc$ curl 172.31.13.90:31896
<!DOCTYPE html>
<meta charset="utf-8" />
<link rel="stylesheet" type="text/css" href="./css/layout.css" />
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.2/

<div class="form-layout">
  <blockquote>
    <p>Hello, hostname-deployment-cb987f77-45z96</p>
  </blockquote>
```

LoadBalancer 타입

Step 1: Service 생성

YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Service
metadata:
  name: svc-lb
spec:
  ports:
    - name: web-port
      port: 80
      targetPort: 80
  selector:
    app: webserver
  type: LoadBalancer
```

LoadBalancer타입

Step 1: Service 생성

준비된 YAML 파일(svc-lb.yaml)로 Service를 생성합니다.

```
kubectl apply -f svc-lb.yaml
```

```
ubuntu@k8s-master:~/svc$ k apply -f svc-lb.yaml
service/svc-lb created
```

생성한 Service를 확인합니다.

```
kubectl get services
```

```
ubuntu@k8s-master:~$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	117m
svc-lb	LoadBalancer	10.109.57.103	<pending>	80:31548/TCP	25s

Lab 07.

Namespace

Namespace 확인하기

Step 1: Namespace 확인

namespace의 목록을 확인합니다.

```
cd ns  
kubectl get namespaces
```

```
ubuntu@k8s-master:~/ns$ k get ns  
NAME                STATUS    AGE  
default             Active    27d  
kube-node-lease     Active    27d  
kube-public         Active    27d  
kube-system         Active    27d
```

Namespace 확인하기

Step 2: Namespace에 생성된 리소스 확인

namespace에 생성된 리소스를 확인합니다.

```
kubectl get pods --namespace default
```

```
ubuntu@k8s-master:~/ns$ k get po -n default  
No resources found in default namespace.
```

다른 네임스페이스(kube-system)의 pod를 확인해봅니다.

```
kubectl get pods --namespace kube-system
```

Namespace 사용하기

Step 1: Namespace 생성

YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Namespace
metadata:
  name: production
```

준비된 YAML 파일(prod-ns.yaml)로 네임스페이스를 생성합니다.

```
kubectl apply -f prod-ns.yaml
```

```
ubuntu@k8s-master:~/ns$ k apply -f prod-ns.yaml
namespace/production created
```

생성한 네임스페이스의 목록을 확인합니다.

```
kubectl get namespace | grep production
```

Namespace 사용하기

Step 2: 네임스페이스에 리소스 생성

YAML 파일을 준비합니다.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-ns
  namespace: production
spec:
  ...
---
apiVersion: v1
kind: Service
metadata:
  name: svc-clusterip-ns
  namespace: production
spec:
  ...
```

Namespace 사용하기

Step 2: 네임스페이스에 리소스 생성

YAML 파일(deploy-svc-ns.yaml)로 리소스를 생성합니다.

```
kubectl apply -f deploy-svc-ns.yaml
```

```
ubuntu@k8s-master:~/ns$ k apply -f deploy-svc-ns.yaml
deployment.apps/hostname-deployment-ns created
service/hostname-svc-clusterip-ns created
```

생성한 리소스를 확인합니다.

```
kubectl get po,svc -n production
```

```
ubuntu@k8s-master:~/ns$ k get po,svc -n production
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-ns-cb987f77-hr7mk	1/1	Running	0	64s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/svc-clusterip-ns	ClusterIP	10.97.224.229	<none>	8080/TCP

Namespace의 서비스에 접근

Step 1: 임시 Pod 생성

테스트를 위해 임시 Pod를 생성합니다.

```
kubectl run -i --tty --rm debug --image=alicek106/ubuntu:curl --restart=Never -- bash
```

```
ubuntu@k8s-master:~/ns$ k run -i --tty --rm debug --image=alicek106/ubuntu:curl --restart=Never -- bash
If you don't see a command prompt, try pressing enter.
root@debug:/#
```


Namespace의 서비스에 접근

Step 2: Service 접근

네임스페이스에 존재하는 Service에 Service 이름으로 접근해봅니다.

```
curl svc-clusterip-ns:8080
```

```
root@debug:/# curl svc-clusterip-ns:8080
curl: (6) Could not resolve host: svc-clusterip-ns
```

Service 이름 뒤에 네임스페이스 이름을 붙여 다른 네임스페이스의 서비스에 접근합니다.

```
curl svc-clusterip-ns.production.svc:8080
```

```
root@debug:/# curl svc-clusterip-ns.production.svc:8080 --silent | grep Hello
<p>Hello, deployment-ns-cb987f77-hr7mk</p> </blockquote>
```

접근이 되었음을 확인한 후에 'exit' 명령어로 빠져나옵니다.

Namespace의 서비스에 접근

Step 3: 리소스 정리

네임스페이스를 삭제합니다. 네임스페이스를 삭제하면, 네임스페이스에 존재하는 모든 리소스 또한 함께 삭제됩니다.

```
kubectl delete namespace production
```

```
ubuntu@k8s-master:~/ns$ k delete ns production  
namespace "production" deleted
```

Lab 08.

ConfigMap, Secret

ConfigMap 생성하기

Step 1: ConfigMap 생성하기

kubectl create configmap을 사용해 ConfigMap을 생성합니다.

```
kubectl create configmap log-level-configmap --from-literal LOG_LEVEL=DEBUG  
kubectl create configmap start-k8s --from-literal k8s=kubernetes --from-literal  
container=docker
```

```
ubuntu@k8s-master:~/cm$ k create cm log-level-configmap --from-literal LOG_LEVEL=DEBUG  
configmap/log-level-configmap created
```

```
ubuntu@k8s-master:~$ kubectl create configmap start-k8s --from-literal k8s=kubernetes --from-literal container=docker  
configmap/start-k8s created
```

ConfigMap 생성하기

Step 2: ConfigMap 확인

ConfigMap에 저장된 설정 값을 확인합니다.

```
kubectl get configmap
```

```
ubuntu@k8s-master:~/cm$ k get cm
NAME                DATA  AGE
kube-root-ca.crt    1      27d
log-level-configmap  1      11h
start-k8s           2      11h
```

ConfigMap의 세부 정보를 확인합니다.

```
kubectl describe configmap log-level-configmap
```

```
ubuntu@k8s-master:~$ kubectl describe configmap log-level-configmap
Name:         log-level-configmap
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
LOG_LEVEL:
```

ConfigMap 생성하기

Step 2: ConfigMap 확인

ConfigMap을 YAML 형식으로 확인합니다.

```
kubectl get configmap log-level-configmap -o yaml
```

```
ubuntu@k8s-master:~/cm$ kubectl get configmap log-level-configmap -o yaml
apiVersion: v1
data:
  LOG_LEVEL: DEBUG
kind: ConfigMap
metadata:
  creationTimestamp: "2023-07-10T06:44:23Z"
  name: log-level-configmap
  namespace: default
  resourceVersion: "28322"
  uid: 50bc0af7-c0e1-4ca5-a9ed-a4ed929284fc
```


ConfigMap을 Pod로 가져오기: 첫 번째 방법

Step 1: YAML 파일 준비하기

configmap의 값을 환경 변수로 사용하는 pod를 생성하기 위한 YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Pod
metadata:
  name: container-env-example
spec:
  containers:
    - name: my-container
      image: busybox
      args: ['tail', '-f', '/dev/null']
      envFrom:
        - configMapRef:
            name: log-level-configmap
        - configMapRef:
            name: start-k8s
```

YAML 파일은 log-level-configmap와 start-k8s을 configmap으로부터 값을 가져와 환경 변수를 생성하도록 설정했습니다.

ConfigMap을 Pod로 가져오기: 첫 번째 방법

Step 2: Pod를 생성

준비한 YAML 파일(all-env-from-configmap.yaml)로 Pod를 생성합니다.

```
kubectl apply -f all-env-from-configmap.yaml
```

```
ubuntu@k8s-master:~/cm$ k apply -f all-env-from-configmap.yaml
pod/container-env-example unchanged
```

ConfigMap을 Pod로 가져오기: 첫 번째 방법

Step 3: 환경 변수 출력

env로 환경 변수를 출력합니다.

```
kubectl exec container-env-example -- env
```

```
ubuntu@k8s-master:~/cm$ k exec container-env-example -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=container-env-example
LOG_LEVEL=DEBUG
container=docker
k8s=kubernetes
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

ConfigMap을 Pod로 가져오기: 첫 번째 방법

Step 4: 특정 데이터만 가져오기

valueFrom과 configMapKeyRef를 사용해 여러 개의 데이터 중에서 특정 데이터를 선택해 환경 변수로 가져옵니다.

```
env:
- name: ENV_KEYNAME_1      # (1.1) 컨테이너에 새롭게 등록될 환경 변수 이름
  valueFrom:
    configMapKeyRef:
      name: log-level-configmap
      key: LOG_LEVEL
- name: ENV_KEYNAME_2      # (1.2) 컨테이너에 새롭게 등록될 환경 변수 이름
  valueFrom:
    configMapKeyRef:
      name: start-k8s        # (2) 참조할 컨피그맵의 이름
      key: k8s              # (3) 가져올 데이터 값의 키
                           # 최종 결과 -> ENV_KEYNAME_2=$(k8s 키에 해당하는 값)
                           # ENV_KEYNAME_2=kubernetes
```

ConfigMap을 Pod로 가져오기: 첫 번째 방법

Step 4: 특정 데이터만 가져오기

준비한 YAML 파일(selective-env-from-configmap.yaml)로 Pod를 생성합니다.

```
kubectl apply -f selective-env-from-configmap.yaml
```

```
ubuntu@k8s-master:~/cm$ k apply -f selective-env-from-configmap.yaml
pod/container-selective-env-example unchanged
```

Pod에 설정된 환경 변수를 확인합니다.

```
kubectl exec container-selective-env-example -- env | grep ENV
```

```
ubuntu@k8s-master:~/cm$ k exec container-selective-env-example -- env | grep ENV
ENV_KEYNAME_1=DEBUG
ENV_KEYNAME_2=kubernetes
```

ConfigMap을 Pod로 가져오기: 두 번째 방법

Step 1: YAML 파일 준비하기

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
    - name: my-container
      image: busybox
      args: [ "tail", "-f", "/dev/null" ]
      volumeMounts:
        - name: configmap-volume          # volumes에서 정의한 컨피그맵 볼륨 이름
          mountPath: /etc/config          # 컨피그맵의 데이터가 위치할 경로
  volumes:
    - name: configmap-volume              # 컨피그맵 볼륨 이름
      configMap:
        name: start-k8s
```


ConfigMap을 Pod로 가져오기: 두 번째 방법

Step 2: Pod 생성

준비한 YAML 파일(volume-mount-configmap.yaml)로 Pod를 생성합니다.

```
kubectl apply -f volume-mount-configmap.yaml
```

```
ubuntu@k8s-master:~/cm$ k apply -f volume-mount-configmap.yaml
pod/configmap-volume-pod created
```

ConfigMap을 Pod로 가져오기: 두 번째 방법

Step 3: 디렉터리 조회

디렉터리를 조회해보면, configmap에 저장되어 있던 두 개의 key:value 데이터가 파일로 존재합니다.

```
kubectl exec configmap-volume-pod -- ls /etc/config
```

```
ubuntu@k8s-master:~/cm$ k exec configmap-volume-pod -- ls /etc/config
container
k8s
```

ConfigMap을 Pod로 가져오기: 두 번째 방법

Step 4: 원하는 데이터만 파일로 가져오기

YAML 파일을 준비합니다.

```

apiVersion: v1
kind: Pod
metadata:
  name: selective-cm-volume-pod
spec:
  containers:
    - name: my-container
      image: busybox
      args: [ "tail", "-f", "/dev/null" ]
      volumeMounts:
        - name: configmap-volume
          mountPath: /etc/config          # 마운트되는 위치
  volumes:
    - name: configmap-volume
      configMap:
        name: start-k8s
        items:                             # configmap에서 가져올 k:v의 목록을 나열
          - key: k8s                       # k8s라는 키에 대응하는 값을 가져옴
            path: k8s_fullname             # 최종 파일 이름은 k8s_fullname이 됨

```

ConfigMap을 Pod로 가져오기: 두 번째 방법

Step 4: 원하는 데이터만 파일로 가져오기

YAML 파일(selective-volume-configmap.yaml)로 Pod를 생성합니다.

```
kubectl apply -f selective-volume-configmap.yaml
```

```
ubuntu@k8s-master:~/cm$ k apply -f selective-volume-configmap.yaml
pod/selective-cm-volume-pod created
```

내부 디렉터리를 살펴보면 해당 파일이 마운트되어, k8s라는 키에 해당하는 값이 k8s_fullname이라는 파일로 Pod 내부에 존재합니다.

```
kubectl exec selective-cm-volume-pod -- ls /etc/config
```

```
ubuntu@k8s-master:~/cm$ k exec selective-cm-volume-pod -- ls /etc/config
k8s_fullname
```

```
kubectl exec selective-cm-volume-pod -- cat /etc/config/k8s_fullname
```

```
ubuntu@k8s-master:~/cm$ k exec selective-cm-volume-pod -- cat /etc/config/k8s_fullname
kubernetesubuntu@k8s-master:~/cm$
```

파일로 ConfigMap 생성하기

Step 1: 파일 생성

파일을 생성합니다.

```
echo Hello, Kubernetes! >> index.html
```

```
ubuntu@k8s-master:~/cm$ echo Hello, Kubernetes! >> index.html
```

파일로 ConfigMap 생성하기

Step 2: 생성한 파일을 configmap으로 가져오기

kubectl create configmap <configmap 이름> --from-file <파일 이름> 으로 파일을 configmap으로 가져올 수 있습니다.

```
kubectl create configmap index-file --from-file index.html
```

```
ubuntu@k8s-master:~/cm$ k create cm index-file --from-file index.html
configmap/index-file created
```


파일로 ConfigMap 생성하기

Step 3: ConfigMap 확인

kubectl create configmap <configmap 이름> --from-file <파일 이름> 으로 파일을 configmap으로 가져올 수 있습니다.

```
kubectl describe configmap index-file
```

```
ubuntu@k8s-master:~/cm$ k describe cm index-file
```

```
Name:          index-file
Namespace:     default
Labels:        <none>
Annotations:   <none>
```

```
Data
```

```
====
```

```
index.html:
```

```
----
```

```
Hello, Kubernetes!
```

Step 1: Secret 생성하기

my-password라는 이름의 secret을 생성합니다.

```
kubectl create secret generic my-password --from-literal password=dkagh123.
```

```
ubuntu@k8s-master:~/cm$ k create secret generic my-password --from-literal password=dkagh123.  
secret/my-password created
```

secret은 --from-file 이나 --from-env-file 옵션을 이용해 파일로부터 생성할 수도 있습니다.

```
echo mypassword > pw1 && echo yourpassword > pw2
```

```
kubectl create secret generic our-password --from-file pw1 --from-file pw2
```

```
ubuntu@k8s-master:~/cm$ echo mypassword > pw1 && echo yourpassword > pw2  
ubuntu@k8s-master:~/cm$ k create secret generic our-password --from-file pw1 --from-file pw2  
secret/our-password created
```

Step 2: 생성한 Secret 확인

생성한 secret을 확인합니다.

```
kubectl get secrets
```

```
ubuntu@k8s-master:~/cm$ k get secrets
NAME             TYPE      DATA  AGE
my-password      Opaque    1       6m21s
our-password     Opaque    2       2m32s
```

Secret의 상세한 내용을 확인합니다. 데이터는 노출되지 않습니다.

```
kubectl describe secrets my-password
```

```
ubuntu@k8s-master:~/cm$ k describe secrets my-password
Name:          my-password
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  Opaque

Data
====
password:  9 bytes
```

Step 2: 생성한 Secret 확인

password가 base64로 인코딩되어 저장됩니다.

```
kubectl get secrets my-password -o yaml
```

```
ubuntu@k8s-master:~/cm$ k get secrets my-password -o yaml
apiVersion: v1
data:
  password: ZGthZ2gxMjMu
kind: Secret
metadata:
  creationTimestamp: "2025-06-17T02:03:44Z"
  name: my-password
  namespace: default
  resourceVersion: "211893"
  uid: 822e8b31-9f3f-43a2-9bd1-25da612636b6
type: Opaque
```

Secret 만들기

Step 2: 생성한 Secret 확인

디코딩하여 패스워드를 확인할 수 있습니다.

```
echo <password(encoding)> | base64 -d
```

```
ubuntu@k8s-master:~/cm$ echo ZGthZ2gxMjMu | base64 -d  
dkagh123.ubuntu@k8s-master:~/cm$
```

Private Registry 접근 시 사용하는 인증 설정

Step 1: Secret 생성하기

로그인 인증 정보를 명시해 Secret을 생성합니다.

```
kubectl create secret docker-registry registry-auth-by-cmd \
--docker-username=joteacher \
--docker-password=dkagh123.
```

```
ubuntu@k8s-master:~/cm$ kubectl create secret docker-registry registry-auth-by-cmd \
> --docker-username=joteacher \
> --docker-password=dkagh123.
secret/registry-auth-by-cmd created
```


Private Registry 접근 시 사용하는 인증 설정

Step 1: Secret 생성하기

--docker-server 옵션에 다른 레지스트리의 주소나 도메인 이름을 입력할 수도 있습니다.

```
kubectl create secret docker-registry registry-auth-registry ₩  
--docker-username=joteacher ₩  
--docker-password=dkagh123. ₩  
--docker-server=joteacher.registry.com
```

```
ubuntu@k8s-master:~/cm$ kubectl create secret docker-registry registry-auth-registry \  
> --docker-username=joteacher \  
> --docker-password=dkagh123. \  
> --docker-server=joteacher.registry.com  
secret/registry-auth-registry created
```

Private Registry 접근 시 사용하는 인증 설정

Step 2: 생성한 Secret 목록 확인

```
kubectl get secrets
```

```
ubuntu@k8s-master:~/cm$ k get secrets
```

NAME	TYPE	DATA	AGE
my-password	Opaque	1	39m
our-password	Opaque	2	35m
registry-auth-by-cmd	kubernetes.io/dockerconfigjson	1	9m11s
registry-auth-registry	kubernetes.io/dockerconfigjson	1	57s

Private Registry 접근 시 사용하는 인증 설정

Step 3: Secret 사용

생성한 Secret은 Deployment나 Pod에서 사설 레지스트리로부터 이미지를 받아올 때 사용할 수 있습니다.

```
spec:
  containers:
    - name: test-container
      image: <여러분의 이미지 이름을 입력하세요.>
      args: ['tail', '-f', '/dev/null']
      imagePullSecrets:
        - name: registry-auth-registry
```

Lab 09.

emptyDir, hostPath, PV/PVC

Local Volume: emptyDir

Step 1: YAML 파일 준비

emptyDir 볼륨을 사용하여 두 컨테이너 간에 데이터를 공유하는 YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
    - name: content-creator
      image: alpine
      args: ["tail", "-f", "/dev/null"]
      volumeMounts:
        - name: my-emptydir-volume
          mountPath: /data

    - name: apache-webserver
      image: httpd:2
      volumeMounts:
        - name: my-emptydir-volume
          mountPath: /usr/local/apache2/htdocs/

  volumes:
    - name: my-emptydir-volume
      emptyDir: {}
```

Local Volume: emptyDir

Step 2: Pod 내 컨테이너 간 임시 데이터 공유

준비한 YAML 파일(emptydir-pod.yaml)로 Pod를 생성합니다.

```
cd volume
```

```
kubectl apply -f emptydir-pod.yaml
```

```
ubuntu@k8s-master:~/volume$ k apply -f emptydir-pod.yaml
pod/emptydir-pod created
```

content-creator 컨테이너 내부로 들어가 /data 디렉터리에 웹 콘텐츠를 생성합니다.

```
kubectl exec -it emptydir-pod -c content-creator -- sh
```

```
echo hello, k8s > /data/test.html
exit
```

```
ubuntu@k8s-master:~/volume$ k exec -it emptydir-pod -c content-creator -- sh
/ # echo hello, k8s > /data/test.html
/ # exit
```

※ 주의: Pod가 생성되는 중이라면 컨테이너 내부로 들어갈 수 없습니다. 접속이 안되면 STATUS를 확인하세요.

Local Volume: emptyDir

Step 2: Pod 내 컨테이너 간 임시 데이터 공유

apache-webserver 컨테이너의 셸로 이동합니다.

```
kubectl exec -it emptydir-pod \
--container apache-webserver -- /bin/bash
```

```
ubuntu@k8s-master:~/volume$ k exec -it emptydir-pod \
> --container apache-webserver -- /bin/bash
root@emptydir-pod:/usr/local/apache2#
```

content-creator 컨테이너에서 작성한 test.html 파일을 확인합니다.

```
cd htdocs/
cat test.html
```

```
root@emptydir-pod:/usr/local/apache2# cd htdocs/
root@emptydir-pod:/usr/local/apache2/htdocs# cat test.html
hello, k8s
```

컨테이너를 빠져나옵니다.

```
exit
```

Local Volume: hostPath

Step 1: YAML 파일 준비

hostPath 볼륨을 사용하여 호스트 노드의 /tmp를 Pod의 /etc/data 디렉터리에 마운트하는 YAML 파일을 준비합니다.

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
    - name: container
      image: busybox
      args: [ "tail", "-f", "/dev/null" ]
      volumeMounts:
        - name: hostpath-volume
          mountPath: /etc/data
  volumes:
    - name: hostpath-volume
      hostPath:
        path: /tmp
```

Local Volume: hostPath

Step 2: Pod 생성

준비한 YAML 파일(hostpath-pod.yaml)로 Pod를 생성합니다.

```
kubectl apply -f hostpath-pod.yaml
```

```
ubuntu@k8s-master:~/volume$ k apply -f hostpath-pod.yaml
pod/hostpath-pod created
```

Pod가 생성된 worker node를 확인합니다.

```
Kubectl get pod -o wide
```

```
ubuntu@k8s-master:~/volume$ k get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
emptydir-pod	2/2	Running	0	37m	192.168.140.61	k8s-worker-2
hostpath-pod	1/1	Running	0	3m32s	192.168.230.60	k8s-worker-1

Local Volume: hostPath

Step 3: 파일 공유 확인

hostpath-pod에서 파일을 생성합니다.

```
kubectl exec -it hostpath-pod -- touch /etc/data/test_data
```

```
ubuntu@k8s-master:~/volume$ k exec -it hostpath-pod -- touch /etc/data/test_data
```

Pod가 생성된 **worker node**에 접속(프롬프트 반드시 확인)해서 컨테이너가 생성한 데이터를 확인합니다.

```
ls /tmp/test_data
```

```
ubuntu@k8s-worker-1:~$ ls /tmp/test_data  
/tmp/test_data
```

Step 1: PV 생성

PV를 정의한 파일을 준비합니다.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv
spec:
  capacity:
    storage: 1G
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /pv
```

준비된 YAML 파일(pv.yaml)로 PV를 생성합니다. **생성은 Master Node에서 진행합니다.**

```
kubectl create -f pv.yaml
```

```
ubuntu@k8s-master:~/volume$ k create -f pv.yaml
persistentvolume/pv created
```

Step 2: PVC 생성

PVC를 정의한 파일을 준비합니다.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1G
```

준비된 YAML 파일(pvc.yaml)로 PVC를 생성합니다.

```
kubectl create -f pvc.yaml
```

```
ubuntu@k8s-master:~/volume$ k create -f pvc.yaml
persistentvolumeclaim/pvc created
```


Step 3: Pod 생성

PVC를 사용하여 PV를 마운트하는 Pod를 정의하는
YAML 파일을 준비합니다.

준비된 YAML 파일(pv-pod.yaml)로 Pod를 생성합니다.

```
kubectl create -f pv-pod.yaml
```

```
ubuntu@k8s-master:~/volume$ k create -f pv-pod.yaml
pod/persistentvolume created
```

```
apiVersion: v1
kind: Pod
metadata:
  name: persistentvolume
spec:
  containers:
  - name: container
    image: redis
    volumeMounts:
    - name: persistentvolume
      mountPath: /mount1
  volumes:
  - name: persistentvolume
    persistentVolumeClaim:
      claimName: pvc
```

Step 4: 생성한 리소스 확인

생성한 PV, PVC, Pod를 조회합니다.

```
kubectl get pv,pvc,po
```

```
ubuntu@k8s-master:~/volume$ k get pv,pvc,po
NAME                                CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
persistentvolume/pv                1G        RWO           Retain          Bound   default/pvc

NAME                                STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS
persistentvolumeclaim/pvc          Bound   pv      1G        RWO           

NAME                                READY  STATUS   RESTARTS  AGE
pod/emptydir-pod                    2/2    Running  2 (68m ago)  5h9m
pod/hostpath-pod                    1/1    Running  1 (68m ago)  4h35m
pod/nfs-server-5868894b96-frx7r     1/1    Running  0          61m
pod/persistentvolume                1/1    Running  0          2m34s
```

Step 5: 컨테이너에서 파일 생성

Pod 내부로 들어갑니다.

```
kubectl exec -it persistentvolume --container container -- /bin/bash
```

```
ubuntu@k8s-master:~/volume$ k exec -it persistentvolume --container container -- /bin/bash
```

※ **주의**: Pod가 생성되는 중이라면 컨테이너 내부로 들어갈 수 없습니다. 접속이 안되면 STATUS를 확인하세요.

마운트 된 디렉터리로 이동 후 테스트 파일을 생성합니다.

```
cd /mount1
echo hello pv >> test.txt
cat test.txt
exit
```

```
root@persistentvolume:/data# cd /mount1
root@persistentvolume:/mount1# echo hello pv >> test.txt
root@persistentvolume:/mount1# cat test.txt
hello pv
root@persistentvolume:/mount1# exit
exit
```

Step 6: 노드에서 파일 생성 확인

Pod가 어떤 노드에 생성되었는지 확인합니다.

```
kubectl get po persistentvolume -o wide
```

```
ubuntu@k8s-master:~/volume$ k get po persistentvolume -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP             NODE
persistentvolume    1/1     Running   0           3m31s  192.168.140.63  k8s-worker-2
```

위에서 확인한 노드(**worker-2**)의 터미널로 이동해 파일이 생성된 것을 확인합니다.

```
cd /pv
cat test.txt
```

```
ubuntu@k8s-worker-2:~$ cd /pv
ubuntu@k8s-worker-2:/pv$ ls
test.txt
ubuntu@k8s-worker-2:/pv$ cat test.txt
hello pv
```

Step 7: 생성한 모든 리소스를 삭제

지금까지 생성한 모든 리소스를 삭제합니다.

```
kubectl delete po,pv,pvc --all
```

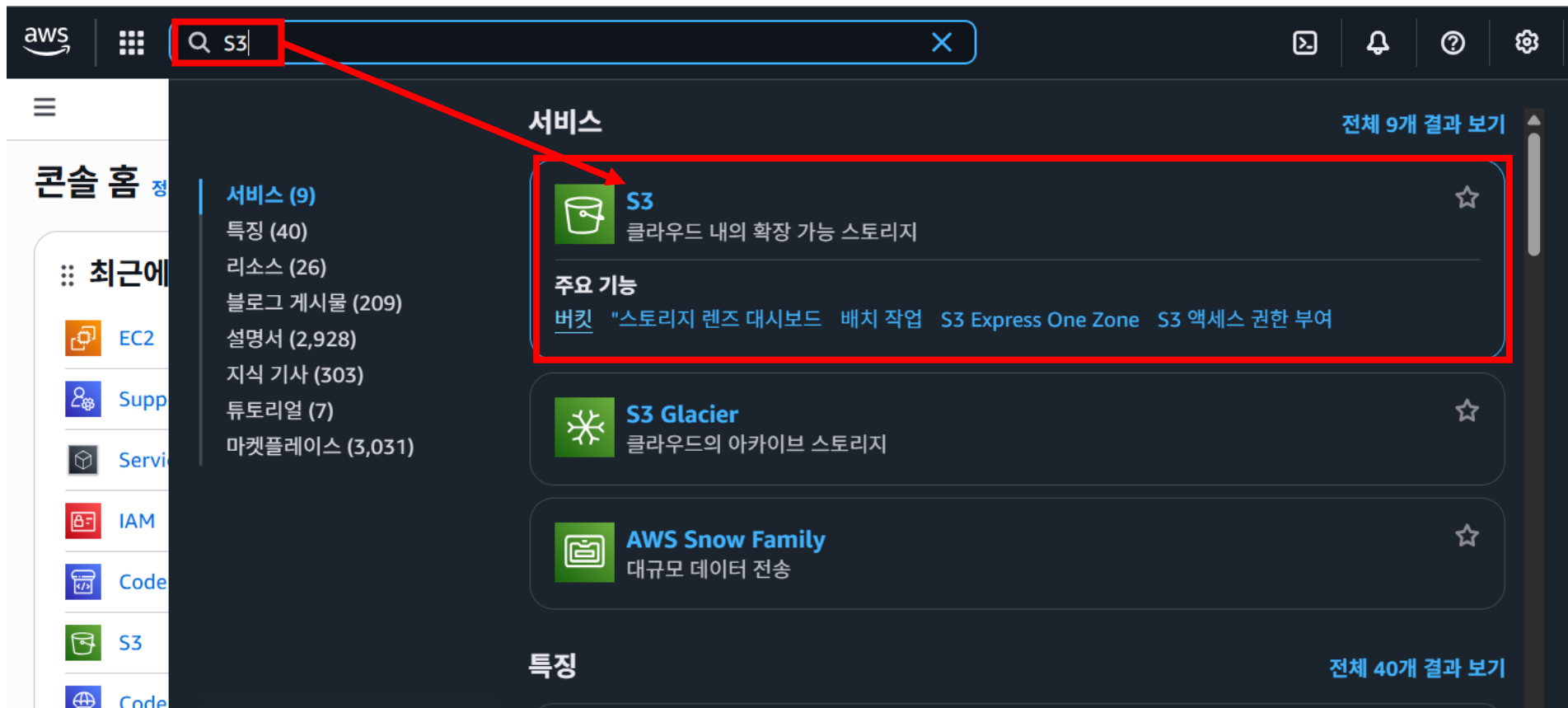
```
ubuntu@k8s-master:~/volume$ k delete po,pv,pvc --all
pod "emptydir-pod" deleted
pod "hostpath-pod" deleted
pod "nfs-server-5868894b96-frx7r" deleted
pod "persistentvolume" deleted
persistentvolume "pv" deleted
persistentvolumeclaim "pvc" deleted
```

Lab 10.

소스 위치로 git 저장소 활용하기

Step 1. Prod Bucket 생성

AWS 콘솔에서 S3 서비스를 클릭합니다.



Step 1. Prod Bucket 생성

범용 버킷에서 "버킷 만들기"를 클릭합니다.

Amazon S3

버킷

범용 버킷

디렉터리 버킷

테이블 버킷

벡터 버킷

액세스 관리 및 보안

액세스 지점

Access Grants

IAM Access Analyzer

스토리지 관리 및 인사이트

Storage Lens

배치 작업

범용 버킷

모든 AWS 리전

디렉터리 버킷

범용 버킷 (12) 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

이름으로 버킷 찾기

< 1 >

버킷 만들기

이름	AWS 리전	생성 날짜

Step 1. Prod Bucket 생성

Prod용 버킷을 생성합니다.

일반 구성

AWS 리전
아시아 태평양(서울) ap-northeast-2

버킷 이름 | 정보

버킷 이름은 3~63자여야 하며 글로벌 네임스페이스 내에서 고유해야 합니다. 또한 버킷 이름은 문자나 숫자로 시작하고 끝나야 합니다. 유효한 문자는 a-z,

기존 버킷에서 설정 복사 - 선택 사항
다음 구성의 버킷 설정만 복사됩니다.

버킷 선택

형식: s3://bucket/prefix

Step 1. Prod Bucket 생성

객체 소유권에서 "ACL 활성화됨"을 체크합니다.

객체 소유권 정보

다른 AWS 계정에서 이 버킷에 작성한 객체의 소유권 및 액세스 제어 목록(ACL)의 사용을 제어합니다. 객체 소유권은 객체에 대한 액세스를 지정할 수 있는 사용자를 결정합니다.

☐ ACL 비활성화됨(권장)

이 버킷의 모든 객체는 이 계정이 소유합니다. 이 버킷과 그 객체에 대한 액세스는 정책을 통해서만 지정됩니다.

☒ ACL 활성화됨

이 버킷의 객체는 다른 AWS 계정에서 소유할 수 있습니다. 이 버킷 및 객체에 대한 액세스는 ACL을 사용하여 지정할 수 있습니다.

⚠ 각 객체의 액세스를 개별적으로 제어하거나 객체 라이터가 업로드하는 데이터를 소유하도록 해야 하는 경우가 아니라면 ACL을 비활성화하는 것이 좋습니다. ACL 대신 버킷 정책을 사용하여 계정 외부의 사용자와 데이터를 공유하면 권한을 관리하고 감사하기가 용이합니다.

Step 1. Prod Bucket 생성

S3는 기본적으로 퍼블릭 액세스 차단 설정이 되어 있습니다. "모든 퍼블릭 액세스 차단"을 체크 해제하고, 아래쪽에 퍼블릭 상태가 될 수 있음에 체크 합니다.

이 버킷의 퍼블릭 액세스 차단 설정

퍼블릭 액세스는 ACL(액세스 제어 목록), 버킷 정책, 액세스 지정 정책 또는 모두를 통해 버킷 및 객체에 부여됩니다. 이 버킷 및 해당 객체는 이 설정이 이 버킷 및 해당 액세스 지정에만 적용됩니다. AWS에서는 모든 퍼블릭 액세스 차단을 활성화하도록 권장하지만, 이 설정이 이 버킷 또는 내부 객체에 대한 어느 정도 수준의 퍼블릭 액세스가 필요한 경우 특정 스토리지 사용 사례에 맞게 아래 개별 설정을 사용하십시오.

☐ 모든 퍼블릭 액세스 차단

이 설정을 활성화하면 아래 4개의 설정을 모두 활성화한 것과 같습니다. 다음 설정 각각은 서로 독립적입니다.

☐ 새 ACL(액세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 새로 추가된 버킷 또는 객체에 적용되는 퍼블릭 액세스 권한을 차단하며, 기존 버킷 및 객체에 대한 새 퍼블릭 액세스 ACL 생성을 금지합니다.

☐ 임의의 ACL(액세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 모든 ACL을 무시합니다.

☐ 새 퍼블릭 버킷 또는 액세스 지정 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 새 버킷 및 액세스 지정 정책을 차단합니다. 이 설정은 S3 리소스에 대한 퍼블릭 액세스를 허용합니다.

☐ 임의의 퍼블릭 버킷 또는 액세스 지정 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 및 교차 계정 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 정책을 사용하는 버킷 또는 액세스 지정에 대한 퍼블릭 및 교차 계정 액세스를 무시합니다.

⚠ 모든 퍼블릭 액세스 차단을 비활성화하면 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있습니다.

정적 웹 사이트 호스팅과 같은 구체적으로 확인된 사용 사례에서 퍼블릭 액세스가 필요한 경우가 아니면 모든 퍼블릭 액세스 차단을 비활성화하지 마십시오.

☒ 현재 설정으로 인해 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있음을 알고 있습니다.

Step 1. Prod Bucket 생성

맨 아래 "버킷 만들기"를 클릭해 Prod용 버킷을 생성하고, 생성된 버킷을 확인합니다.

범용 버킷 (2) 정보

 ARN 복사 비어 있음 삭제 버킷 만들기

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

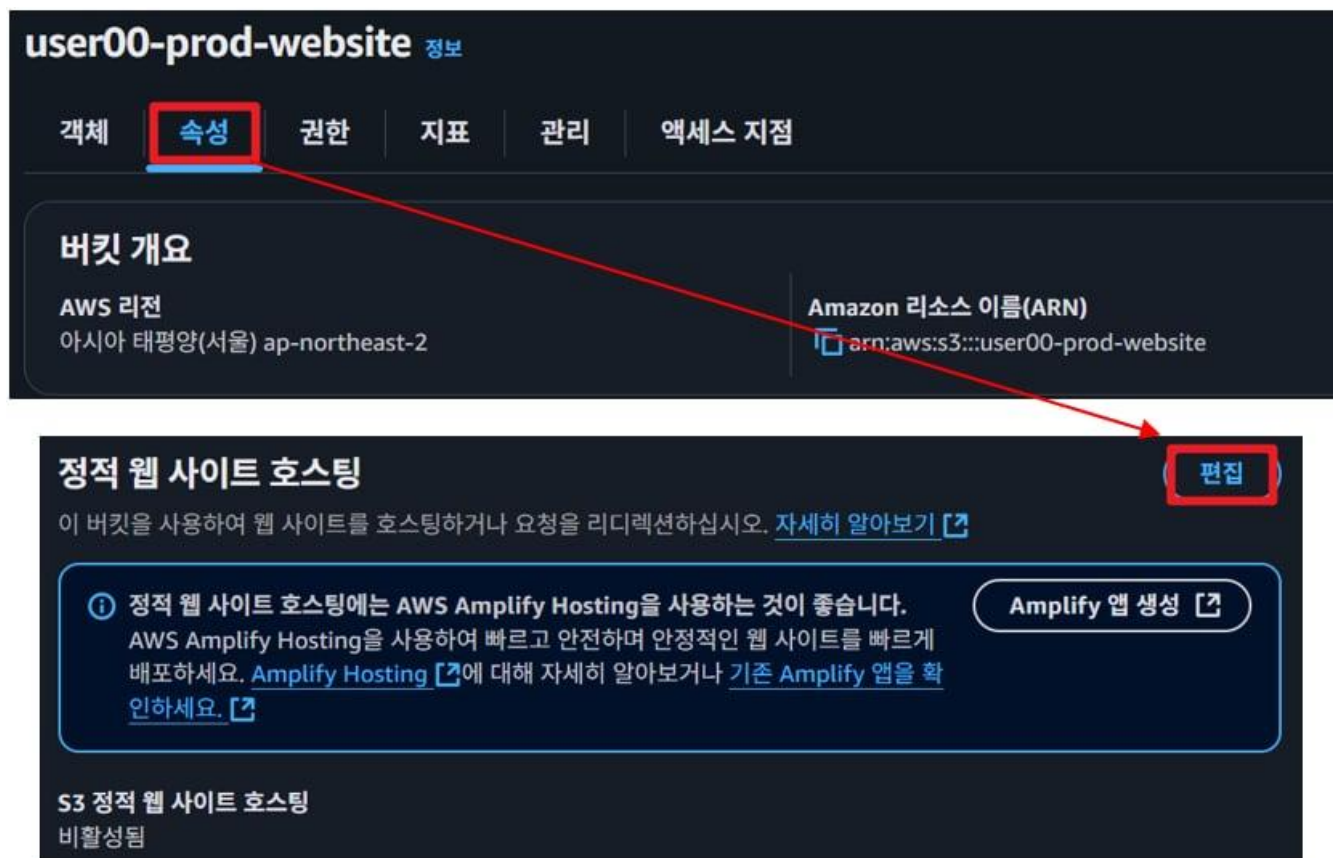
< 1 >



	이름	AWS 리전	생성 날짜
☐	user00-prod-website	아시아 태평양(서울) ap-northeast-2	

Step 2. 정적 웹 사이트 호스팅

Prod 버킷에서 "속성"을 클릭하고, 아래쪽에 정적 웹 사이트 호스팅에서 "편집"을 클릭합니다.



The screenshot shows the AWS Management Console interface. At the top, the breadcrumb is 'user00-prod-website 정보'. Below it, there are tabs: '객체', '속성' (highlighted with a red box), '권한', '지표', '관리', and '액세스 지점'. The '속성' tab shows the '버킷 개요' (Bucket Overview) section with 'AWS 리전' (Asia Pacific (Seoul) ap-northeast-2) and 'Amazon 리소스 이름(ARN)' (arn:aws:s3:::user00-prod-website). A red arrow points from the '속성' tab to the '정적 웹 사이트 호스팅' (Static website hosting) section below. In this section, there is an 'Amplify 앱 생성' button and an 'Amplify 앱 생성' button. The 'Amplify 앱 생성' button is highlighted with a red box. Below this, there is a section titled 'S3 정적 웹 사이트 호스팅' with the status '비활성됨' (Inactive).

Step 2. 정적 웹 사이트 호스팅

정적 웹 사이트 호스팅을 "활성화" 합니다.

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

정적 웹 사이트 호스팅

☐ 비활성화

☒ 활성화

인덱스 문서와 오류 문서를 아래와 같이 지정하고, "변경 사항 저장" 을 클릭합니다.

인덱스 문서

웹 사이트의 홈 페이지 또는 기본 페이지를 지정합니다.

index.html

오류 문서 - 선택 사항

오류가 발생하면 반환됩니다.

error.html

Step 3. 버킷 웹 사이트 엔드포인트 확인

Prod 버킷의 정적 웹 사이트 호스팅에서 "버킷 웹 사이트 엔드포인트"를 확인합니다.

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

① 정적 웹 사이트 호스팅에는 AWS Amplify Hosting을 사용하는 것이 좋습니다.

AWS Amplify Hosting을 사용하여 빠르고 안전하며 안정적인 웹 사이트를 빠르게 배포하세요. [Amplify Hosting](#)에 대해 자세히 알아보기

S3 정적 웹 사이트 호스팅

활성화됨

호스팅 유형

버킷 호스팅

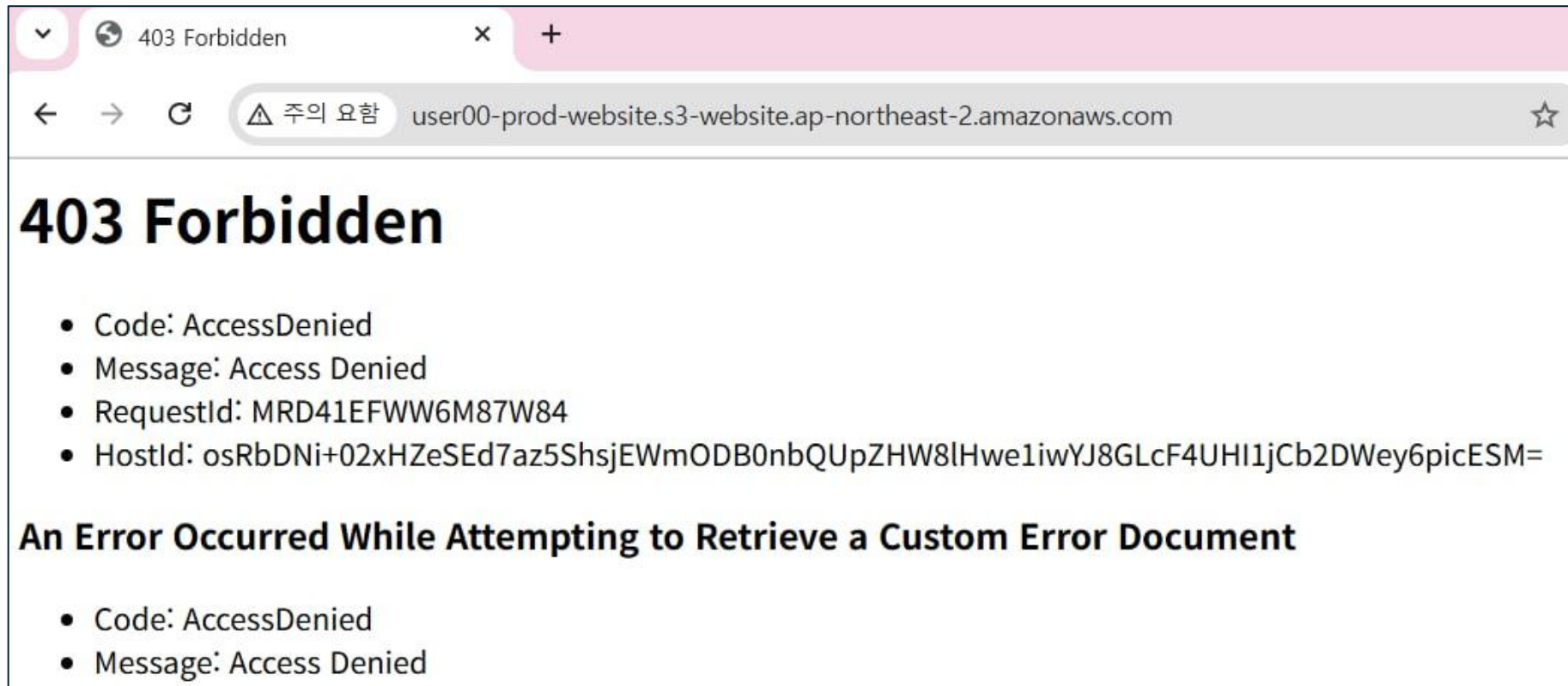
버킷 웹 사이트 엔드포인트

버킷을 정적 웹 사이트로 구성하면 해당 웹 사이트를 버킷의 AWS 리전별 웹 사이트 엔드포인트에서 사용할 수 있습니다. [자세히 알아보기](#)

<http://user00-prod-website.s3-website-ap-northeast-2.amazonaws.com>

Step 3. 버킷 웹 사이트 엔드포인트 확인

엔드포인트를 복사하고, 웹 브라우저에서 접속합니다.(접속 안됨)



Step 4. 웹 페이지 업로드

실습파일로 제공된 "test-website.zip" 파일을 업로드합니다.

user00-prod-website 정보

객체 | 속성 | 권한 | 지표 | 관리 | 액세스 지점

객체 (1)   S3 URI 복사  URL 복사  다운로드

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#) 를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. [자세히 알아보기](#) 

 접두사로 객체 찾기  버전 표시

☐	이름	▲	유형	▼	마지막 수정	▼
☐	 test-website.zip		zip			

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

AWS CodePipeline을 생성합니다.



Step 1: Pipeline 생성

"사용자 지정 파이프라인 빌드"를 선택합니다.

생성 옵션 선택 정보

Step 1 of 7

Category

☐ Deployment

☐ Continuous Integration

☐ Automation

☒ 사용자 지정 파이프라인 빌드

취소

다음

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

파이프라인의 이름(user##-github-pipeline)을 입력합니다.

파이프라인 설정 선택 정보

Step 2 of 7

파이프라인 설정

파이프라인 이름

파이프라인 이름을 입력합니다. 생성된 후에는 파이프라인 이름을 편집할 수 없습니다.

user00-github-pipeline

100자를 초과할 수 없음

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

소스 스테이지 추가에서 소스공급자를 "GitHub(GitHub 앱을 통해)"로 추가합니다.

소스 스테이지 추가 정보

Step 3 of 7

소스

소스 공급자

파이프라인의 입력 아티팩트를 저장한 위치입니다. 공급자를 선택한 후 연결 세부 정보를 제공하십시오.

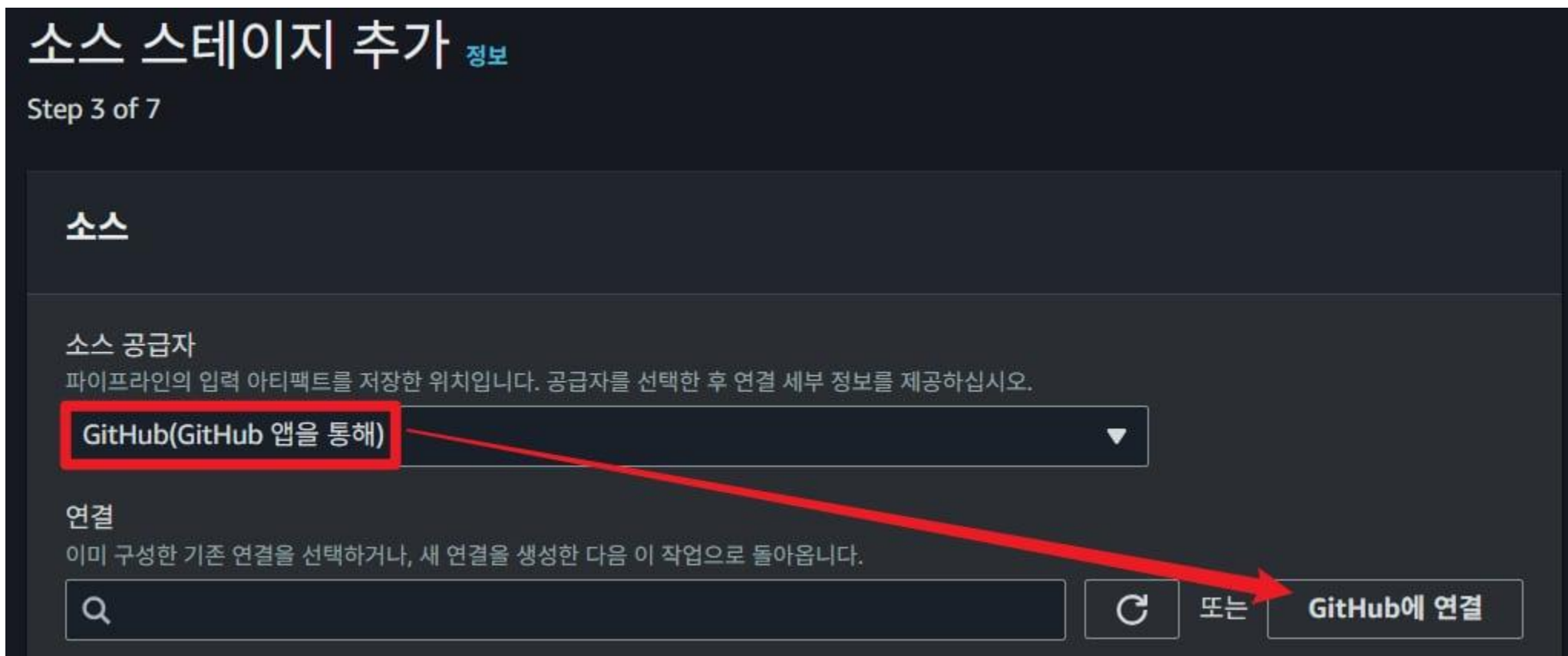
GitHub(GitHub 앱을 통해) ▼

연결

이미 구성한 기존 연결을 선택하거나, 새 연결을 생성한 다음 이 작업으로 돌아옵니다.

Q

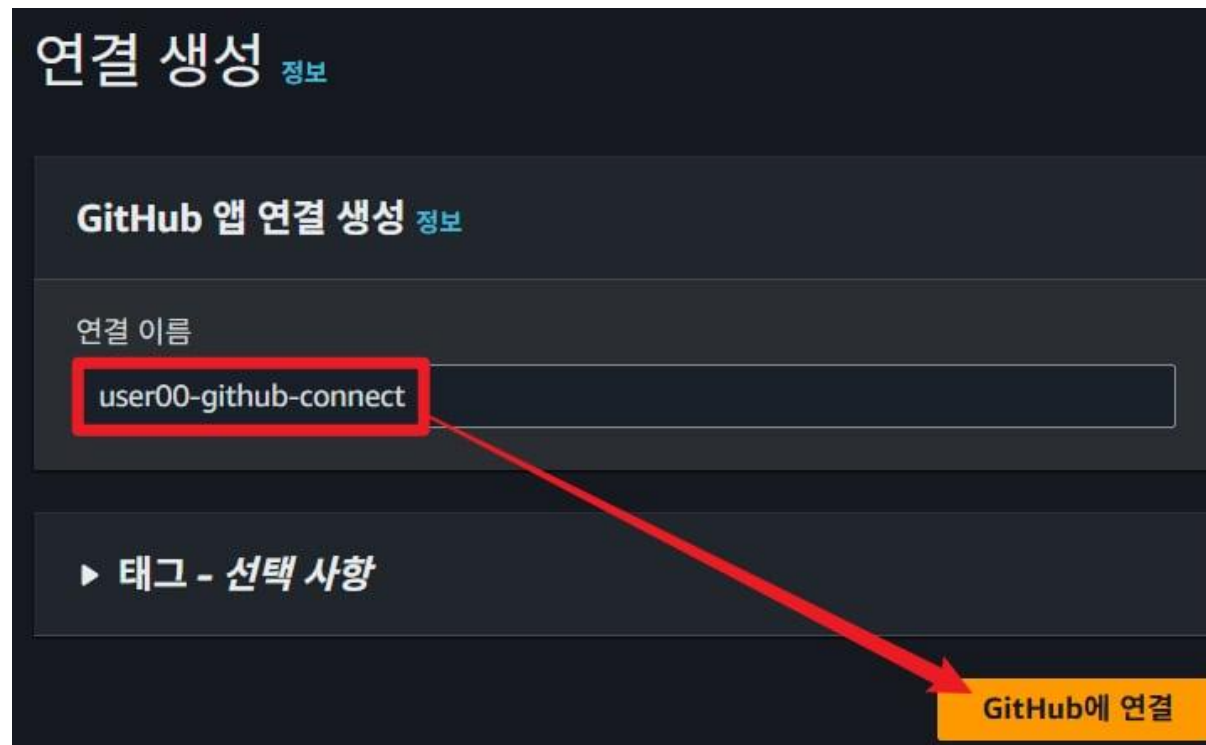
↻ 또는 **GitHub에 연결**



GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

연결 생성에서 연결 이름을 작성하고, "GitHub에 연결"을 클릭합니다.



연결 생성 정보

GitHub 앱 연결 생성 정보

연결 이름

user00-github-connect

▶ 태그 - 선택 사항

GitHub에 연결

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

새 앱 설치를 클릭합니다.(이미 설치한 게 있다면 선택해도 됩니다.)

GitHub에 연결

GitHub 연결 설정 정보

연결 이름

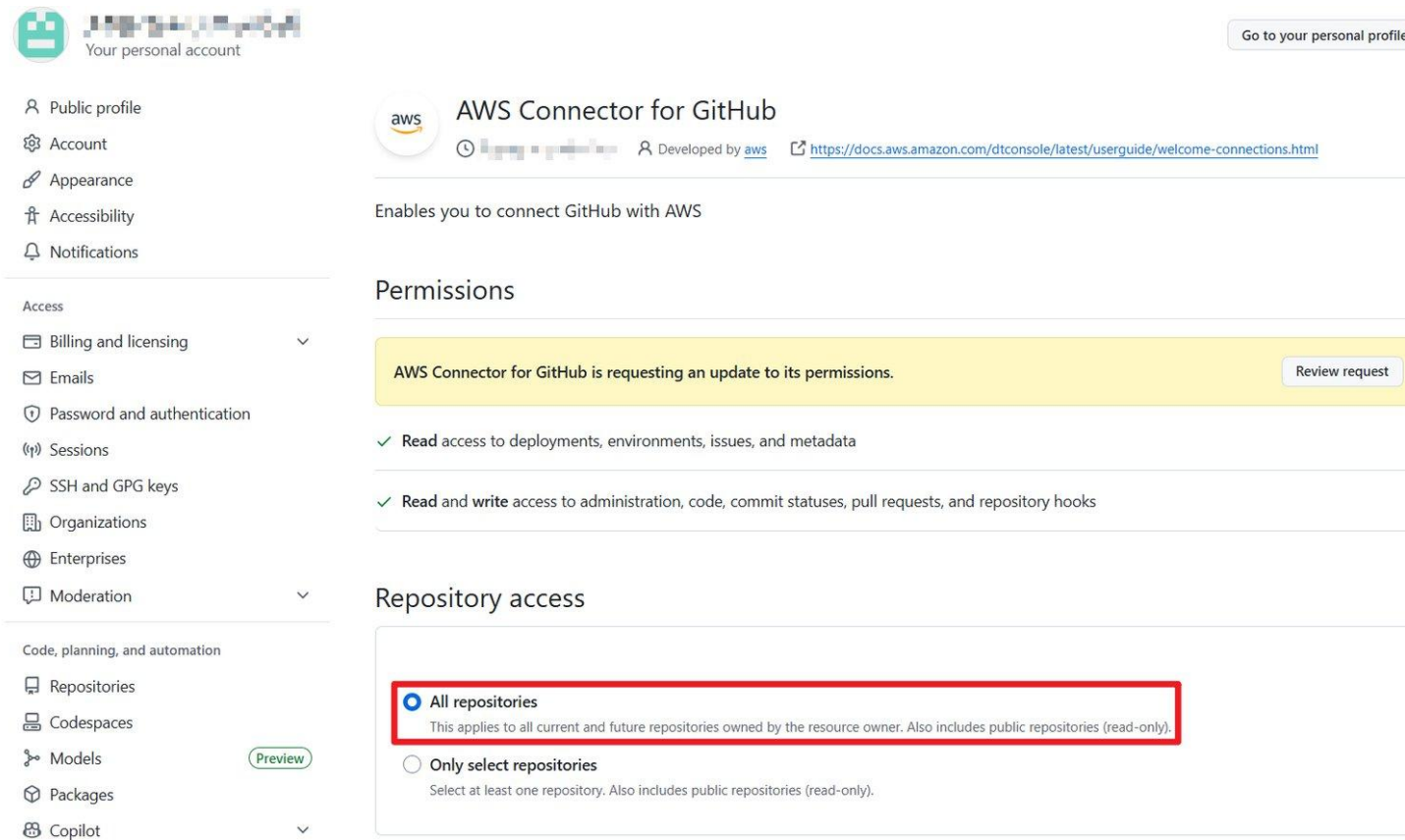
앱 설치 - 선택 사항

GitHub 앱을 설치하여 봇으로 연결합니다. 또는 GitHub 사용자로 연결하여 AWS CodeBuild 프로젝트에서 사용할 수 있도록 비워 둘 수도 있습니다.

 또는 **새 앱 설치**

Step 1: Pipeline 생성

Repository access를 "All repositories"로 선택합니다. (UI가 다를 수도 있으니 참고 바랍니다.)



The screenshot shows the GitHub Actions configuration page for the 'AWS Connector for GitHub' workflow. The left sidebar contains the user's profile and navigation links. The main content area shows the workflow details, including a permissions update notification and the 'Repository access' section. The 'All repositories' option is selected and highlighted with a red box.

Left Sidebar:

- Public profile
- Account
- Appearance
- Accessibility
- Notifications
- Access
 - Billing and licensing
 - Emails
 - Password and authentication
 - Sessions
 - SSH and GPG keys
 - Organizations
 - Enterprises
 - Moderation
- Code, planning, and automation
 - Repositories
 - Codespaces
 - Models
 - Packages
 - Copilot

Main Content:

- AWS Connector for GitHub**
Developed by [aws](#)
<https://docs.aws.amazon.com/dtconsole/latest/userguide/welcome-connections.html>
- Enables you to connect GitHub with AWS
- Permissions**
 - AWS Connector for GitHub is requesting an update to its permissions. [Review request](#)
 - ✓ Read access to deployments, environments, issues, and metadata
 - ✓ Read and write access to administration, code, commit statuses, pull requests, and repository hooks
- Repository access**
 - ☒ **All repositories**
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).
 - ☐ Only select repositories
Select at least one repository. Also includes public repositories (read-only).

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

"연결"을 클릭해 Git Repository와 연결합니다.

GitHub에 연결

GitHub 연결 설정 정보

연결 이름

앱 설치 - 선택 사항

GitHub 앱을 설치하여 봇으로 연결합니다. 또는 GitHub 사용자로 연결하여 AWS CodeBuild 프로젝트에서 사용할 수 있도록 비워 둘 수도 있습니다.

▶ 태그 - 선택 사항

연결

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

리포지토리 이름("test-website-repo")과 기본 브랜치(main)를 선택하고, "다음"을 클릭합니다.

리포지토리 이름

GitHub 계정에서 리포지토리를 선택합니다.

제공된 자격 증명이 액세스할 수 있는 모든 프로젝트에 그룹 경로를 입력하거나 붙여넣을 수 있습니다. '그룹/하위 그룹/프로젝트' 형식을 사용합니다.

기본 브랜치

기본 브랜치는 파이프라인 실행이 다른 소스에서 시작되거나 수동으로 시작된 경우에만 사용됩니다.

빌드와 테스트 스테이지는 건너 뛩니다.

취소

이전

빌드 스테이지 건너뛰기

다음

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

배포 공급자, 리전, 입력 아티팩트, 버킷을 작성하고, "배포하기 전에 파일 압축 풀기"를 체크합니다.

배포 공급자
애플리케이션 또는 콘텐츠를 배포할 방법을 선택합니다. 공급자를 선택한 다음 해당 공급자에 대한 구성 세부 정보를 입력하세요.

Amazon S3

리전

아시아 태평양 (서울)

입력 아티팩트
이 작업의 입력 아티팩트를 선택합니다. 자세히 알아보십시오

SourceArtifact X
Source에 의해 정의됨

100자 이하

버킷

user00-prod-website X

배포 경로 - 선택 사항

☒ 배포하기 전에 파일 압축 풀기
아티팩트를 배포하기 전에 아티팩트의 압축을 풀니다.

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

추가 구성에서 표준 ACL을 "public-read"로 설정하고, 다음을 클릭합니다.

▼ 추가 구성

KMS 암호화 키 ARN - 선택 사항

KMS 암호화 키를 사용하여 객체를 암호화합니다. 키를 입력하지 않으면 객체가 암호화되지 않은 상태로 유지됩니다.

arn:aws:kms:<region-ID>:<account-ID>;key/<key-ID> 또는 arn:aws:kms:<region-ID>:<account-ID>;alias/<alias-name>

표준 ACL - 선택 사항

버킷에 대한 Amazon S3 표준 ACL(액세스 제어 목록)을 지정합니다.

public-read ▼

캐시 제어 - 선택 사항

Amazon S3 버킷에서 요청된 객체에 대한 캐시 제어를 설정합니다.

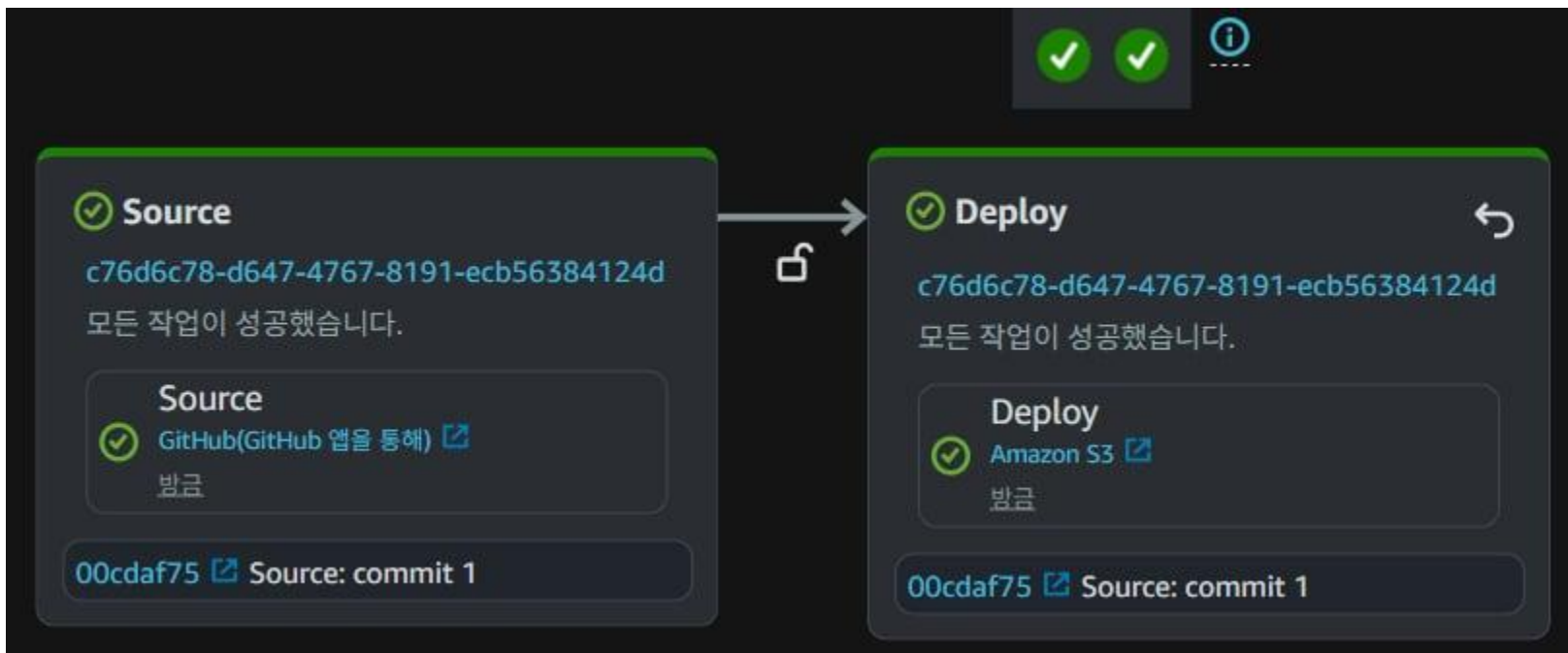
☒ 스테이지 장애 시 자동 롤백 구성

☐ 스테이지 장애 시 자동 재시도 활성화

GitHub 기반 Pipeline 구성

Step 1: Pipeline 생성

검토 단계를 확인하고, 파이프라인을 생성합니다.



Step 2: 업로드 및 정상 접근 확인

prod 버킷에 파일이 업로드 되었는지 확인합니다.

user00-prod-website 정보

객체

속성

권한

지표

관리

액세스 지점

객체 (3)

S3 URI 복사

URL 복사

다운로드

열기

삭제

작업

폴더 만들기

업로드

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. 다른 사용자가 객체에 액세스할 수 있게 하려면 명시적으로 권한을 부여해야 합니다. [자세히 알아보기](#)

Q

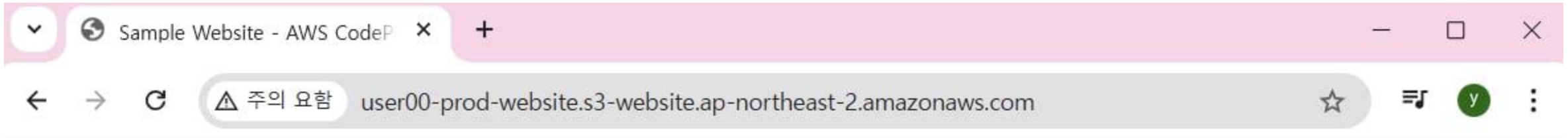
접두사로 객체 찾기

< 1 >

☐	이름	▲	유형	▼	마지막 수정	▼	크기	▼	스토리지 클래스	▼
☐	error.html		html				428.0B		Standard	
☐	index.html		html				405.0B		Standard	
☐	test-website.zip		zip				782.0B		Standard	

Step 2: 업로드 및 정상 접근 확인

"속성 > 버킷 웹 사이트 엔드포인트"로 페이지에서 정상 접근을 확인합니다.



Welcome to Our Website!

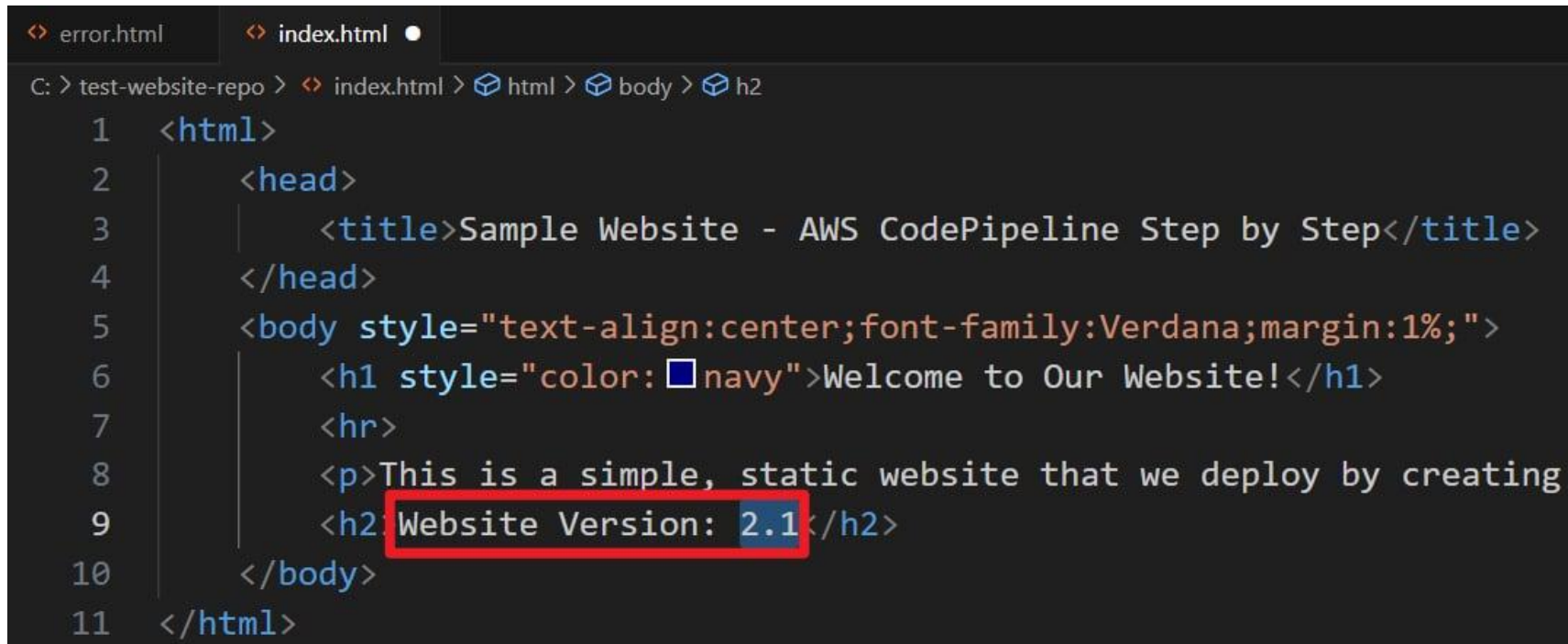
This is a simple, static website that we deploy by creating a pipeline on AWS CodePipeline.

Website Version: 2.0

Git 명령어로 트리거 실행하기

Step 1: 파일 수정

vscode에서 웹 페이지 파일(index.html)의 일부를 수정하고, 저장(ctrl + s)합니다.



```
<?xml version="1.0" encoding="UTF-8" ?>
<html>
  <head>
    <title>Sample Website - AWS CodePipeline Step by Step</title>
  </head>
  <body style="text-align:center;font-family:Verdana;margin:1%;">
    <h1 style="color: navy">Welcome to Our Website!</h1>
    <hr>
    <p>This is a simple, static website that we deploy by creating</p>
    <h2>Website Version: 2.1</h2>
  </body>
</html>
```

Git 명령어로 트리거 실행하기

Step 2: commit, push

VS코드의 터미널에서 commit과 push를 합니다. (commit message는 자유롭게 구성합니다.)

```
git commit -a -m "2.1 update"
git push origin main
```

```
PS C:\test-website-repo> git commit -m "commit 1"
[master (root-commit) 00cdaf7] commit 1
3 files changed, 22 insertions(+)
create mode 100644 error.html
create mode 100644 index.html
create mode 100644 test-website.zip
```

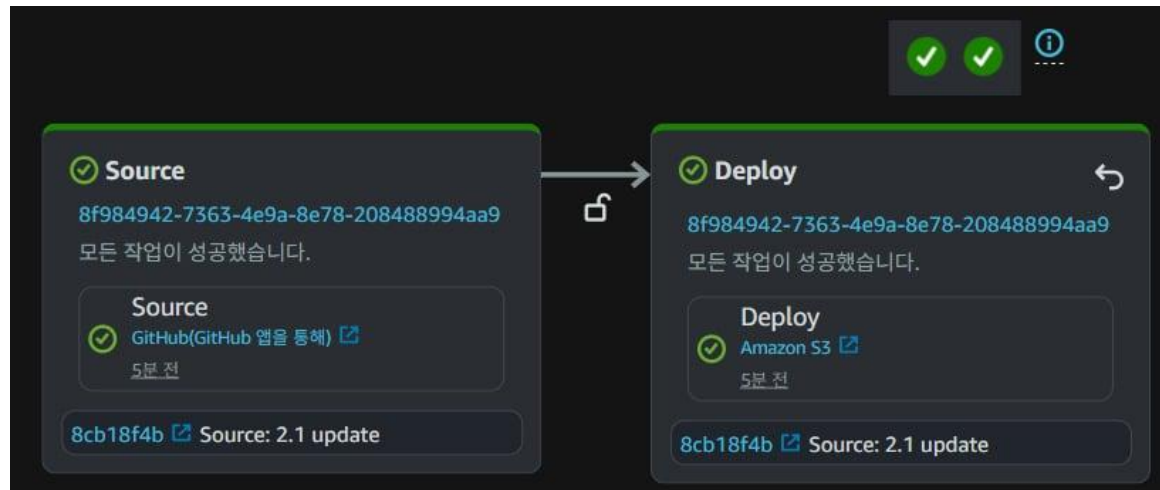
```
PS C:\test-website-repo>
PS C:\test-website-repo> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 18 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5) 1.21 KiB | 1.21 MiB/s done
```

Git 명령어로 트리거 실행하기

Step 3: 결과 확인

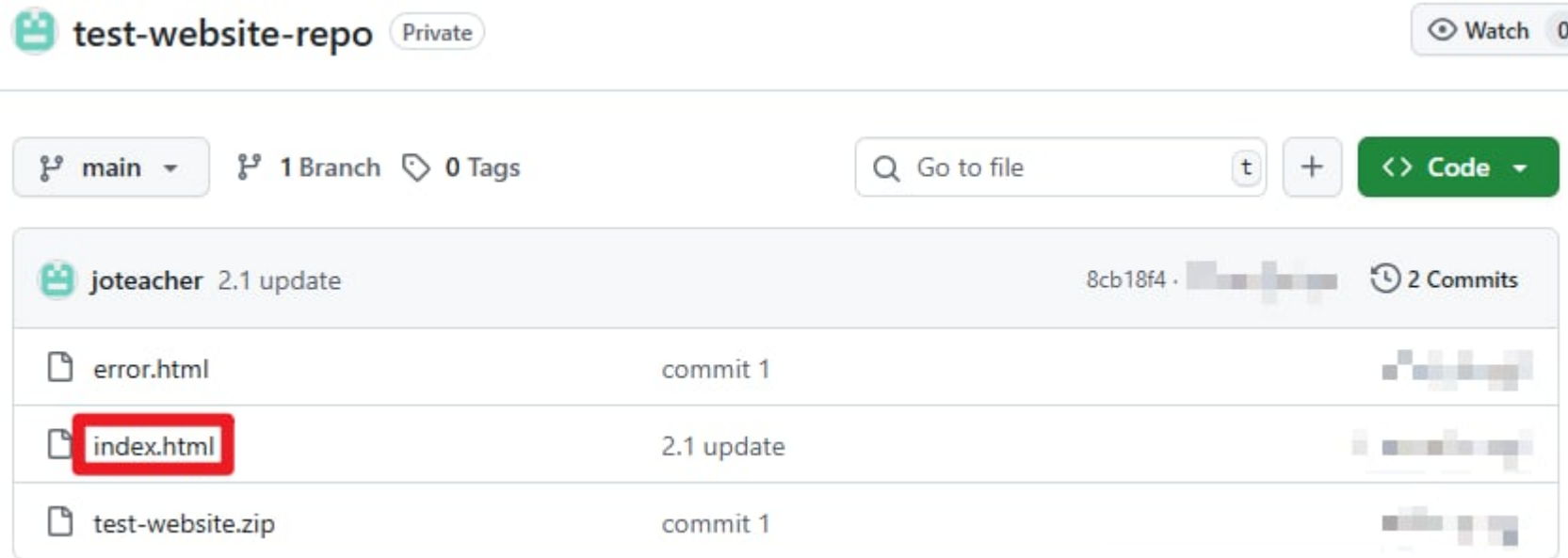
파이프라인에서 업데이트 내역을 확인하고,

웹 페이지의 업데이트를 확인합니다.



Step 1: 소스 수정

GitHub에서 소스(index.html)를 수정합니다.



The screenshot shows the GitHub interface for a repository named 'test-website-repo'. The repository is marked as 'Private'. At the top right, there is a 'Watch' button with a count of '0'. Below the repository name, there are buttons for 'main' (selected), '1 Branch', and '0 Tags'. A search bar labeled 'Go to file' is present, along with a 't' icon, a '+' icon, and a green 'Code' button. The file list shows three files: 'error.html' (commit 1), 'index.html' (2.1 update), and 'test-website.zip' (commit 1). The 'index.html' file is highlighted with a red box. The commit hash '8cb18f4' and '2 Commits' are also visible.

File	Commit
error.html	commit 1
index.html	2.1 update
test-website.zip	commit 1

Step 1: 소스 수정

편집 버튼을 클릭합니다.

test-website-repo / index.html

joteacher 2.1 update 8cb18f4 · 18 minutes ago History

Code Blame 11 lines (11 loc) · 405 Bytes Raw Copy Download Edit

```
1 <html>
2   <head>
3     <title>Sample Website - AWS CodePipeline Step by Step</title>
4   </head>
5   <body style="text-align:center;font-family:Verdana;margin:1%;">
6     <h1 style="color:navy">Welcome to Our Website!</h1>
7     <hr>
8     <p>This is a simple, static website that we deploy by creating a pipeline on AWS CodePipeline.</p>
9     <h2>Website Version: 2.1</h2>
10  </body>
11 </html>
```


Step 1: 소스 수정

버전을 수정하고, "Commit changes"를 클릭합니다.



Step 1: 소스 수정

commit message를 입력하고, "Commit changes"를 클릭합니다.

Commit changes

Commit message

Update index.html

Extended description

Add an optional extended description...

☒ Commit directly to the main branch

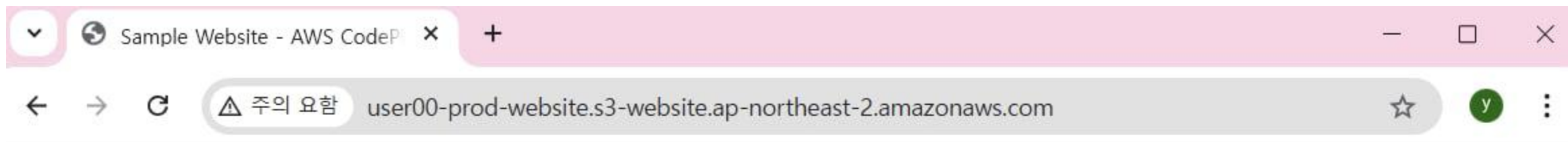
☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel

Commit changes

Step 2: 웹 페이지 확인

코드가 수정되었으므로 웹페이지에서 수정된 것을 확인합니다.



Welcome to Our Website!

This is a simple, static website that we deploy by creating a pipeline on AWS CodePipeline.

Website Version: 3.0

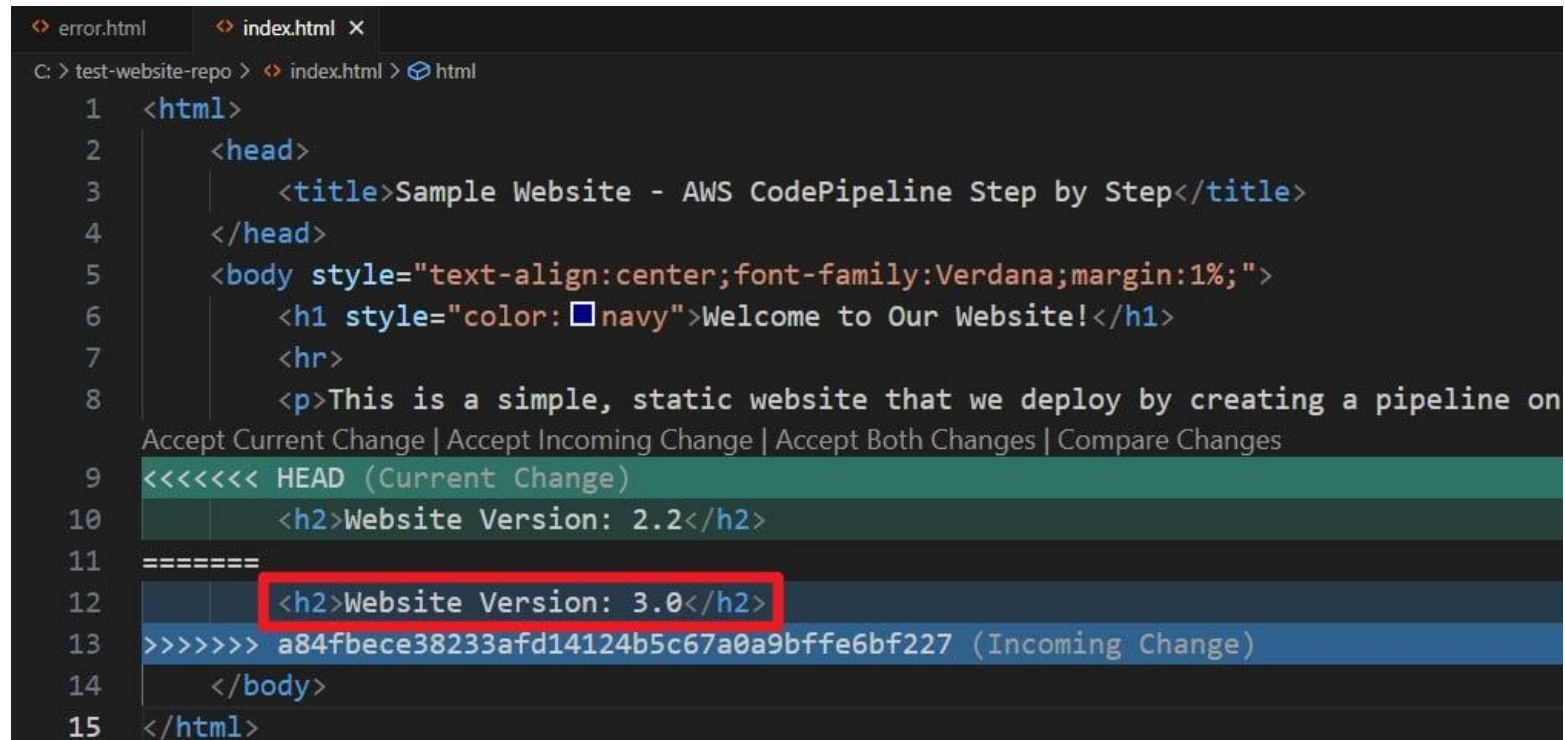
Step 3: VS Code 파일 수정

VS Code에서 pull 작업을 통해 파일을 업데이트 합니다.

```
PS C:\test-website-repo> git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1001 bytes | 125.00 KiB/s, done.
From https://github.com/joteacher/test-website-repo
* branch                main                -> FETCH_HEAD
   8cb18f4..a84fbec      main                -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

Step 3: VS Code 파일 수정

파일이 업데이트 되었음을 확인합니다.



```
error.html index.html X
C: > test-website-repo > index.html > html
1 <html>
2   <head>
3     <title>Sample Website - AWS CodePipeline Step by Step</title>
4   </head>
5   <body style="text-align:center;font-family:Verdana;margin:1%;">
6     <h1 style="color: navy">Welcome to Our Website!</h1>
7     <hr>
8     <p>This is a simple, static website that we deploy by creating a pipeline on
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
9 <<<<<< HEAD (Current Change)
10    <h2>Website Version: 2.2</h2>
11 =====
12    <h2>Website Version: 3.0</h2>
13 >>>>>> a84fbece38233afd14124b5c67a0a9bffe6bf227 (Incoming Change)
14    </body>
15 </html>
```

Lab 11.

AWS CodeBuild로 코드 빌드하기

Repository와 배포용 Bucket 생성

Step 1: Repository 생성

GitHub에서 새로운 Repository를 생성합니다.

Create a new repository [Preview](#) [Switch back to classic experience](#)

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *



Repository name *

test-angular-repo

✓ test-angular-repo is available.

Great repository names are short and memorable. How about [fuzzy-octo-winner](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

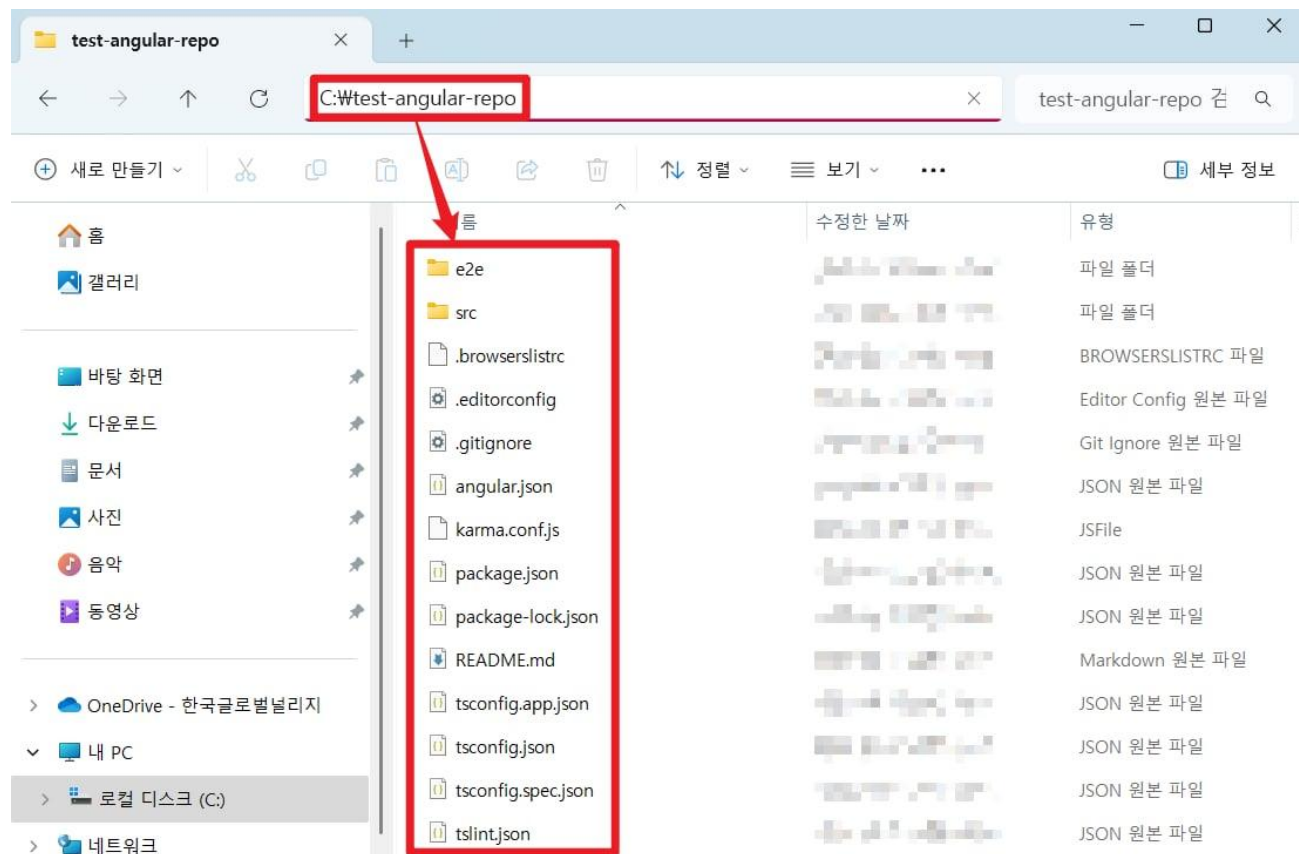
No license

Create repository

Repository와 배포용 Bucket 생성

Step 2: 파일 준비

제공된 실습 파일을 로컬에 저장합니다.(C드라이브 하위)



Repository와 배포용 Bucket 생성

Step 3: 원격 연결

로컬에 저장된 파일 위치에서 터미널에 접속하여 GitHub Repository와 원격 연결합니다.

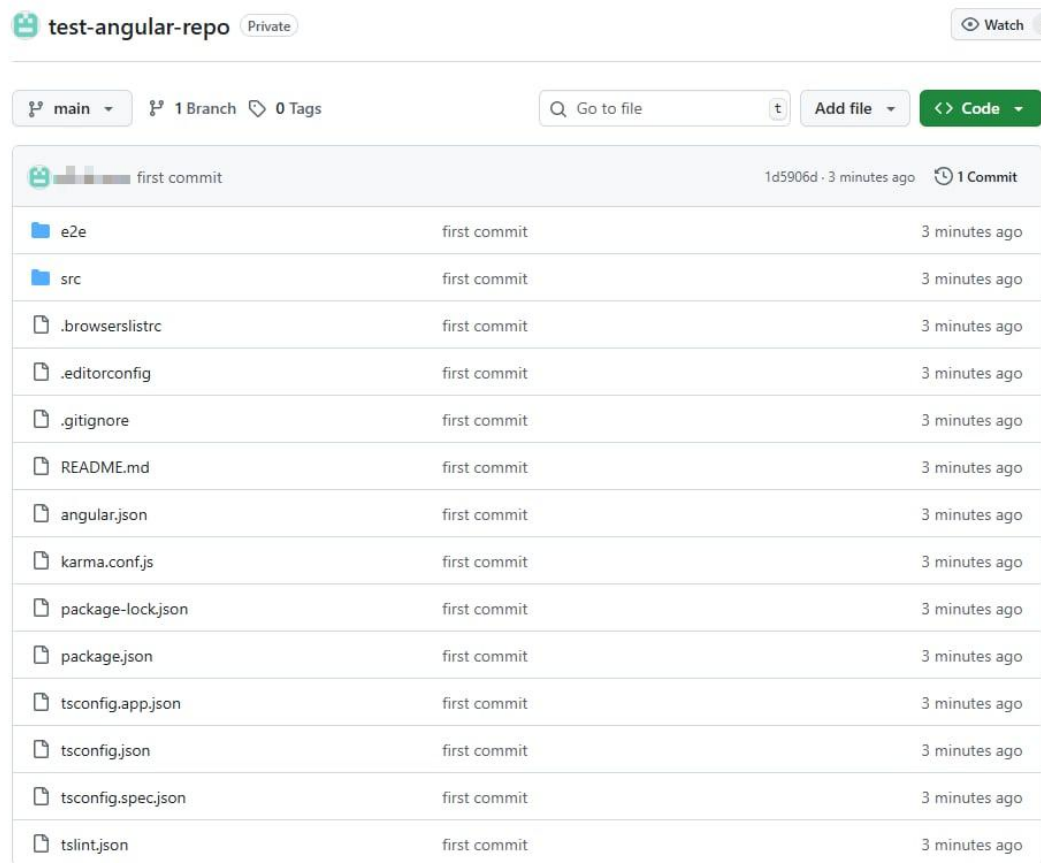
```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/<계정명>/test-angular-repo.git
git push -u origin main
```

```
PS C:\test-angular-repo> git branch -M main
PS C:\test-angular-repo> git remote add origin https://github.com/[redacted]/test-angular-repo.git
PS C:\test-angular-repo> git push -u origin main
Enumerating objects: 42, done.
Counting objects: 100% (42/42), done.
Delta compression using up to 18 threads
Compressing objects: 100% (40/40), done.
Writing objects: 100% (42/42), 137.78 KiB | 4.92 MiB/s, done.
Total 42 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/joteacher/test-angular-repo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Repository와 배포용 Bucket 생성

Step 3: 원격 연결

GibHub Repository에 업로드 된 파일을 확인합니다.



The screenshot shows a GitHub repository page for 'test-angular-repo' (Private). The repository has 1 branch (main) and 0 tags. The 'first commit' (1d5906d, 3 minutes ago) contains the following files:

File Name	Commit	Time
e2e	first commit	3 minutes ago
src	first commit	3 minutes ago
.browserslistrc	first commit	3 minutes ago
.editorconfig	first commit	3 minutes ago
.gitignore	first commit	3 minutes ago
README.md	first commit	3 minutes ago
angular.json	first commit	3 minutes ago
karma.conf.js	first commit	3 minutes ago
package-lock.json	first commit	3 minutes ago
package.json	first commit	3 minutes ago
tsconfig.app.json	first commit	3 minutes ago
tsconfig.json	first commit	3 minutes ago
tsconfig.spec.json	first commit	3 minutes ago
tslint.json	first commit	3 minutes ago

Repository와 배포용 Bucket 생성

Step 4: S3 버킷 생성

Amazon S3 버킷의 이름(user##-angular-website)을 지정합니다.

버킷 만들기 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

일반 구성

AWS 리전

아시아 태평양(서울) ap-northeast-2

버킷 이름 정보

user00-angular-website

버킷 이름은 3~63자여야 하며 글로벌 네임스페이스 내에서 고유해야 합니다. 또한 버킷 이름은 문자나 숫자로 시작하고 끝나야 합니다. 유효한 문자는 a-z, 0-9, 마침표(.),

기존 버킷에서 설정 복사 - 선택 사항

다음 구성의 버킷 설정만 복사됩니다.

버킷 선택

형식: s3://bucket/prefix

Repository와 배포용 Bucket 생성

Step 4: S3 버킷 생성

객체 소유권에서 "ACL 활성화됨"을 체크합니다.

객체 소유권 정보

다른 AWS 계정에서 이 버킷에 작성한 객체의 소유권 및 액세스 제어 목록(ACL)의 사용을 제어합니다. 객체 소유권은 객체에 대한 액세스를 지정할 수 있는 사용자를 결정합니다.

☐ ACL 비활성화됨(권장)

이 버킷의 모든 객체는 이 계정이 소유합니다. 이 버킷과 그 객체에 대한 액세스는 정책을 통해서만 지정됩니다.

☒ ACL 활성화됨

이 버킷의 객체는 다른 AWS 계정에서 소유할 수 있습니다. 이 버킷 및 객체에 대한 액세스는 ACL을 사용하여 지정할 수 있습니다.

⚠ 각 객체의 액세스를 개별적으로 제어하거나 객체 라이터가 업로드하는 데이터를 소유하도록 해야 하는 경우가 아니라면 ACL을 비활성화하는 것이 좋습니다. ACL 대신 버킷 정책을 사용하여 계정 외부의 사용자와 데이터를 공유하면 권한을 관리하고 감사하기가 용이합니다.

Repository와 배포용 Bucket 생성

Step 4: S3 버킷 생성

이 버킷의 퍼블릭 액세스 차단 설정에서 "모든 퍼블릭 액세스 차단"을 해제합니다.

이 버킷의 퍼블릭 액세스 차단 설정

퍼블릭 액세스는 ACL(엑세스 제어 목록), 버킷 정책, 액세스 지점 정책 또는 모두를 통해 버킷 및 객체에 부여됩니다. 이 버킷 및 해당 객체에 대한 퍼블릭 액세스가 차단되었는지 확인하려면 모든 퍼블릭 액세스 차단을 활성화합니다. 이 설정은 이 버킷 및 해당 액세스 지점에만 적용됩니다. AWS에서는 모든 퍼블릭 액세스 차단을 활성화하도록 권장하지만, 이 설정을 적용하기 전에 퍼블릭 액세스가 없어도 애플리케이션이 올바르게 작동하는지 확인합니다. 이 버킷 또는 내부 객체에 대한 어느 정도 수준의 퍼블릭 액세스가 필요한 경우 특정 스토리지 사용 사례에 맞게 아래 개별 설정을 사용자 지정할 수 있습니다. [자세히 알아보기](#)

☐ 모든 퍼블릭 액세스 차단

이 설정을 활성화하면 아래 4개의 설정을 모두 활성화한 것과 같습니다. 다음 설정 각각은 서로 독립적입니다.

☐ 새 ACL(엑세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 새로 추가된 버킷 또는 객체에 적용되는 퍼블릭 액세스 권한을 차단하며, 기존 버킷 및 객체에 대한 새 퍼블릭 액세스 ACL 생성을 금지합니다. 이 설정은 ACL을 사용하여 S3 리소스에 대한 퍼블릭 액세스를 허용하는 기존 권한을 변경하지 않습니다.

☐ 임의의 ACL(엑세스 제어 목록)을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 모든 ACL을 무시합니다.

☐ 새 퍼블릭 버킷 또는 액세스 지점 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 새 버킷 및 액세스 지점 정책을 차단합니다. 이 설정은 S3 리소스에 대한 퍼블릭 액세스를 허용하는 기존 정책을 변경하지 않습니다.

☐ 임의의 퍼블릭 버킷 또는 액세스 지점 정책을 통해 부여된 버킷 및 객체에 대한 퍼블릭 및 교차 계정 액세스 차단

S3은 버킷 및 객체에 대한 퍼블릭 액세스를 부여하는 정책을 사용하는 버킷 또는 액세스 지점에 대한 퍼블릭 및 교차 계정 액세스를 무시합니다.

⚠ 모든 퍼블릭 액세스 차단을 비활성화하면 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있습니다.

정적 웹 사이트 호스팅과 같은 구체적으로 확인된 사용 사례에서 퍼블릭 액세스가 필요한 경우가 아니면 모든 퍼블릭 액세스 차단을 활성화하는 것이 좋습니다.

☒ 현재 설정으로 인해 이 버킷과 그 안에 포함된 객체가 퍼블릭 상태가 될 수 있음을 알고 있습니다.

Repository와 배포용 Bucket 생성

Step 4: S3 버킷 생성

버킷을 생성하고, 잘 만들어졌는지 확인합니다.

범용 버킷

모든 AWS 리전

디렉터리 버킷

범용 버킷 (4) 정보

버킷은 S3에 저장되는 데이터의 컨테이너입니다.

ARN 복사

비어 있음

삭제

버킷 만들기

이름으로 버킷 찾기

< 1 >

⚙

이름	AWS 리전	생성 날짜
codepipeline-ap-northeast-2-b535e2b505e8-46f1-8b6e-bf77671cff98	아시아 태평양(서울) ap-northeast-2	
user00-angular-website	아시아 태평양(서울) ap-northeast-2	
user00-prod-website	아시아 태평양(서울) ap-northeast-2	
user00-source-website	아시아 태평양(서울) ap-northeast-2	

Step 4: S3 버킷 생성

생성한 버킷의 속성에서 정적 웹 사이트 호스팅의 편집을 클릭합니다.

편집

Repository와 배포용 Bucket 생성

Step 4: S3 버킷 생성

정적 웹 사이트 호스팅을 "활성화"로 변경하고, 인덱스 문서와 오류 문서를 둘 다 "index.html"로 변경합니다.

정적 웹 사이트 호스팅 편집

정보

정적 웹 사이트 호스팅

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. [자세히 알아보기](#)

정적 웹 사이트 호스팅

☐ 비활성화
 ☒ 활성화

호스팅 유형

☒ 정적 웹 사이트 호스팅
버킷 엔드포인트를 웹 주소로 사용합니다. [자세히 알아보기](#)

☐ 객체에 대한 요청 리디렉션
요청을 다른 버킷 또는 도메인으로 리디렉션합니다. [자세히 알아보기](#)

고객이 웹 사이트 엔드포인트의 콘텐츠에 액세스할 수 있게 하려면 모든 콘텐츠를 공개적으로 참조하십시오.

인덱스 문서

웹 사이트의 홈 페이지 또는 기본 페이지를 지정합니다.

index.html

오류 문서 - 선택 사항

오류가 발생하면 반환됩니다.

index.html

Step 4: S3 버킷 생성

버킷의 속성 > 정적 웹 사이트 호스팅에서 "버킷 웹 사이트 엔드포인트"를 확인합니다.

정적 웹 사이트 호스팅

편집

이 버킷을 사용하여 웹 사이트를 호스팅하거나 요청을 리디렉션하십시오. 자세히 알아보기 [\[?\]](#)

정적 웹 사이트 호스팅에는 AWS Amplify Hosting을 사용하는 것이 좋습니다.

AWS Amplify Hosting을 사용하여 빠르고 안전하며 안정적인 웹 사이트를 빠르게 배포하세요. [Amplify Hosting](#)에 대해 자세히 알아보거나 [기존 Amplify 앱을 확인하세요](#).

Amplify 앱 생성 [↗](#)

S3 정적 웹 사이트 호스팅

활성화됨

호스팅 유형

버킷 호스팅

버킷 웹 사이트 엔드포인트

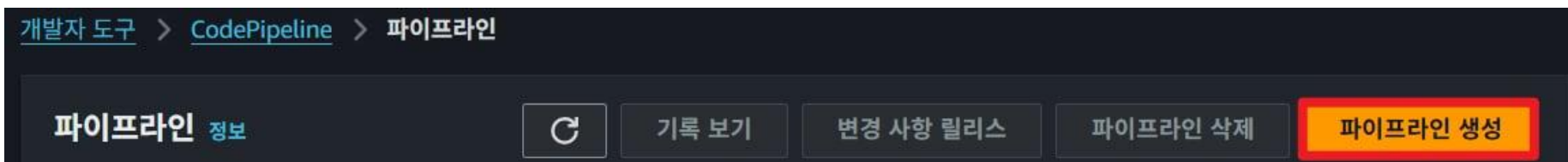
버킷을 정적 웹 사이트로 구성하면, 해당 웹 사이트를 버킷의 AWS 리전별 웹 사이트 엔드포인트에서 사용할 수 있습니다. 자세히 알아보기 [\[?\]](#)

<http://user00-angular-website.s3-website.ap-northeast-2.amazonaws.com>

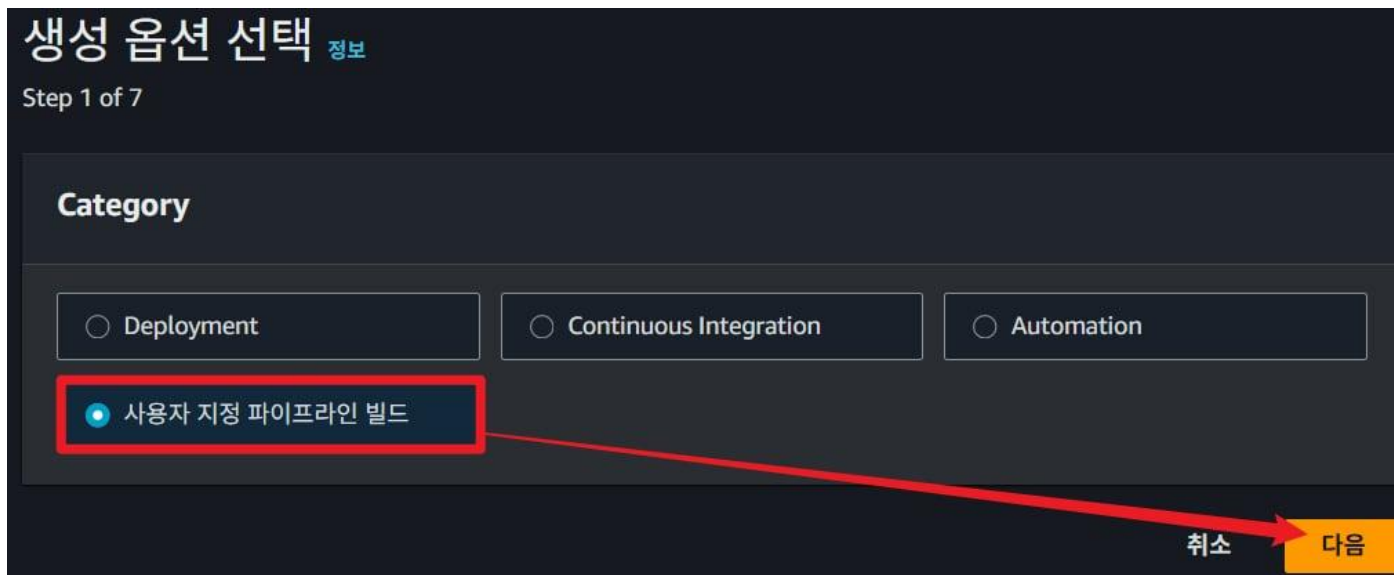
Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

AWS의 CodePipeline에서 "파이프라인 생성"을 클릭합니다.



생성 옵션에서 "사용자 지정 파이프라인 빌드"를 선택합니다.



Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

파이프라인 이름(user##-AngularPipeline)을 설정합니다.

파이프라인 설정 선택

정보

Step 2 of 7

파이프라인 설정

파이프라인 이름

파이프라인 이름을 입력합니다. 생성된 후에는 파이프라인 이름을 편집할 수 없습니다.

user00-AngularPipeline

100자를 초과할 수 없음

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

소스 공급자를 "GitHub(GitHub 앱을 통해)"를 선택하고, "GitHub에 연결"을 클릭합니다.

소스 스테이지 추가 정보

Step 3 of 7

소스

소스 공급자

파이프라인의 입력 아티팩트를 저장한 위치입니다. 공급자를 선택한 후 연결 세부 정보를 제공하십시오.

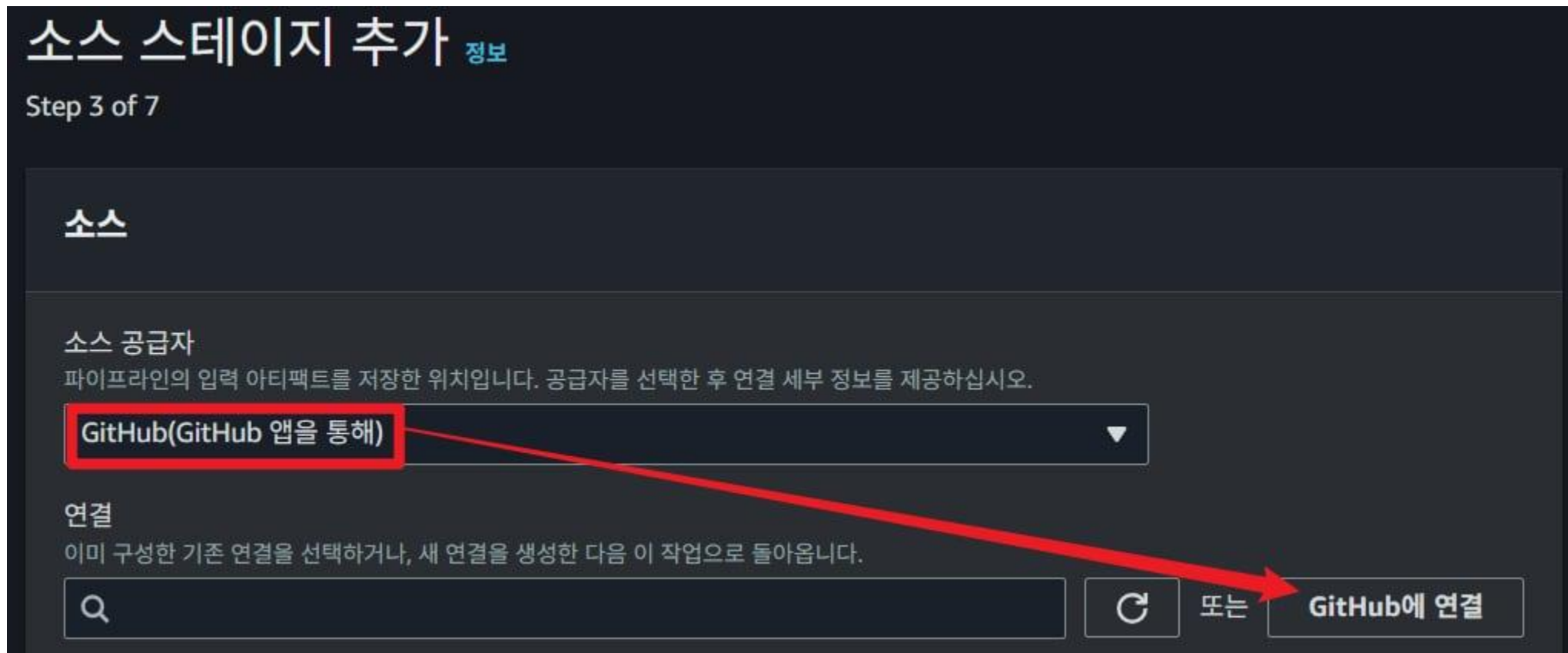
GitHub(GitHub 앱을 통해) ▼

연결

이미 구성된 기존 연결을 선택하거나, 새 연결을 생성한 다음 이 작업으로 돌아옵니다.

Q

↻ 또는 **GitHub에 연결**



Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

연결 이름(user##-github-angular)을 작성하고, GitHub에 연결을 클릭합니다.

연결 생성 정보

GitHub 앱 연결 생성 정보

연결 이름

user00-github-angular

▶ 태그 - 선택 사항

GitHub에 연결

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

기존에 생성했던 앱을 선택하고, "연결"을 클릭합니다.

GitHub에 연결

GitHub 연결 설정 정보

연결 이름

앱 설치 - 선택 사항

GitHub 앱을 설치하여 봇으로 연결합니다. 또는 GitHub 사용자로 연결하여 AWS CodeBuild 프로젝트에서 사용할 수 있도록 비워 둘 수도 있습니다.

×
또는

▶ 태그 - 선택 사항

연결

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

리포지토리 이름과 기본 브랜치를 선택하고, 다음을 클릭합니다.

리포지토리 이름
GitHub 계정에서 리포지토리를 선택합니다.

Q [redacted]/test-angular-repo X

제공된 자격 증명이 액세스할 수 있는 모든 프로젝트에 그룹 경로를 입력하거나 붙여넣을 수 있습니다. '그룹/하위 그룹/프로젝트' 형식을 사용합니다.

기본 브랜치
기본 브랜치는 파이프라인 실행이 다른 소스에서 시작되거나 수동으로 시작된 경우에만 사용됩니다.

Q main X

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

빌드 공급자를 "기타 빌드 공급자"로 체크하고, "AWS CodeBuild"를 선택한 후, "프로젝트 생성"을 클릭합니다.

빌드 스테이지 추가

정보

Step 4 of 7

빌드 - 선택 사항

빌드 공급자

빌드 명령을 실행하고 빌드 작업의 아티팩트를 지정하는 데 사용할 도구를 선택합니다.

☐ 명령

☒ 기타 빌드 공급자

AWS CodeBuild

▼

프로젝트 이름

AWS CodeBuild 콘솔에서 이미 생성한 빌드 프로젝트를 선택합니다. 또는 AWS CodeBuild 콘솔에서 빌드 프로젝트를 생성한 후 이 작업으로 돌아옵니다.

또는

프로젝트 생성

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

빌드 프로젝트 생성에서 프로젝트의 이름(user##-AngularBuild)을 작성합니다.

빌드 프로젝트 생성

프로젝트 구성

프로젝트 이름

user00-AngularBuild

프로젝트 이름은 2~255자여야 합니다. 글자(A-Z 및 a-z), 숫자(0-9) 및 특수 문자(- 및 _)를 포함할 수 있습니다.

프로젝트 유형

만들려는 프로젝트 유형을 선택합니다. [정보](#)

☒ 기본 프로젝트

사용자 지정 CodeBuild 프로젝트를 생성합니다.

☐ 러너 프로젝트

GitHub Actions, GitHub Enterprise Actions, GitLab 또는 Buildkite에서 워크플로를 위한 CodeBuild 관리형 러너를 만듭니다.

▼ 추가 구성

설명, 공개 빌드 액세스, 빌드 배지, 동시 빌드 제한, 태그

설명 - 선택 사항

The build project for my Angular pipeline

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

아래쪽의 환경에서 운영체제와 런타임, 이미지를 선택합니다.

▼ 환경

환경 이미지

☒ 관리형 이미지
AWS CodeBuild에서 관리하는 이미지 사용

☐ 사용자 지정 이미지
도커 이미지 지정

실행 모드

☒ 컨테이너
Docker 컨테이너에서 실행

☐ 인스턴스
EC2 인스턴스에서 직접 실행

운영 체제

Ubuntu

▼

런타임

Standard

▼

이미지

aws/codebuild/standard:7.0

▼

이미지 버전

이 런타임 버전에 항상 최신 이미지 사용

▼

☐ GPU 강화 컴퓨팅 사용

KT AIVLE School

201

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

Buildspec에서 "buildspec 파일 사용"을 체크하고, 아래쪽에 "CodePipeline으로 계속"을 클릭합니다.

▼ Buildspec

빌드 사양

☐ 빌드 명령 삽입
빌드 명령을 빌드 프로젝트 구성으로 저장

☒ buildspec 파일 사용
YAML 형식의 buildspec 파일에 빌드 명령 저장

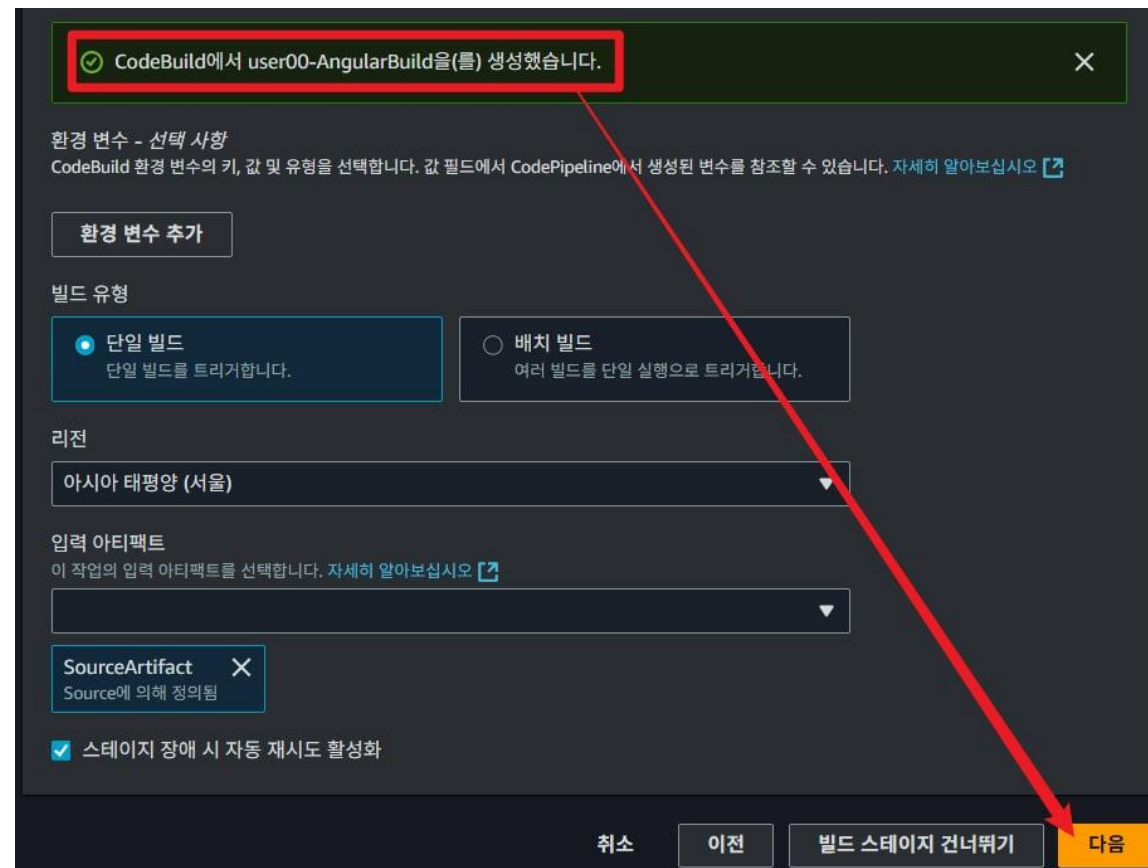
Buildspec 이름 - 선택 사항
기본적으로 CodeBuild는 소스 코드 루트 디렉터리에서 buildspec.yml 파일을 찾습니다. buildspec 파일이 다른 이름 또는 위치를 사용하는 경우 여기에 소스 루트의 경로를 입력하십시오(예: buildspec-two.yml 또는 configuration/buildspec.yml).

buildspec.yml

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

CodeBuild에서 프로젝트 생성됨을 확인하고, "다음"을 클릭해 다음 단계로 넘어갑니다.



CodeBuild에서 user00-AngularBuild을(를) 생성했습니다.

환경 변수 - 선택 사항
CodeBuild 환경 변수의 키, 값 및 유형을 선택합니다. 값 필드에서 CodePipeline에서 생성된 변수를 참조할 수 있습니다. 자세히 알아보십시오

환경 변수 추가

빌드 유형

☒ 단일 빌드
단일 빌드를 트리거합니다.

☐ 배치 빌드
여러 빌드를 단일 실행으로 트리거합니다.

리전
아시아 태평양 (서울)

입력 아티팩트
이 작업의 입력 아티팩트를 선택합니다. 자세히 알아보십시오

SourceArtifact X
Source에 의해 정의됨

☒ 스테이지 장애 시 자동 재시도 활성화

취소 이전 빌드 스테이지 건너뛰기 다음

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

테스트 스테이지는 건너뛰니다.

테스트 스테이지 추가
정보

Step 5 of 7

테스트 - 선택 사항

테스트 제공자

애플리케이션 또는 콘텐츠를 테스트할 방법을 선택합니다. 공급자를 선택한 다음 해당 공급자에 대한 구성 세부 정보를 입력하세요.

▼

☒ 스테이지 장애 시 자동 재시도 활성화

취소

이전

테스트 스테이지 건너뛰기

다음

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

배포 스테이지에서 배포 공급자, 버킷 등을 설정하고, "다음"을 클릭합니다.

배포 스테이지 추가 정보

Step 6 of 7

배포 - 선택 사항

배포 공급자
에클립스 또는 본연조를 배포할 방법을 선택합니다. 공급자를 선택한 다음 해당 공급자에 대한 구성 세부 정보를 입력하세요.

Amazon S3

리전
아시아 태평양 (서울)

입력 아티팩트
이 작업의 입력 아티팩트를 선택합니다. 자세히 알아보십시오

BuildArtifact
Build에 의해 정의됨
100자 이하

버킷
user00-angular-website

배포 경로 - 선택 사항

☒ 배포하기 전에 파일 압축 풀기
아티팩트를 배포하기 전에 아티팩트의 압축을 풉니다.

추가 구성

KMS 암호화 키 ARN - 선택 사항
KMS 암호화 키를 사용하여 객체를 암호화합니다. 키를 입력하지 않으면 객체가 암호화되지 않은 상태로 유지됩니다.

표준 ACL - 선택 사항
버킷에 대한 Amazon S3 표준 ACL(역세스 제어 목록)을 지정합니다.

public-read

캐시 제어 - 선택 사항
Amazon S3 버킷에서 요청된 객체에 대한 캐시 제어를 설정합니다.

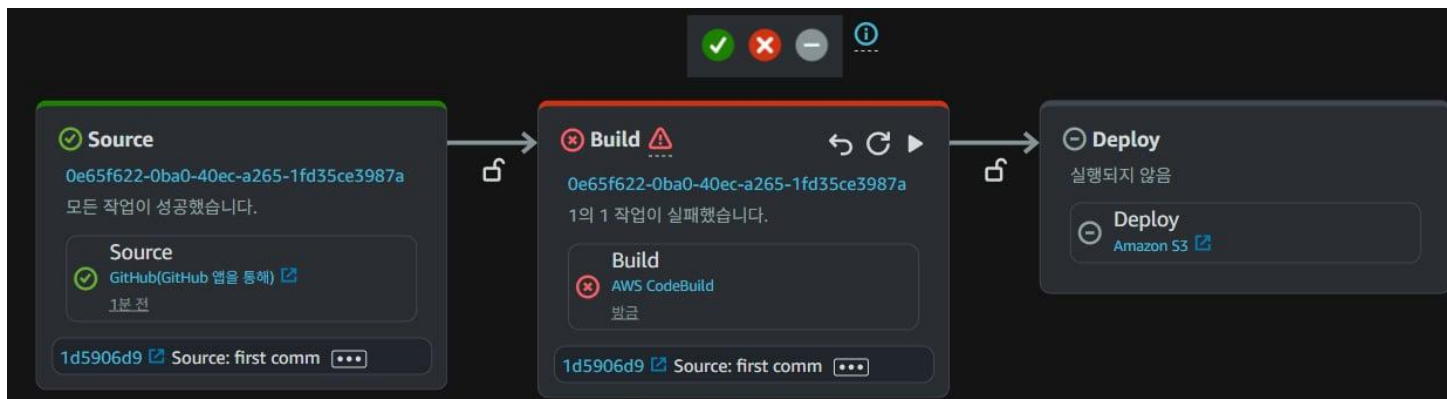
☒ 스테이지 장에 시 자동 목록 구성
스테이지 장에 시 자동 재시도 활성화

취소 이전 배포 스테이지 건너뛰기 다음

Pipeline 생성 with Build Stage

Step 1: Pipeline 생성

파이프라인을 생성하면 Source 단계는 성공하지만 Build에서 실패합니다.(2~3분소요)



로그를 확인해(에러 메시지를 클릭) 빌드가 실패한 원인을 파악합니다.

로그

요약

입력

빌드 로그의 마지막 7줄을 표시하는 중입니다. 전체 로그 보기

테일 로그

이전 로그 없음

```

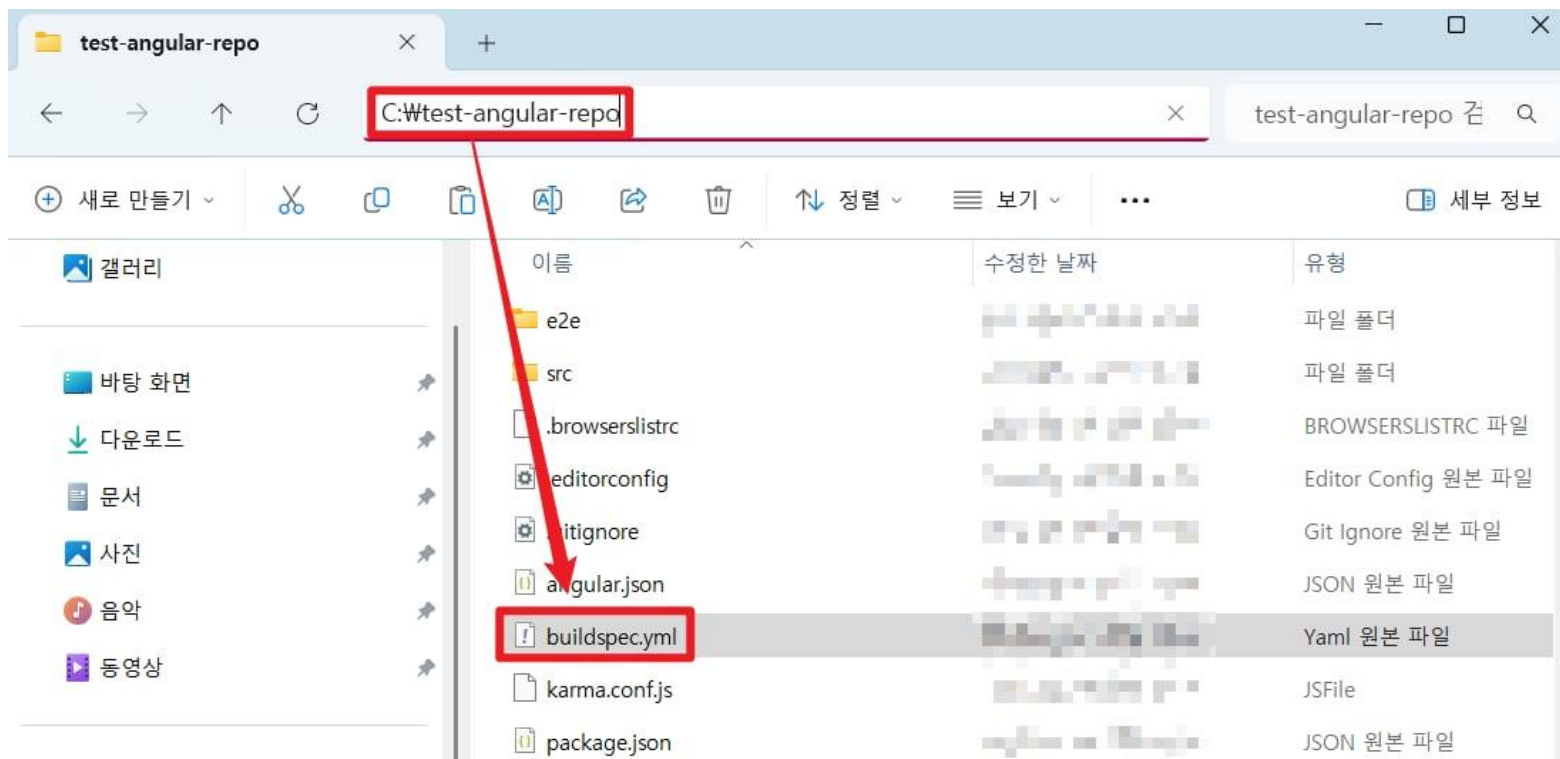
1 [Container] 2025/08/14 02:27:48.220312 Running on CodeBuild On-demand
2 [Container] 2025/08/14 02:27:48.220323 Waiting for agent ping
3 [Container] 2025/08/14 02:27:48.421843 Waiting for DOWNLOAD_SOURCE
4 [Container] 2025/08/14 02:27:50.832618 Phase is DOWNLOAD_SOURCE
5 [Container] 2025/08/14 02:27:50.833741 CODEBUILD_SRC_DIR=/codebuild/output/src1301412082/src
6 [Container] 2025/08/14 02:27:50.840155 Phase complete: DOWNLOAD_SOURCE State: FAILED
7 [Container] 2025/08/14 02:27:50.840249 Phase context status code: YAML_FILE_ERROR Message: YAML file does not exist
8

```

Buildspec 파일 생성

Step 1: Buildspec 파일 업로드

제공 받은 파일(buildspec.yml)을 로컬에 업로드합니다.



Buildspec 파일 생성

Step 2: GitHub에 업데이트

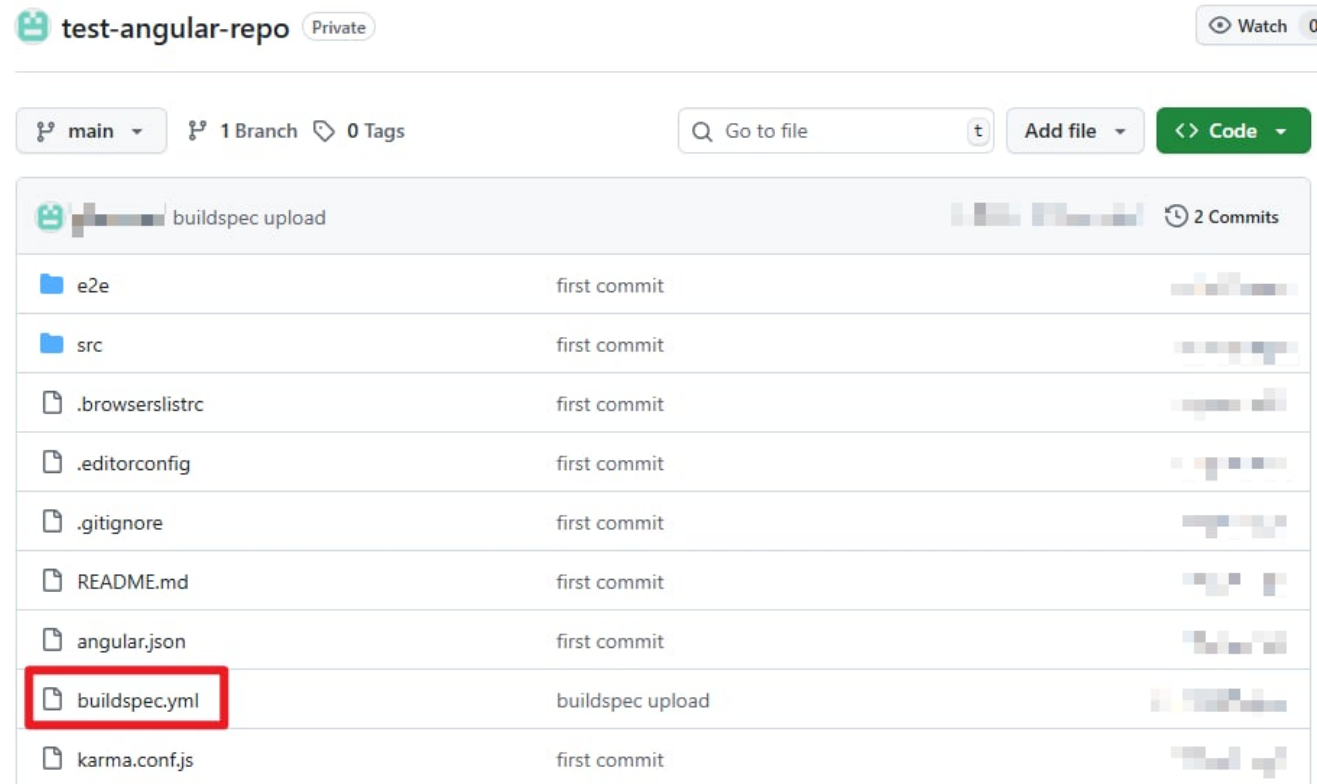
해당 파일이 위치한 곳에서 터미널을 실행하여 GitHub에 파일을 업데이트합니다.

```
git add .  
git commit -m "buildspec upload"  
git push -u origin main
```

```
PS C:\test-angular-repo> git commit -m "buildspec upload"  
[main d19ccd3] buildspec upload  
1 file changed, 17 insertions(+)  
create mode 100644 buildspec.yml  
PS C:\test-angular-repo> git push origin main  
Enumerating objects: 4, done.  
Counting objects: 100% (4/4), done.  
Delta compression using up to 18 threads  
Compressing objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 443 bytes | 443.00 KiB/s, done.  
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/joteacher/test-angular-repo.git  
1d5906d..d19ccd3 main -> main
```

Step 2: GitHub에 업데이트

GitHub에 "buildspec.yaml" 파일이 업로드 되었는지 확인합니다.



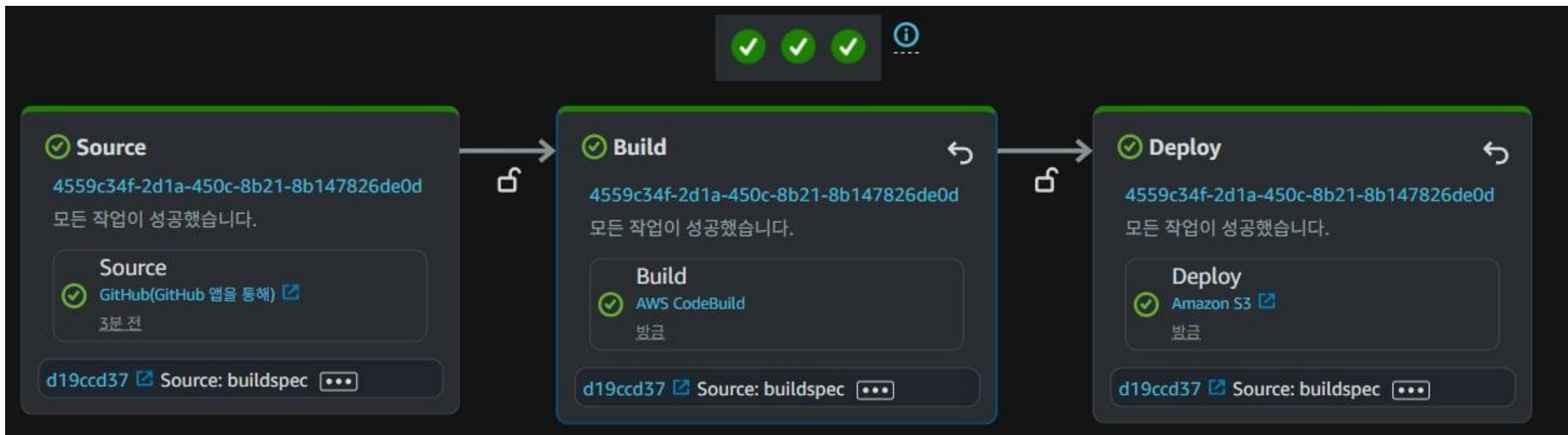
The screenshot shows the GitHub interface for a repository named 'test-angular-repo'. The repository is private and has 1 branch (main) and 0 tags. The file list shows the following files and their commit history:

File Name	Commit History
e2e	first commit
src	first commit
.browserslistrc	first commit
.editorconfig	first commit
.gitignore	first commit
README.md	first commit
angular.json	first commit
buildspec.yaml	buildspec upload
karma.conf.js	first commit

Buildspec 파일 생성

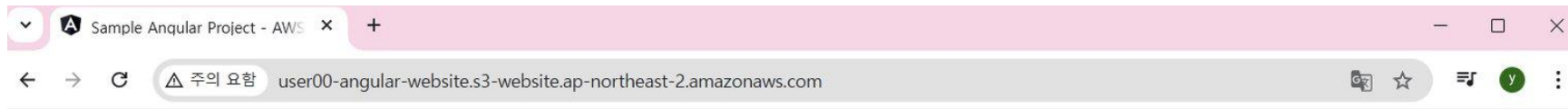
Step 3: Build 단계 확인

AWS CodePipeline에서 Build 단계가 성공했음을 확인합니다.



Step 4: 웹 사이트 접속

웹사이트 접속이 되는지 확인합니다.



Sample Angular App for AWS CodePipeline Step by Step

Version: 1.0

Congratulations! You successfully built and deployed your code.

This is a simple single-page calculator app developed with Angular and Bootstrap for the build examples on the AWS CodePipeline Step by Step course.

Simple Calculator

Please select an operator



Clear

Calculate

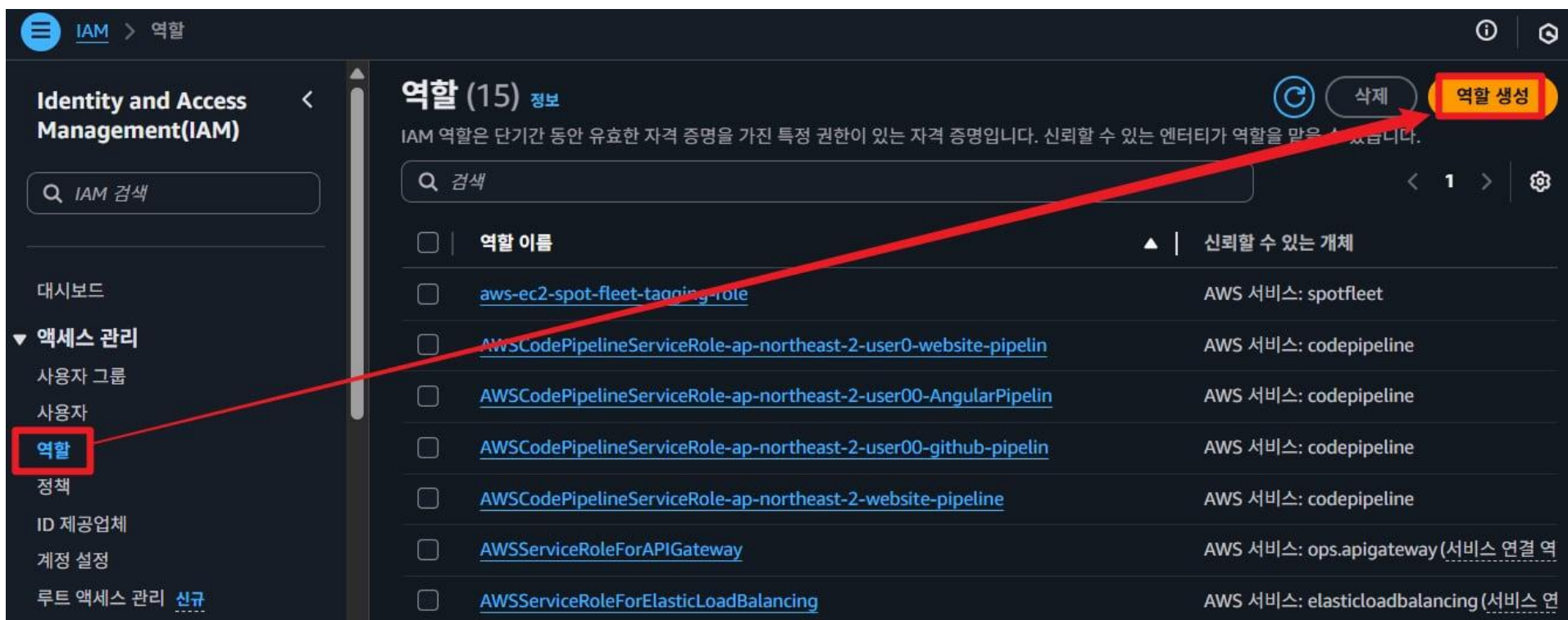
Lab 12.

AWS CodeDeploy로 EC2 인스턴스에 배포하기

EC2 Instance Configuration

Step 1: IAM Role 생성

IAM 서비스에서 "역할 생성"을 클릭합니다.



EC2 Instance Configuration

Step 1: IAM Role 생성

신뢰할 수 있는 엔터티 유형은 "AWS 서비스"를 선택하고, 사용 사례는 "EC2"를 선택합니다.

신뢰할 수 있는 엔터티 선택
정보

신뢰할 수 있는 엔터티 유형

☒ AWS 서비스
EC2, Lambda 등의 AWS 서비스가 이 계정에 서 작업을 수행하도록 허용합니다.

☐ AWS 계정
사용자 또는 서드 파티에 속한 다른 AWS 계정의 엔터티가 이 계정에서 작업을 수행하도록 허용합니다.

☐ 웹 자격 증명
지정된 외부 웹 ID 제공업체와 연동된 사용자가 이 역할을 맡아 이 계정에서 작업을 수행하도록 허용합니다.

☐ SAML 2.0 페더레이션
기업 디렉터리에서 SAML 2.0과 연동된 사용자가 이 계정에서 작업을 수행할 수 있도록 허용합니다.

☐ 사용자 지정 신뢰 정책
다른 사용자가 이 계정에서 작업을 수행할 수 있도록 사용자 지정 신뢰 정책을 생성합니다.

사용 사례

EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

서비스 또는 사용 사례

EC2

지정된 서비스에 대한 사용 사례를 선택합니다.

사용 사례

☒ EC2
Allows EC2 instances to call AWS services on your behalf.

☐ EC2 Role for AWS Systems Manager
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.

EC2 Instance Configuration

Step 1: IAM Role 생성

"CodeDeploy"라고 검색하면, "AmazonEC2RoleforAWSCodeDeploy"가 있습니다. 선택하세요.

권한 추가

권한 정책 (1/1081)

새 역할에 연결할 정책을 하나 이상 선택합니다.

CodeDeploy

✕

필터링 기준 유형

모든 유형

11 개 일치

정책 이름

유형

설명

☒	AmazonEC2RoleforAWSCodeDeploy	AWS 관리형	Provides EC2 access to S3 bucket to download revision. This role is needed ...
☐	AmazonEC2RoleforAWSCodeDeployLimited	AWS 관리형	Provides EC2 limited access to S3 bucket to download revision. This role is ...
☐	AWSCodeDeployDeployerAccess	AWS 관리형	Provides access to register and deploy a revision.

EC2 Instance Configuration

Step 1: IAM Role 생성

역할 이름(user##-WebServerRole)을 지정하고, "역할 생성"을 클릭합니다.

이름 지정, 검토 및 생성

역할 세부 정보

역할 이름

이 역할을 식별하는 의미 있는 이름을 입력합니다.

user00-WebServerRole

최대 64자입니다. 영숫자 및 '+', '@', '-' 문자를 사용하세요.

설명

이 역할에 대하여 간단한 설명을 추가합니다.

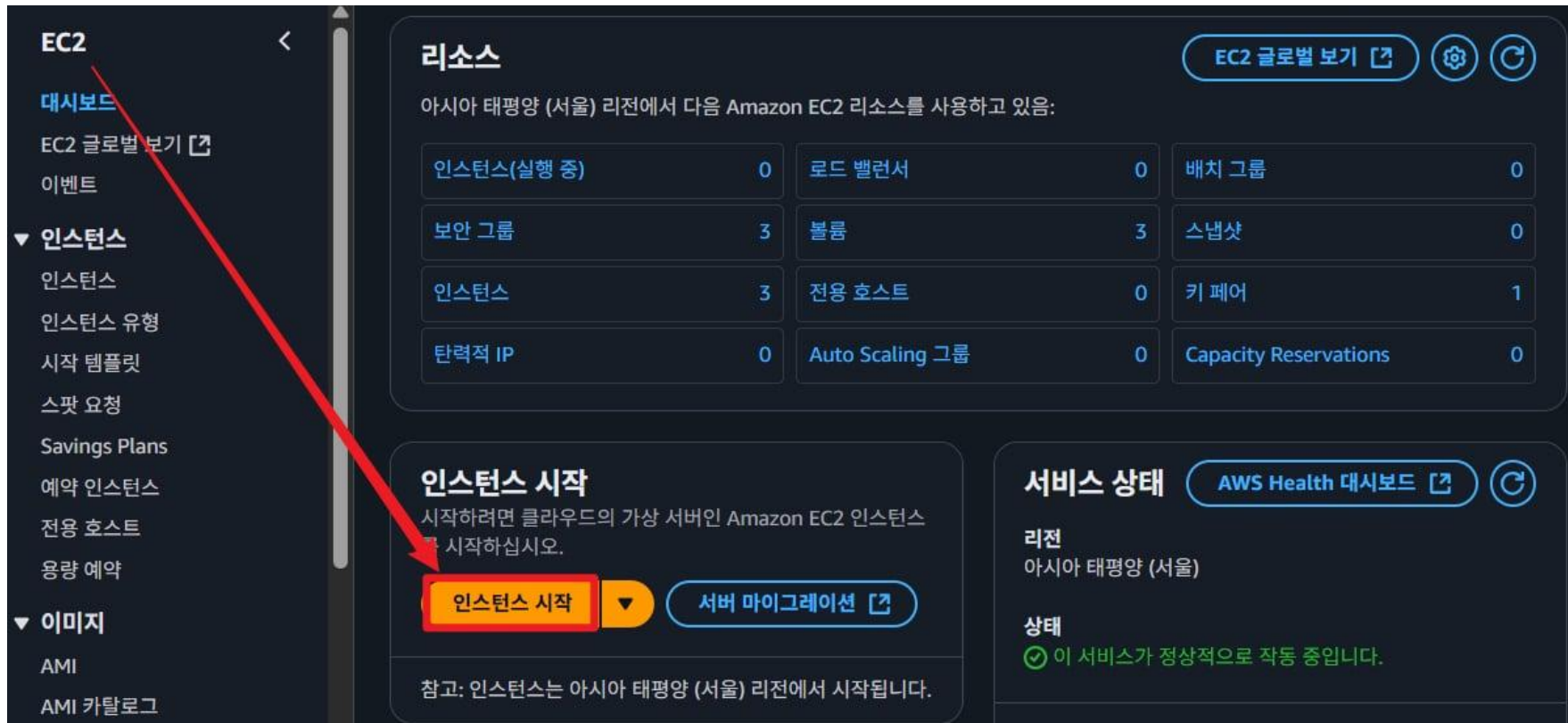
Allows EC2 instances to call AWS services on your behalf.

최대 문자 수: 1000. 문자(A~Z 및 a~z), 숫자(0~9), 탭, 새 줄 또는 다음 문자 중 하나를 사용합니다: _+=, @-/[0]!#\$%^&*(){};:'" `

EC2 Instance Configuration

Step 2: EC2 Instance 생성

EC2 서비스에서 "인스턴스 시작"을 클릭합니다.



EC2

대시보드

EC2 글로벌 보기

이벤트

▼ 인스턴스

인스턴스

인스턴스 유형

시작 템플릿

스팟 요청

Savings Plans

예약 인스턴스

전용 호스트

용량 예약

▼ 이미지

AMI

AMI 카탈로그

리소스

아시아 태평양 (서울) 리전에서 다음 Amazon EC2 리소스를 사용하고 있음:

인스턴스(실행 중)	0	로드 밸런서	0	배치 그룹	0
보안 그룹	3	볼륨	3	스냅샷	0
인스턴스	3	전용 호스트	0	키 페어	1
탄력적 IP	0	Auto Scaling 그룹	0	Capacity Reservations	0

인스턴스 시작

시작하려면 클라우드의 가상 서버인 Amazon EC2 인스턴스를 시작하십시오.

인스턴스 시작 서버 마이그레이션

참고: 인스턴스는 아시아 태평양 (서울) 리전에서 시작됩니다.

서비스 상태 AWS Health 대시보드

리전
아시아 태평양 (서울)

상태
이 서비스가 정상적으로 작동 중입니다.

EC2 Instance Configuration

Step 2: EC2 Instance 생성

인스턴스의 이름, AMI, 유형, 키 페어, 네트워크 설정합니다.

- ✓ 이름: user##-AngularProject
- ✓ Amazon Machine Image(AMI): Amazon Linux 2023 Kernel-6.1 AMI
- ✓ 인스턴스 유형: t3.micro
- ✓ 키 페어(로그인): 키 생성(user##-keypair) 후 선택
- ✓ 네트워크 설정: 다음에서 SSH 트래픽 허용, 인터넷에서 HTTPS 트래픽 허용, 인터넷에서 HTTP 트래픽 허용 체크

EC2 Instance Configuration

Step 2: EC2 Instance 생성

인스턴스의 이름, AMI, 유형, 키 페어, 네트워크 설정합니다.

✓ 네트워크 설정: 다음에서 SSH 트래픽 허용, 인터넷에서 HTTPS 트래픽 허용, 인터넷에서 HTTP 트래픽 허용 체크

네트워크 설정 정보

네트워크 정보

vpc-07d66028b71795240

서브넷 정보

subnet-033335edc6c793d96 | my-subnet-01

퍼블릭 IP 자동 할당 정보

비활성화

방화벽(보안 그룹) 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

보안 그룹 생성

기존 보안 그룹 선택

다음 규칙을 사용하여 'launch-wizard-1'(이)라는 새 보안 그룹을 생성합니다.

다음에서 SSH 트래픽 허용

인스턴스 연결에 도움

인터넷에서 HTTPS 트래픽 허용

예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

인터넷에서 HTTP 트래픽 허용

예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

위치 무관

0.0.0.0/0

⚠ 소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

KT AIVLE School

219

EC2 Instance Configuration

Step 2: EC2 Instance 생성

인스턴스의 이름, AMI, 유형, 키 페어, 네트워크 설정합니다.

✓ 네트워크 설정: 퍼블릭 IP 자동 할당을 활성화로 변경

▼

네트워크 설정

정보

VPC – 필수

정보

vpc-07d66028b71795240

172.31.0.0/16

(기본값) ▼

↺

서브넷

정보

subnet-033335edc6c793d96

my-subnet-01

VPC: vpc-07d66028b71795240 소유자: 698047352521

가용 영역: ap-northeast-2d (apne2-az4) 영역 유형: 가용 영역 사용 가능한 IP 주소: 4088

CIDR: 172.31.0.0/20

▼

↺

새 서브넷 생성 ↗

퍼블릭 IP 자동 할당

정보

활성화

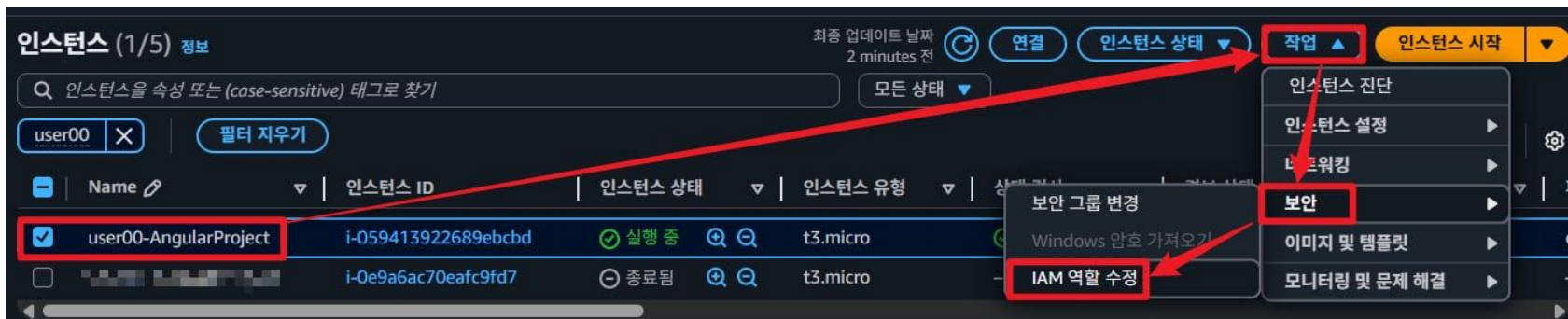
▼

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

EC2 Instance Configuration

Step 2: EC2 Instance 생성

생성한 인스턴스를 선택하고, "작업 > 보안 > IAM 역할 수정"을 클릭합니다.



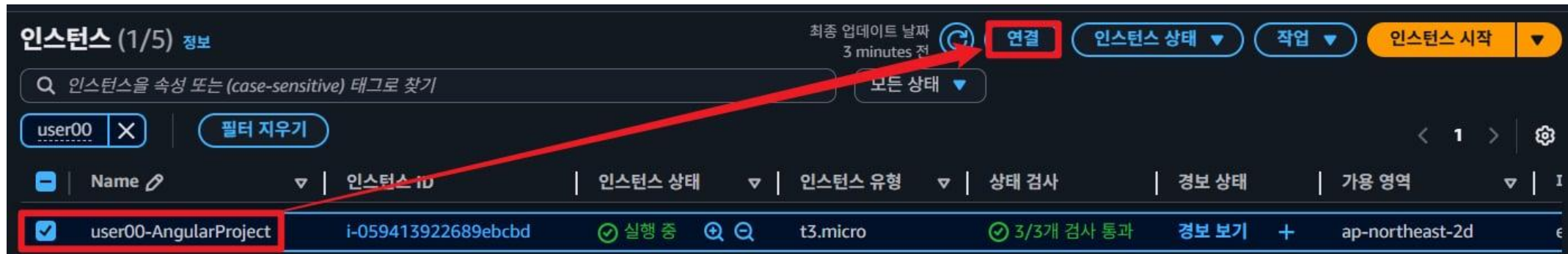
생성한 역할(user##-WebServerRole)을 선택하고, "IAM 역할 업데이트"를 클릭합니다.



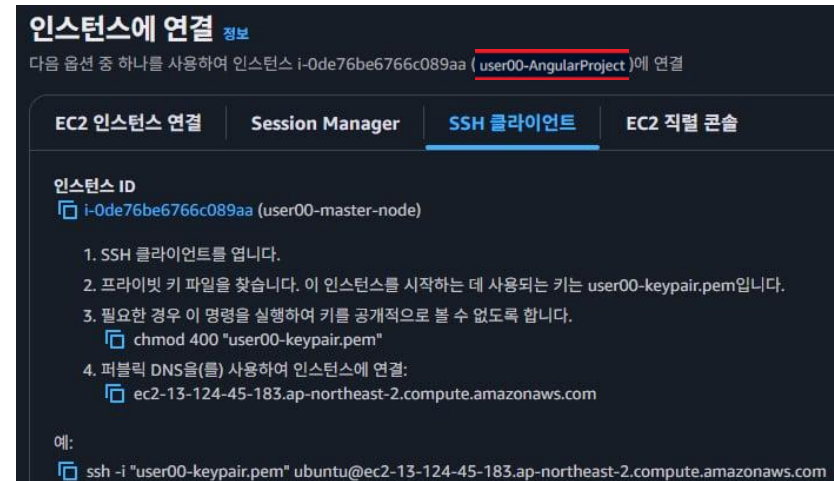
EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

인스턴스를 선택하고, "연결"을 클릭해 인스턴스에 연결합니다.



SSH 클라이언트 명령어로 CloudShell을 통해 접속합니다.



EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

패키지를 업데이트 하고, 필요한 도구를 설치합니다.

```
sudo yum update -y
sudo yum install -y wget ruby
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-10-173 ~]$ sudo yum install -y wget ruby
Last metadata expiration check: 0:00:15 ago on Mon Aug 18 02:07:51 2025.
Package wget-1.21.3-1.amzn2023.0.4.x86_64 is already installed.
Dependencies resolved.
```

Package	Architecture	Version
Installing:		
ruby3.2	x86_64	3.2.8-184.amzn2023.0.4
Installing dependencies:		
ruby3.2-default-gems	noarch	3.2.8-184.amzn2023.0.4
ruby3.2-libs	x86_64	3.2.8-184.amzn2023.0.4
ruby3.2-rubygem-io-console	x86_64	0.6.0-184.amzn2023.0.4
ruby3.2-rubygem-json	x86_64	2.6.3-184.amzn2023.0.4
ruby3.2-rubygem-psych	x86_64	5.0.1-184.amzn2023.0.4
Installing weak dependencies:		
ruby3.2-rubygem-bigdecimal	x86_64	3.1.3-184.amzn2023.0.4
ruby3.2-rubygem-bundler	noarch	2.4.19-184.amzn2023.0.4
ruby3.2-rubygem-rdoc	noarch	6.5.1.1-184.amzn2023.0.4
ruby3.2-rubygems	noarch	3.4.19-184.amzn2023.0.4

```
Transaction Summary
```


Step 3: 인스턴스 연결 및 설정

CodeDeploy 의 install 패키지 설치합니다.

```
wget https://aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.amazonaws.com/latest/install
```

```
[ec2-user@ip-172-31-10-173 ~]$ wget https://aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.amazonaws.com/latest/install
--2025-08-18 02:14:51-- https://aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.amazonaws.com/latest/install
Resolving aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.amazonaws.com (aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.
0, 52.219.204.50, ...
Connecting to aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.amazonaws.com (aws-codedeploy-ap-northeast-2.s3.ap-northeast-2.
ected.
HTTP request sent, awaiting response... 200 OK
Length: 19045 (19K) []
Saving to: 'install'

install                                     100%[=====]

2025-08-18 02:14:51 (39.2 MB/s) - 'install' saved [19045/19045]
```

Step 3: 인스턴스 연결 및 설정

"install"은 실행 권한이 없습니다. 실행 권한을 부여합니다.

```
chmod +x install  
ls -al
```

```
[ec2-user@ip-172-31-10-173 ~]$ chmod +x install  
[ec2-user@ip-172-31-10-173 ~]$ ls -al  
total 32  
drwx----- 3 ec2-user ec2-user 89 Aug 18 02:14 .  
drwxr-xr-x 3 root root 22 Aug 18 01:54 ..  
-rw-r--r-- 1 ec2-user ec2-user 18 Jan 28 2023 .bash_logout  
-rw-r--r-- 1 ec2-user ec2-user 141 Jan 28 2023 .bash_profile  
-rw-r--r-- 1 ec2-user ec2-user 492 Jan 28 2023 .bashrc  
drwx----- 2 ec2-user ec2-user 29 Aug 18 01:54 .ssh  
-rwxr-xr-x 1 ec2-user ec2-user 19045 Aug 5 19:12 install
```

Step 3: 인스턴스 연결 및 설정

"install"을 실행합니다.

```
sudo ./install auto
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo ./install auto
I, [2025-08-18T02:19:07.553369 #26461] INFO -- : Starting Ruby version check.
W, [2025-08-18T02:19:07.553446 #26461] WARN -- : The Ruby version in /usr/bin/ruby3.2 is 3.2.8, .
I, [2025-08-18T02:19:07.553474 #26461] INFO -- : Starting update check.
I, [2025-08-18T02:19:07.553502 #26461] INFO -- : Attempting to automatically detect supported pack
I, [2025-08-18T02:19:07.568907 #26461] INFO -- : Checking AWS_REGION environment variable for regi
I, [2025-08-18T02:19:07.568954 #26461] INFO -- : Checking EC2_metadata service for region informat
I, [2025-08-18T02:19:07.584574 #26461] INFO -- : Checking AWS_DOMAIN environment variable for doma
I, [2025-08-18T02:19:07.584614 #26461] INFO -- : Checking EC2_metadata service for domain informat
I, [2025-08-18T02:19:07.587460 #26461] INFO -- : Downloading version file from bucket aws-codedeplo
I, [2025-08-18T02:19:07.587575 #26461] INFO -- : Endpoint: https://aws-codedeploy-ap-northeast-2.s
I, [2025-08-18T02:19:07.649573 #26461] INFO -- : Downloading package from bucket aws-codedeploy-ap
m...
I, [2025-08-18T02:19:07.649689 #26461] INFO -- : Endpoint: https://aws-codedeploy-ap-northeast-2.s
.noarch.rpm
I, [2025-08-18T02:19:07.719975 #26461] INFO -- : Executing `/usr/bin/yum -y localinstall /tmp/code
Last metadata expiration check: 0:11:17 ago on Mon Aug 18 02:07:51 2025.
Dependencies resolved.
```

EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

CodeDeploy Agent가 정상 동작 중인지 확인합니다.

```
sudo systemctl status codedeploy-agent.service
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo systemctl status codedeploy-agent.service
● codedeploy-agent.service - AWS CodeDeploy host Agent
   Loaded: loaded (/usr/lib/systemd/system/codedeploy-agent.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-08-18 02:19:09 UTC; 25s ago
     Main PID: 26600 (ruby)
       Tasks: 3 (limit: 1057)
      Memory: 65.6M
         CPU: 1.239s
       CGroup: /system.slice/codedeploy-agent.service
               └─26600 "codedeploy-agent: master 26600"
                 └─26602 "codedeploy-agent: booting child"
```

Step 3: 인스턴스 연결 및 설정

nginx를 설치합니다.

```
sudo yum install -y nginx
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo yum install -y nginx
Last metadata expiration check: 0:24:14 ago on Mon Aug 18 02:07:51 2025.
Dependencies resolved.
=====
Package                                Architecture                          Version
=====
Installing:
  nginx                                x86_64                                1:1.28.0-1.amzn2023.0.1
Installing dependencies:
  generic-logos-httpd                  noarch                                18.0.0-12.amzn2023.0.3
  gperftools-libs                       x86_64                                2.9.1-1.amzn2023.0.3
  libunwind                             x86_64                                1.4.0-5.amzn2023.0.2
  nginx-core                           x86_64                                1:1.28.0-1.amzn2023.0.1
  nginx-filesystem                      noarch                                1:1.28.0-1.amzn2023.0.1
  nginx-mimetypes                      noarch                                2.1.49-3.amzn2023.0.3
Transaction Summary
=====
```


EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

nginx를 실행합니다.

```
sudo service nginx start  
sudo service nginx status
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo service nginx start  
Redirecting to /bin/systemctl start nginx.service  
[ec2-user@ip-172-31-10-173 ~]$ sudo service nginx status  
Redirecting to /bin/systemctl status nginx.service  
• nginx.service - The nginx HTTP and reverse proxy server  
  Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)  
  Active: active (running) since Mon 2025-08-18 03:32:38 UTC; 2s ago  
    Process: 29554 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)  
    Process: 29555 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)  
    Process: 29556 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)  
   Main PID: 29557 (nginx)  
     Tasks: 3 (limit: 1057)  
    Memory: 3.2M  
       CPU: 55ms  
    CGroup: /system.slice/nginx.service  
            └─29557 "nginx: master process /usr/sbin/nginx"  
              └─29558 "nginx: worker process"  
                └─29559 "nginx: worker process"
```


EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

인스턴스가 재부팅 된다면, 실행 상태가 유지되지 않습니다. 재부팅 되어도 계속 실행되게 설정합니다.

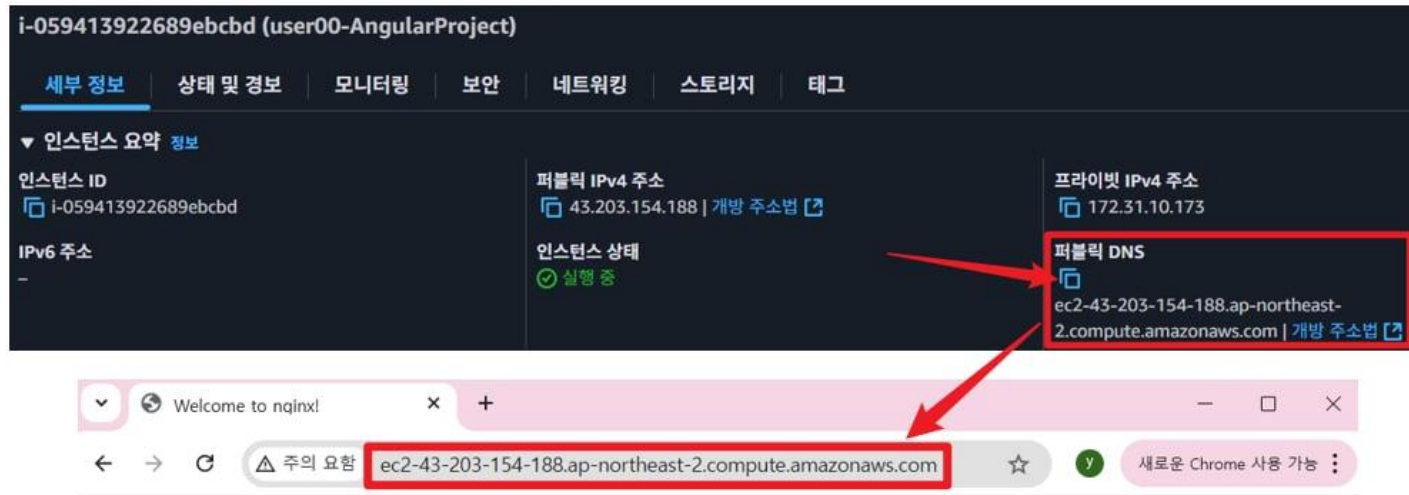
```
sudo systemctl enable nginx.service
```

```
[ec2-user@ip-172-31-10-173 ~]$ sudo systemctl enable nginx.service
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-10-173 ~]$
```

EC2 Instance Configuration

Step 3: 인스턴스 연결 및 설정

해당 인스턴스의 "퍼블릭 DNS"를 확인하고, 웹 브라우저로 접근해봅니다. (nginx가 실행 중입니다.)



The screenshot shows the AWS Management Console for an EC2 instance named 'i-059413922689ebcbd' under the user 'user00-AngularProject'. The 'Instance Summary' tab is selected, showing the instance ID, public IPv4 address (43.203.154.188), and private IPv4 address (172.31.10.173). The 'Public DNS' is highlighted with a red box and labeled '퍼블릭 DNS'. Below this, a browser window is shown with the address bar containing the public DNS: 'ec2-43-203-154-188.ap-northeast-2.compute.amazonaws.com'. A red arrow points from the 'Public DNS' in the console to the browser address bar.

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

EC2 Instance Configuration

Step 4: nginx 파일 수정

nginx 구성 파일을 수정하기 위해 디렉토리를 생성합니다.

```
sudo mkdir -p /var/www/test-angular-project
```

nginx의 구성 파일에서 root의 위치를 생성한 디렉터리로 변경합니다.

```
sudo vim /etc/nginx/nginx.conf
```

```
/var/www/test-angular-project;
```

i : 수정

-- INSERT --

esc : 수정 종료

:qa : 저장 후 닫기

:qa

```
35 include /etc/nginx/conf.d/*.conf;
36
37 server {
38     listen      80;
39     listen      [::]:80;
40     server_name  _;
41     root         /var/www/test-angular-project;
42
43     # Load configuration files for the default server block.
44     include /etc/nginx/default.d/*.conf;
45
46     error_page 404 /404.html;
47     location = /404.html {
48     }
49
50     error_page 500 502 503 504 /50x.html;
51     location = /50x.html {
52     }
53 }
```

Step 4: nginx 파일 수정

nginx를 재시작하고, 웹 브라우저에서 확인합니다.(403 Forbidden)

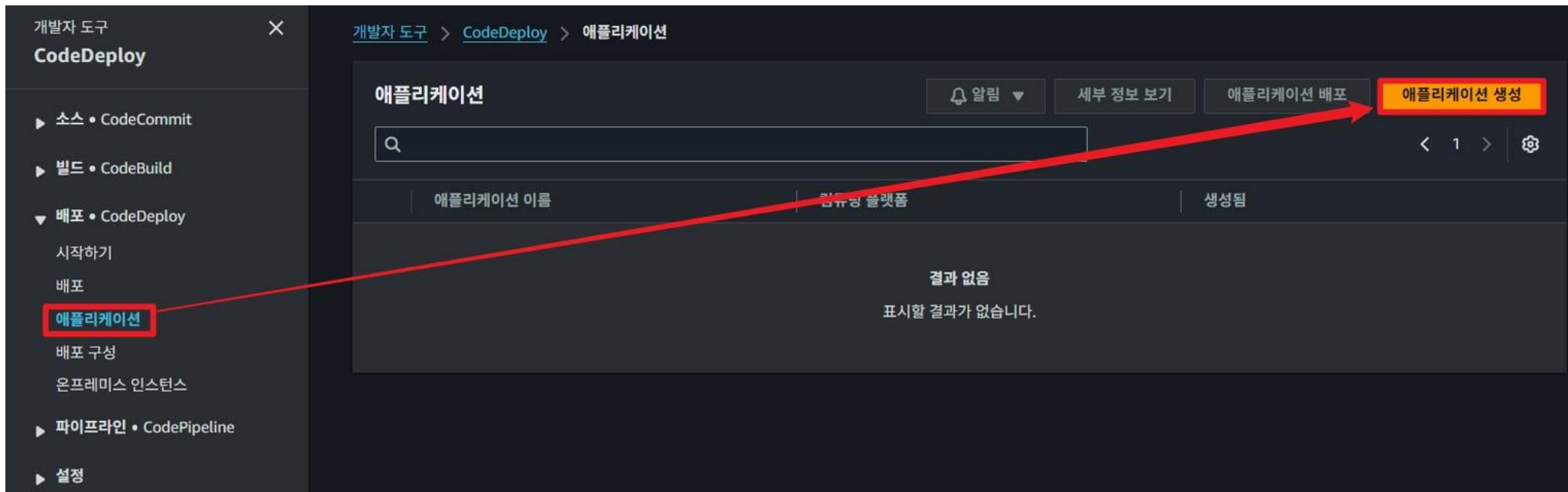
```
sudo service nginx restart
```



CodeDeploy 배포 그룹 만들기

Step 1: 애플리케이션 생성

CodeDeploy에서 "애플리케이션 생성"을 클릭합니다.



CodeDeploy 배포 그룹 만들기

Step 1: 애플리케이션 생성

애플리케이션 이름과 컴퓨팅 플랫폼을 선택합니다.

애플리케이션 생성

애플리케이션 구성

애플리케이션 이름

애플리케이션 이름을 입력합니다

user00AngularApp

100자 이내

컴퓨팅 플랫폼

컴퓨팅 플랫폼 선택

EC2/온프레미스

태그

태그 추가

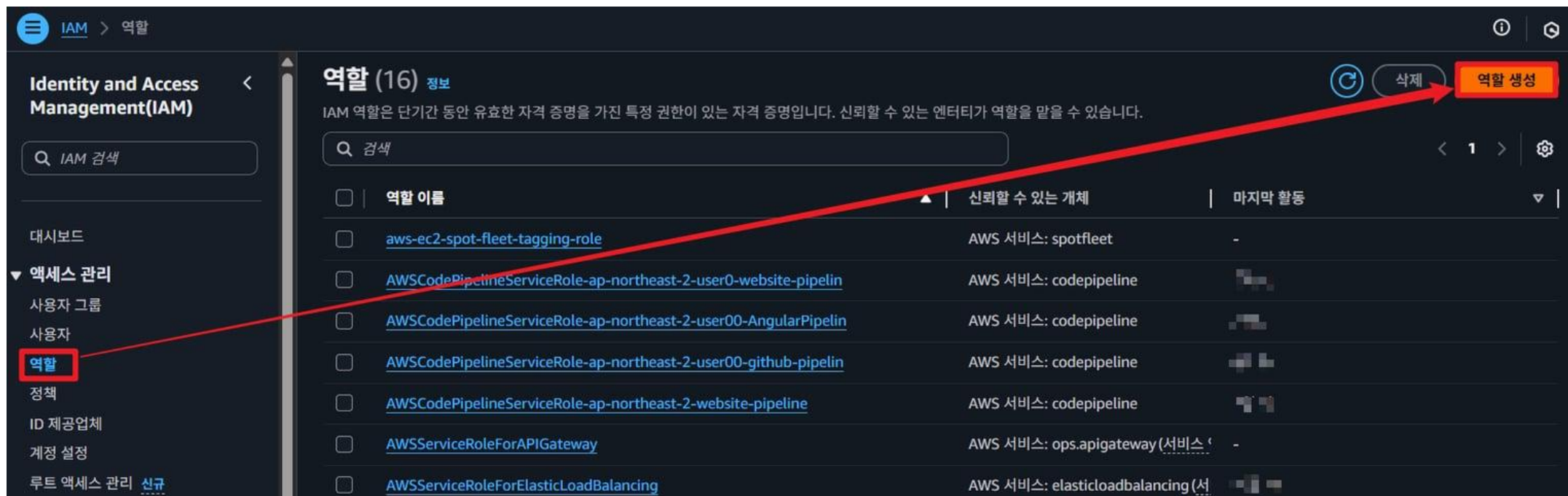
취소

애플리케이션 생성

CodeDeploy 배포 그룹 만들기

Step 2: IAM 역할 생성

CodeDeploy를 위한 IAM 역할을 생성합니다. IAM 서비스에서 "역할 생성"을 클릭합니다.



The screenshot shows the AWS IAM console interface. On the left sidebar, under 'Identity and Access Management(IAM)', the 'Roles' link is highlighted with a red box. A red arrow points from this link to the 'Create Role' button (labeled '역할 생성') in the top right corner of the main content area. The main content area displays a list of existing roles with columns for 'Role Name', 'Trusted Entities', and 'Last Activity'.

역할 이름	신뢰할 수 있는 개체	마지막 활동
aws-ec2-spot-fleet-tagging-role	AWS 서비스: spotfleet	-
AWSCodePipelineServiceRole-ap-northeast-2-user0-website-pipelin	AWS 서비스: codepipeline	
AWSCodePipelineServiceRole-ap-northeast-2-user00-AngularPipelin	AWS 서비스: codepipeline	
AWSCodePipelineServiceRole-ap-northeast-2-user00-github-pipelin	AWS 서비스: codepipeline	
AWSCodePipelineServiceRole-ap-northeast-2-website-pipeline	AWS 서비스: codepipeline	
AWSServiceRoleForAPIGateway	AWS 서비스: ops.apigateway(서비스)	-
AWSServiceRoleForElasticLoadBalancing	AWS 서비스: elasticloadbalancing(서)	

CodeDeploy 배포 그룹 만들기

Step 2: IAM 역할 생성

사용 사례에서 "CodeDeploy"를 선택합니다.

신뢰할 수 있는 엔티티 선택
정보

신뢰할 수 있는 엔티티 유형

☒ AWS 서비스
EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

☐ AWS 계정
사용자 또는 서드 파티에 속한 다른 AWS 계정의 엔티티가 이 계정에서 작업을 수행하도록 허용합니다.

☐ 웹 자격 증명
지정된 외부 웹 ID 제공업체와 연동된 사용자가 이 역할을 맡아 이 계정에서 작업을 수행하도록 허용합니다.

☐ SAML 2.0 페더레이션
기업 디렉터리에서 SAML 2.0과 연동된 사용자가 이 계정에서 작업을 수행할 수 있도록 허용합니다.

☐ 사용자 지정 신뢰 정책
다른 사용자가 이 계정에서 작업을 수행할 수 있도록 사용자 지정 신뢰 정책을 생성합니다.

사용 사례

EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

서비스 또는 사용 사례

CodeDeploy

지정된 서비스에 대한 사용 사례를 선택합니다.

사용 사례

☒ CodeDeploy
Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.

☐ CodeDeploy for Lambda
Allows CodeDeploy to route traffic to a new version of an AWS Lambda function version on your behalf.

☐ CodeDeploy - ECS
Allows CodeDeploy to read S3 objects, invoke Lambda functions, publish to SNS topics, and update ECS services on your behalf.

취소 다음

CodeDeploy 배포 그룹 만들기

Step 2: IAM 역할 생성

AWS 관리형 정책으로 "AWSCodeDeployRole"을 확인합니다.

권한 추가

권한 정책 (1)

선택한 역할 유형에는 다음 정책이 필요합니다.

정책 이름	유형
 AWSCodeDeployRole	AWS 관리형

▶ 권한 경계 설정 - 선택 사항

취소

이전

다음

CodeDeploy 배포 그룹 만들기

Step 2: IAM 역할 생성

역할 이름(user##-CodeDeployEC2ServiceRole)을 지정합니다.

이름 지정, 검토 및 생성

역할 세부 정보

역할 이름

이 역할을 식별하는 의미 있는 이름을 입력합니다.

user00-CodeDeployEC2ServiceRole

최대 64자입니다. 영숫자 및 '+', '@', '_' 문자를 사용하세요.

설명

이 역할에 대하여 간단한 설명을 추가합니다.

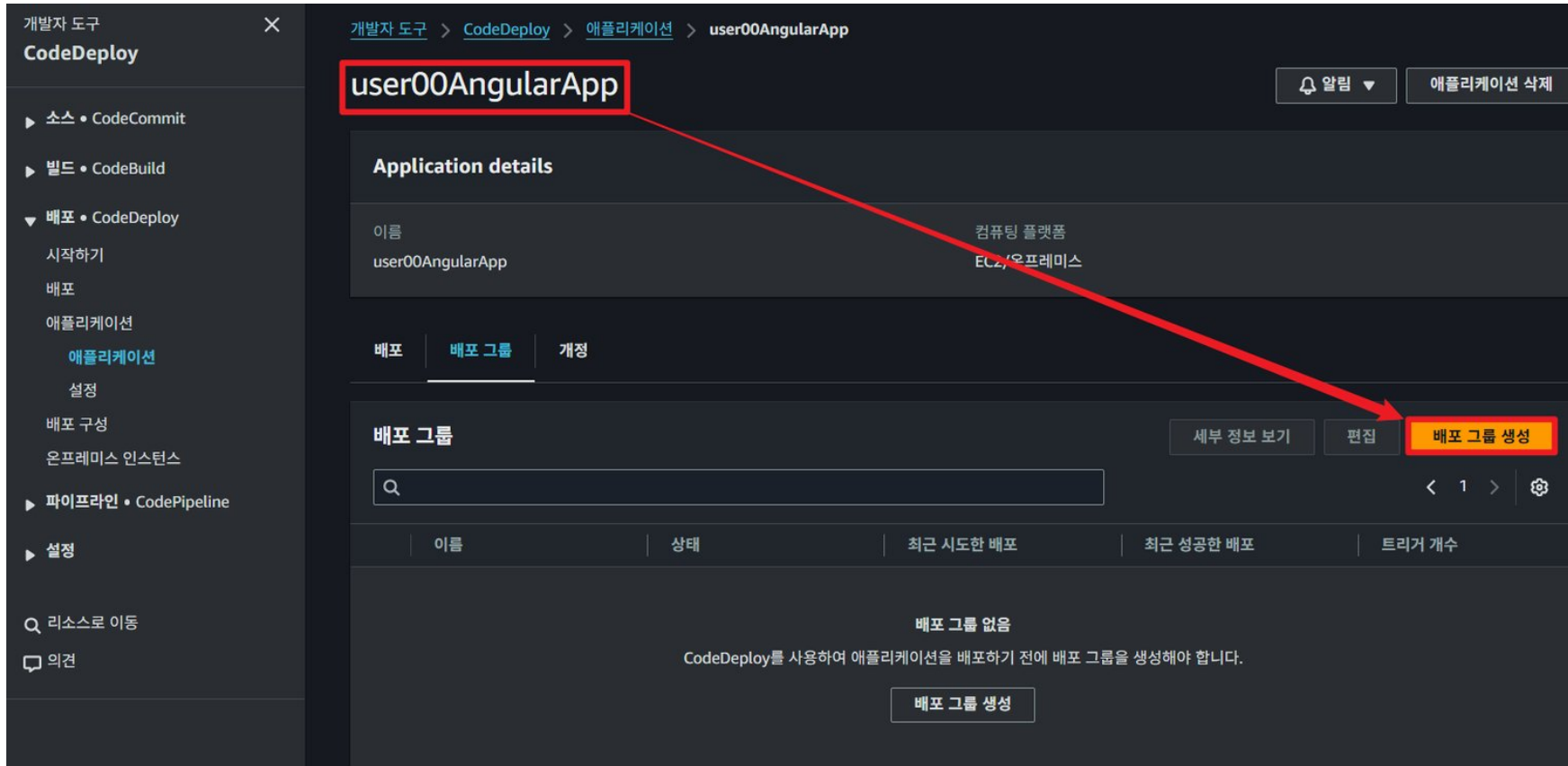
Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.

최대 문자 수: 1000. 문자(A~Z 및 a~z), 숫자(0~9), 탭, 새 줄 또는 다음 문자 중 하나를 사용합니다: _+=, @-/[0]!#\$%^*()~:"'`

CodeDeploy 배포 그룹 만들기

Step 3: 배포 그룹 생성

CodeDeploy에서 "배포 그룹 생성"을 클릭합니다.



개발자 도구

CodeDeploy

소스 • CodeCommit

빌드 • CodeBuild

▼ 배포 • CodeDeploy

시작하기

배포

애플리케이션

애플리케이션

설정

배포 구성

온프레미스 인스턴스

▶ 파이프라인 • CodePipeline

▶ 설정

리소스로 이동

의견

개발자 도구 > CodeDeploy > 애플리케이션 > user00AngularApp

user00AngularApp

알림 ▼

애플리케이션 삭제

Application details

이름

user00AngularApp

컴퓨팅 플랫폼

EC2/온프레미스

배포

배포 그룹

개정

배포 그룹

세부 정보 보기

편집

배포 그룹 생성

이름

상태

최근 시도한 배포

최근 성공한 배포

트리거 개수

배포 그룹 없음

CodeDeploy를 사용하여 애플리케이션을 배포하기 전에 배포 그룹을 생성해야 합니다.

배포 그룹 생성

CodeDeploy 배포 그룹 만들기

Step 3: 배포 그룹 생성

배포 그룹의 이름을 입력하고, 서비스 역할을 선택합니다.

애플리케이션

애플리케이션

user00AngularApp

컴퓨팅 유형

EC2/온프레미스

배포 그룹 이름

배포 그룹 이름 입력

User00TaggedEC2Instances

100자 제한

서비스 역할

서비스 역할 입력

AWS CodeDeploy가 대상 인스턴스에 액세스하도록 허용하는 CodeDeploy 권한이 있는 서비스 역할을 입력합니다.

Q

user00-CodeDeployEC2ServiceRole

arn:aws:iam::698047352521:role/user00-CodeDeployEC2ServiceRole

KT AIVLE School

240

CodeDeploy 배포 그룹 만들기

Step 3: 배포 그룹 생성

환경 구성에서 "Amazon EC2 인스턴스"를 체크하고, 키와 값을 아래와 같이 선택합니다.

환경 구성

이 배포에 추가할 Amazon EC2 Auto Scaling 그룹, Amazon EC2 인스턴스 및 온프레미스 인스턴스의 조합 선택

☐ Amazon EC2 Auto Scaling 그룹
 ☒ Amazon EC2 인스턴스

1개의 일치하는 고유한 인스턴스. 자세한 내용을 보려면 여기를 클릭하십시오. [\[?\]](#)

EC2 인스턴스의 태그 그룹을 세 개까지 이 배포 그룹에 추가할 수 있습니다.
 단일 태그 그룹: 해당 태그 그룹이 식별한 인스턴스가 배포됩니다.
 다중 태그 그룹: 모든 태그 그룹이 식별한 인스턴스만 배포됩니다.

태그 그룹 1

키

값 - 선택 사항

☐ 온프레미스 인스턴스

CodeDeploy 배포 그룹 만들기

Step 3: 배포 그룹 생성

로드 밸런서에서 "로드 밸런싱 활성화" 체크를 해제하고, 배포 그룹을 생성합니다.

로드 밸런서

배포 프로세스 중에 수신 트래픽을 관리할 로드 밸런서를 선택합니다. 로드 밸런서는 배포 중인 각 인스턴스에서 트래픽을 차단하고 배포 성공 후 인스턴스에 대한 트래픽을 다시 허용합니다.

☐ 로드 밸런싱 활성화

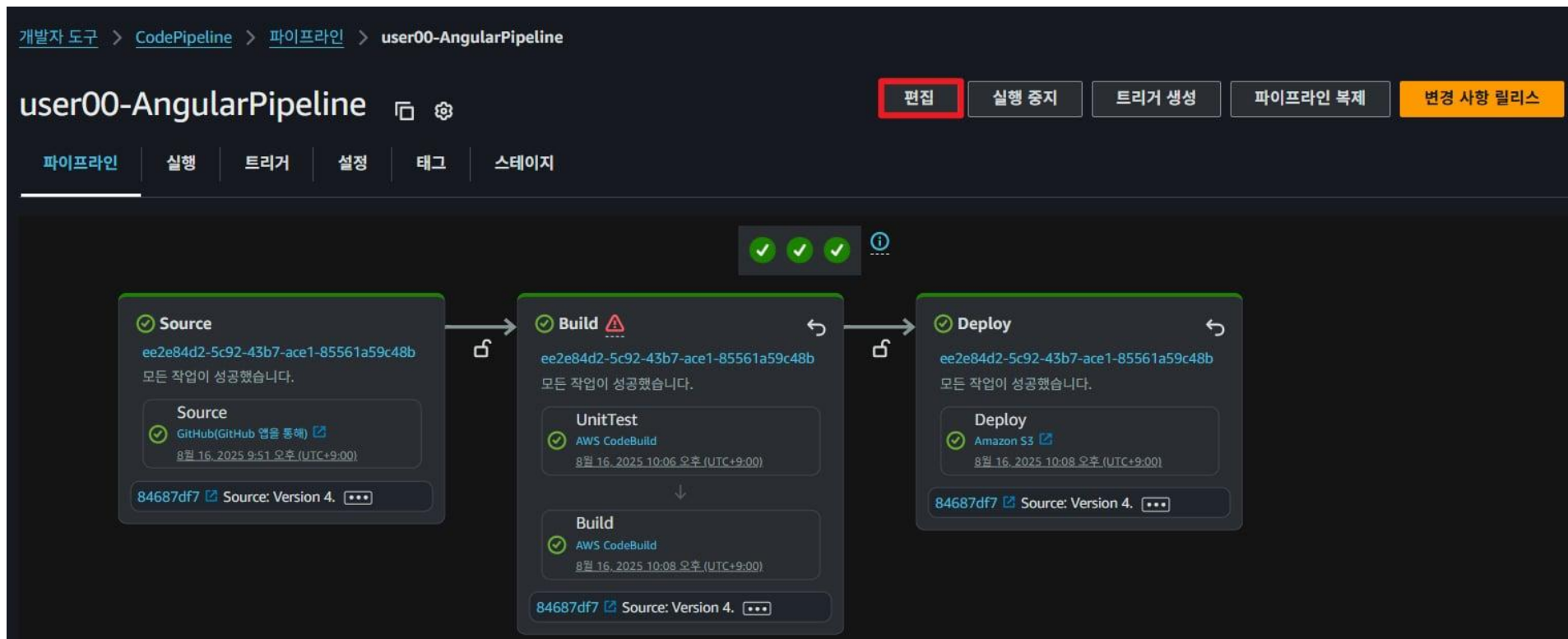
▶ 고급 - 선택 사항

[취소](#)[배포 그룹 생성](#)

Pipeline에 CodeDeploy 배포 작업 추가

Step 1: CodePipeline에서 Deploy 수정

CodePipeline에서 파이프라인을 선택한 후, "편집"을 클릭합니다.



개발자 도구 > CodePipeline > 파이프라인 > user00-AngularPipeline

user00-AngularPipeline  

편집 실행 중지 트리거 생성 파이프라인 복제 변경 사항 릴리스

파이프라인 실행 트리거 설정 태그 스테이지

Source Build Deploy

ee2e84d2-5c92-43b7-ace1-85561a59c48b ee2e84d2-5c92-43b7-ace1-85561a59c48b ee2e84d2-5c92-43b7-ace1-85561a59c48b

모든 작업이 성공했습니다. 모든 작업이 성공했습니다. 모든 작업이 성공했습니다.

Source Deploy

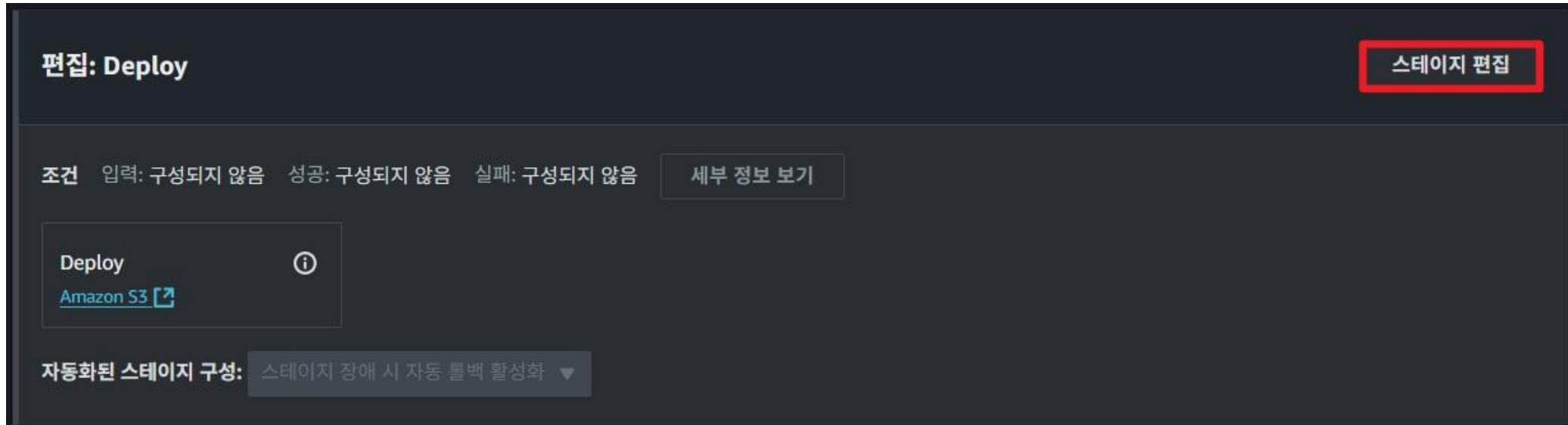
GitHub(GitHub 앱을 통해) Amazon S3

8월 16, 2025 9:51 오후 (UTC+9:00) 8월 16, 2025 10:06 오후 (UTC+9:00) 8월 16, 2025 10:08 오후 (UTC+9:00)

84687df7 Source: Version 4. 84687df7 Source: Version 4. 84687df7 Source: Version 4.

Pipeline에 CodeDeploy 배포 작업 추가

Step 1: CodePipeline에서 Deploy 수정
Deploy 단계에서 "스테이지편집"을 클릭합니다.



Pipeline에 CodeDeploy 배포 작업 추가

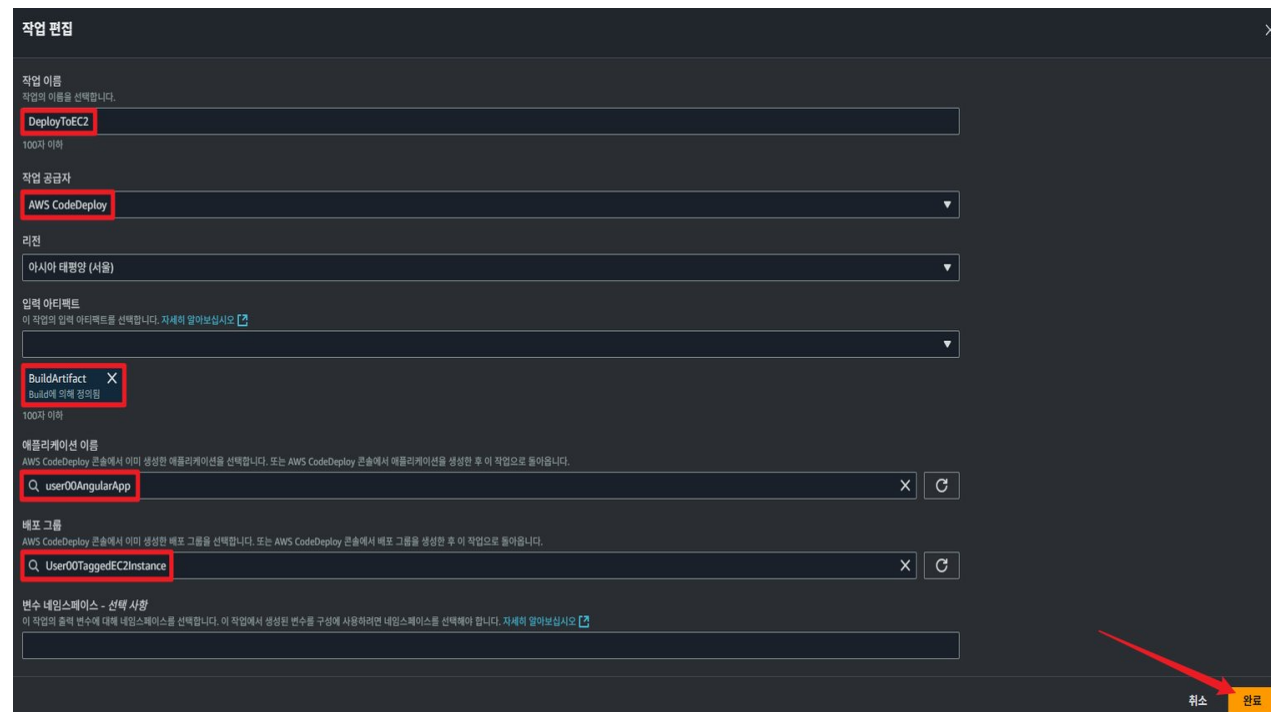
Step 1: CodePipeline에서 Deploy 수정

기존 Amazon S3를 제거하고, "작업 그룹 추가"를 클릭한 뒤,

작업 이름, 작업 공급자, 입력 아티팩트, 애플리케이션 이름, 배포 그룹을 선택합니다.

- 작업 이름: DeployToEC2
- 작업 공급자: AWS CodeDeploy
- 입력 아티팩트: BuildArtifact
- 애플리케이션 이름: user##AngularApp
- 배포 그룹: User##TaggedEC2Instance

"완료" 후 오른쪽 상단에 "저장"을 클릭합니다.



개발자 도구 > CodePipeline > 파이프라인 > user00-AngularPipeline > user00-AngularPipeline 편집

편집 중: user00-AngularPipeline

삭제

취소

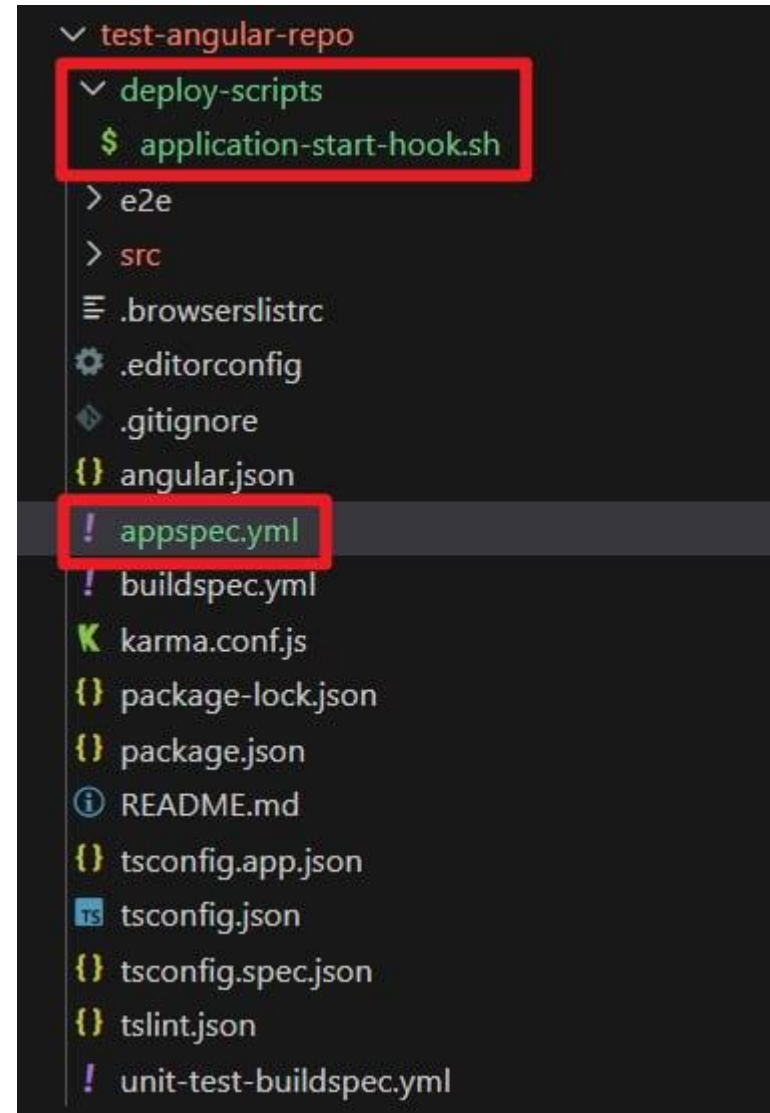
저장

Pipeline에 CodeDeploy 배포 작업 추가

Step 2: EC2 인스턴스 배포를 위한 AppSpec 파일 생성

Appspec은 CodeDeploy에서 배포를 관리하는데 사용하는 파일입니다. 배포가 정상적으로 될 수 있도록 Appspec 파일을 수정합니다.

제공 받은 파일(appspec.yml, deploy-scripts)을 VSCode에 업로드 합니다.



Pipeline에 CodeDeploy 배포 작업 추가

Step 2: EC2 인스턴스 배포를 위한 AppSpec 파일 생성

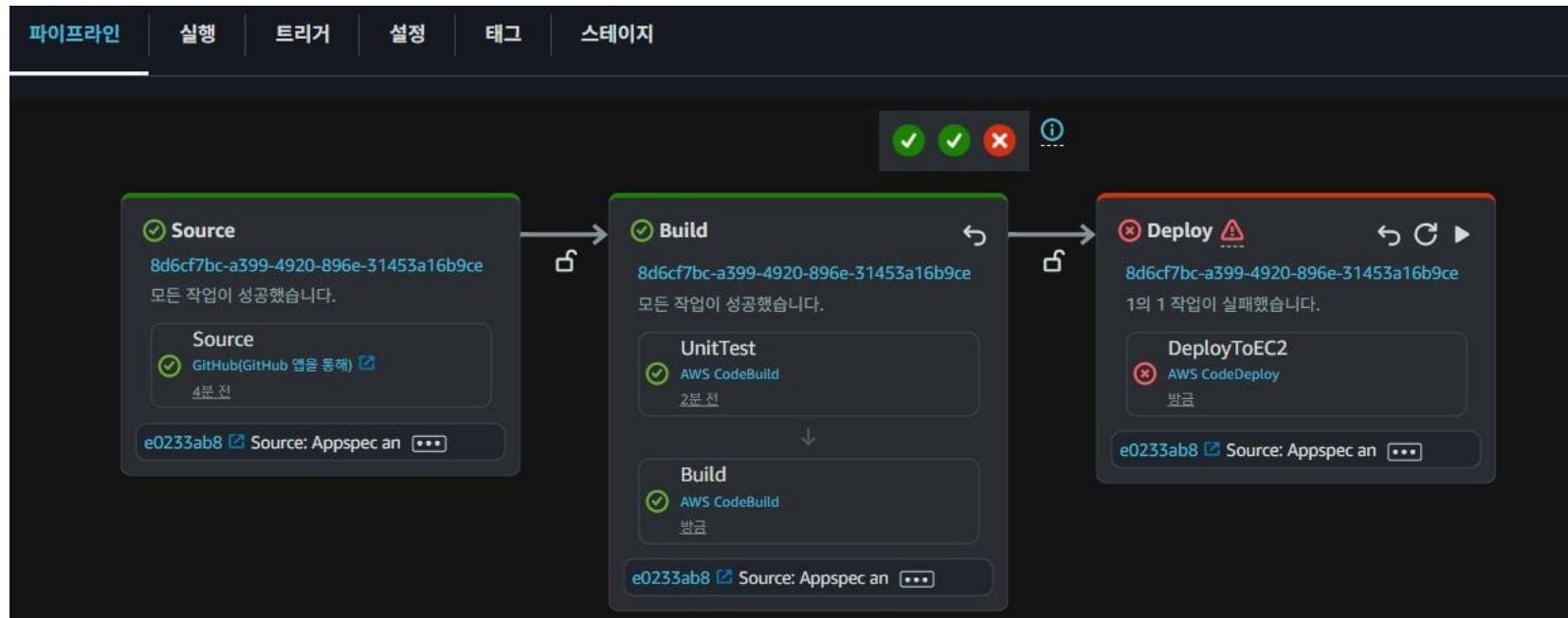
VSCode 터미널에서 GitHub에 push 합니다.

```
git add .  
git commit -m "Appspec and deploy scripts were added."  
git push origin main
```

```
PS C:\test-angular-repo> git push origin main  
Enumerating objects: 8, done.  
Counting objects: 100% (8/8), done.  
Delta compression using up to 18 threads  
Compressing objects: 100% (5/5), done.  
Writing objects: 100% (6/6), 808 bytes | 404.00 KiB/s, done.  
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.  
To https://github.com/[redacted]/test-angular-repo.git  
84687df..e0233ab main -> main
```

Pipeline에 CodeDeploy 배포 작업 추가

Step 2: EC2 인스턴스 배포를 위한 AppSpec 파일 생성
Pipeline에서 배포에 실패했음을 확인합니다.



Pipeline에 CodeDeploy 배포 작업 추가

Step 2: EC2 인스턴스 배포를 위한 AppSpec 파일 생성
CodeDeploy에서 appspec을 찾지 못해 발생하는 에러입니다.

배포 오류 메시지

❌ The overall deployment failed because too many individual instances failed deployment, too few healthy instances are available for deployment, or some instances in your deployment group are experiencing problems

buildspec.yml 파일을 수정합니다.

⚙ .editorconfig	17	artifacts:
🔍 .gitignore	18	files:
📄 angular.json	19	- 'dist/my-angular-project/**/*'
⚠ appspec.yml	20	- appspec.yml
⚠ buildspec.yml	21	- 'deploy-scripts/**/*'
📄 karma.conf.js		
📄 package-lock.json		

Pipeline에 CodeDeploy 배포 작업 추가

Step 2: EC2 인스턴스 배포를 위한 AppSpec 파일 생성

GitHub에 push 합니다.

```
git commit -a -m "aligned the build artifact"
git push origin main
```

Pipeline에서 모든 단계가 성공 되었음을 확인하고, 홈페이지 접속도 확인합니다.

